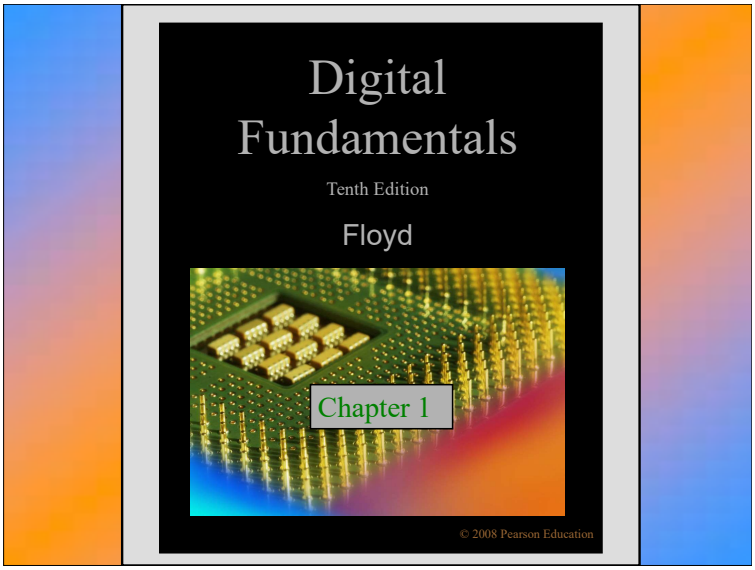


Digital and Analog: Basic Concepts

- **Continuous**: Without breaks, smooth, no interruptions; a range of values forming a line or curve without gaps or discontinuities
- **Discrete**: Broken down into pieces; opposite of continuous; a single part or point that can unambiguously be defined

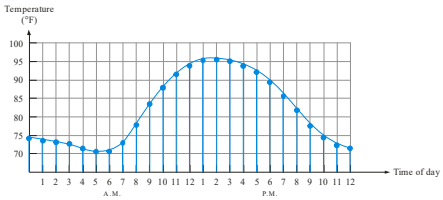
Digital and Analog: Basic Concepts

- **Analog**: An **analog** or **analogue signal** is any continuous signal for which the time varying feature (variable) of the signal is a representation of some other time varying quantity, i.e., analogous to another time varying signal . For instance, voltage changes may represent changes in temperature. An analog signal has a theoretically **infinite resolution**(Time & Amplitude)
- **Digital**: A **digital signal** is a physical signal that is a representation of a sequence of discrete values (a quantified discrete-time signal). Digital electronics usually involves binary or base-2 number systems.

Summary

Digital and Analog Quantities

Most natural quantities that we see are **analog** and vary continuously. Examples of analog quantities are temperature, time, pressure, distance, and sound



Digital systems can process, store, and transmit data more efficiently but can only assign discrete values to each point.

Summary

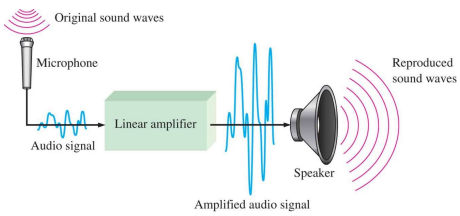
The Digital Advantage

- digital data can be **processed** and **transmitted** more efficiently and reliably than analog data.
- digital form can be **stored** more compactly and **reproduced** with greater accuracy and clarity than is possible when it is in analog form.
- **Noise** (unwanted voltage fluctuations) does not affect digital data nearly as much as it does analog signals

Summary

An Analog Systems

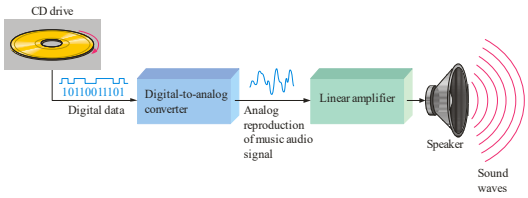
A public address system, used to amplify sound so that it can be heard by a large audience, is one simple example of an application of analog electronics



Summary

Analog and Digital Systems

Many systems use a mix of analog and digital electronics to take advantage of each technology. A typical CD player accepts digital data from the CD drive and converts it to an analog signal for amplification.

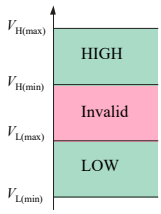


Summary

Binary Digits and Logic Levels

Digital electronics uses circuits that have two states, which are represented by two different voltage levels called HIGH and LOW. **The voltages represent numbers in the binary system.**

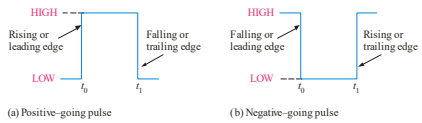
In binary, a single number is called a *bit* (for *b*inary dig*it*). A bit can have the value of either a 0 or a 1, depending on if the voltage is HIGH or LOW.



Summary

Digital Waveforms

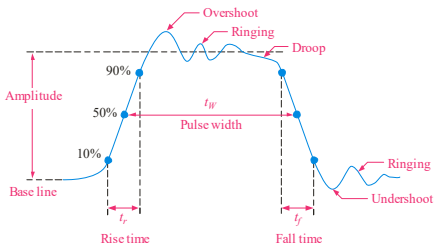
Digital waveforms change between the LOW and HIGH levels. A positive going pulse is one that goes from a normally LOW logic level to a HIGH level and then back again. Digital waveforms are made up of a series of pulses.



Summary

Pulse Definitions

Actual pulses are not ideal but are described by the rise time, fall time, amplitude, and other characteristics.



Summary

Periodic Pulse Waveforms

Periodic pulse waveforms are composed of pulses that repeats in a fixed interval called the **period**. The **frequency** is the rate it repeats and is measured in hertz.

$$f = \frac{1}{T} \qquad T = \frac{1}{f}$$

The **clock** is a basic timing signal that is an example of a periodic wave.

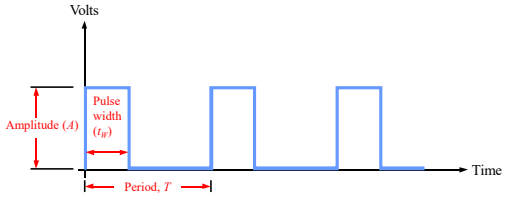
Example What is the period of a repetitive wave if $f = 3.2$ GHz?

Solution $T = \frac{1}{f} = \frac{1}{3.2\text{ GHz}} =$ 313 ps

Summary

Pulse Definitions

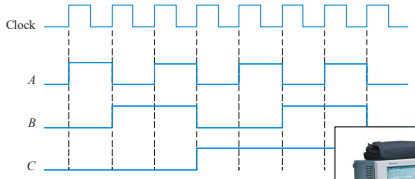
In addition to frequency and period, repetitive pulse waveforms are described by the amplitude (A), pulse width (t_W) and duty cycle. Duty cycle is the ratio of t_W to T.



Summary

Timing Diagrams

A timing diagram is used to show the **relationship** between two or more digital waveforms,

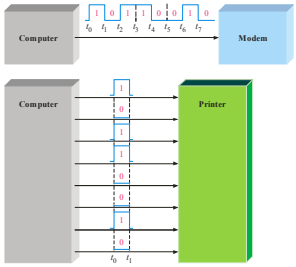


A diagram like this can be observed directly on a logic analyzer.

Summary

Data Transfer: Serial & Parallel

Data can be transmitted by either serial transfer(ex. usb) or parallel transfer.



Summary

Question

(a) Determine the total time required to serially transfer the eight bits contained in waveform *A* of Figure 1–14, and indicate the sequence of bits. The left-most bit is the first to be transferred. The 1 MHz clock is used as reference.
(b) What is the total time to transfer the same eight bits in parallel?

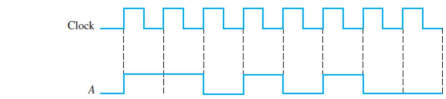


FIGURE 1–14

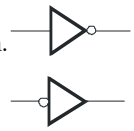
Summary

Basic Logic Functions

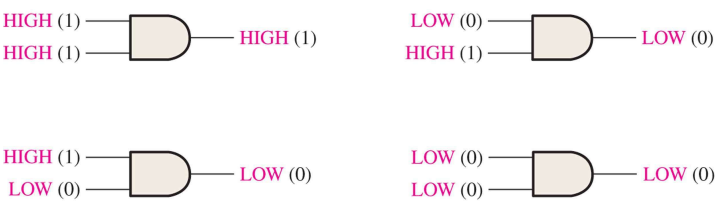
AND True only if ***all*** input conditions are true.

OR True only if ***one or more*** input conditions are true.

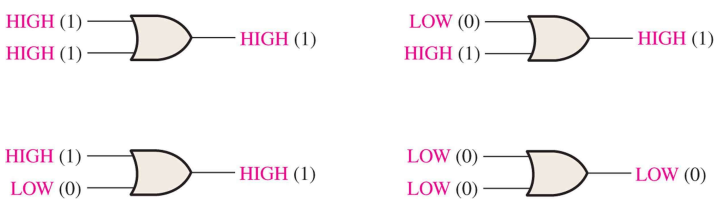
NOT Indicates the ***opposite*** condition.



And Gate



Or Gate



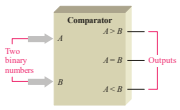
Not Gate

Summary

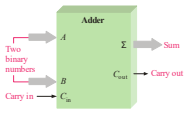
Basic System Functions

And, **or**, and **not** elements can be combined to form various logic functions. A few examples are:

The comparison function



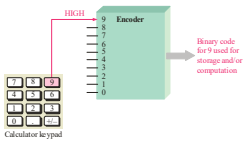
Basic arithmetic functions



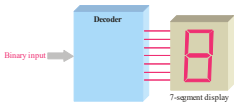
Summary

Basic System Functions

The encoding function



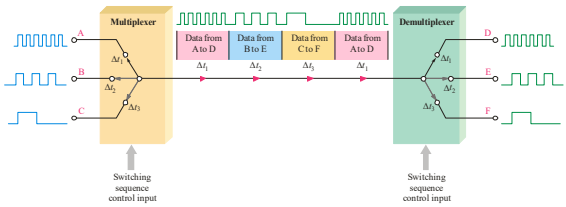
The decoding function



Summary

Basic System Functions

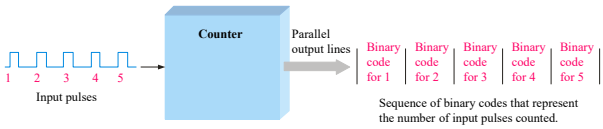
The data selection function



Summary

Basic System Functions

The counting function

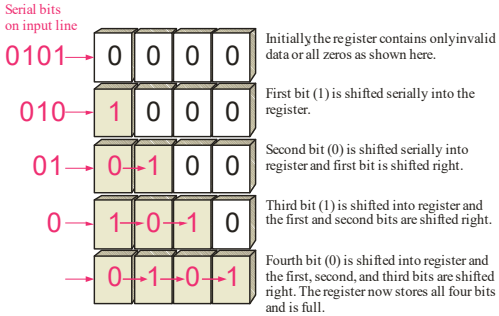


…and other functions such as code conversion and storage.

Summary

Basic System Functions

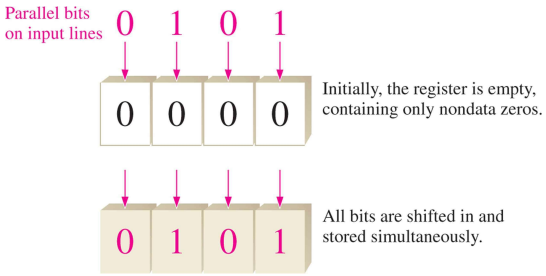
One type of storage function is the shift register, that moves and stores data each time it is clocked.



Summary

Basic System Functions

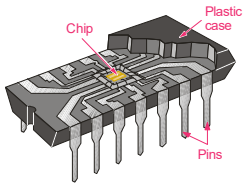
Bits are stored in a parallel register simultaneously from parallel lines



Summary

Integrated Circuits

Cutaway view of DIP (Dual-In-line Pins) chip:



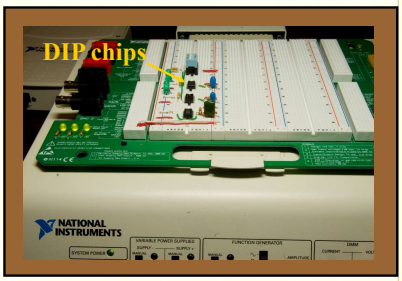
The TTL series, available as DIPs are popular for laboratory experiments with logic.

Summary

Integrated Circuits

An example of laboratory prototyping is shown. The circuit is wired using DIP chips and tested.

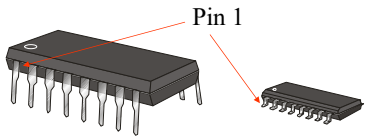
In this case, testing can be done by a computer connected to the system.



Summary

Integrated Circuits

DIP chips and surface mount chips

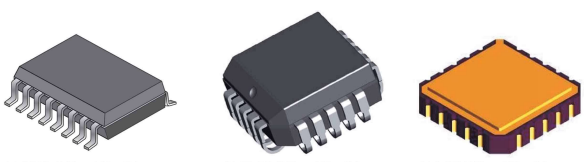


Dual in-line package Small outline IC (SOIC)

Summary

Integrated Circuits

Other surface mount packages:



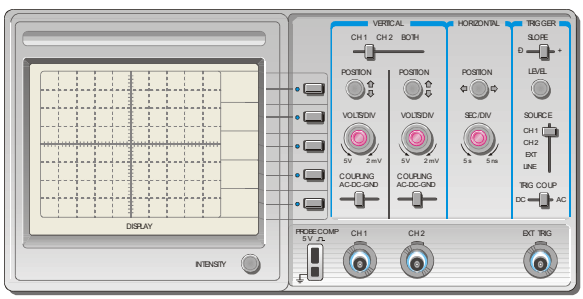
(a) SSOP (153 × 193 mils) (b) PLCC (350 × 350 mils) (c) LCC (350 × 350 mils)

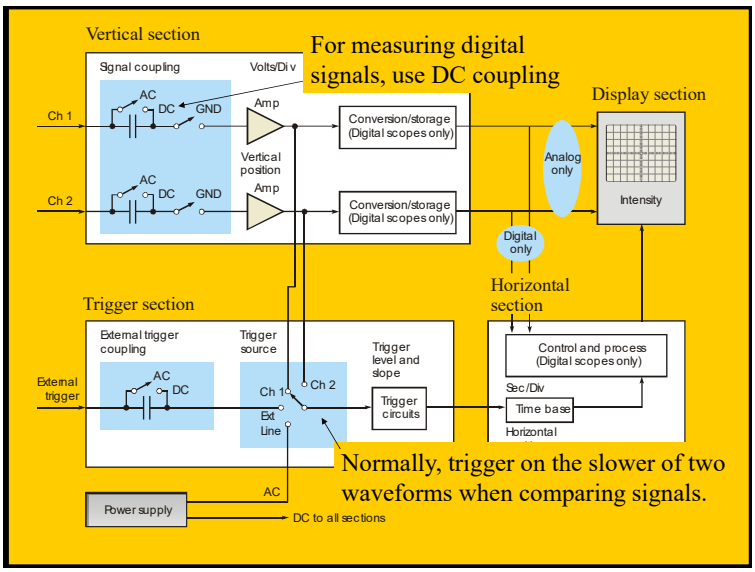
(d) LQFP (7 × 7 mm) (e) Laminate CSP (3.5 × 3.5 mm) (f) FBGA (4 × 4 mm)

Summary

Test and Measurement Instruments

The front panel controls for a general-purpose oscilloscope can be divided into four major groups.

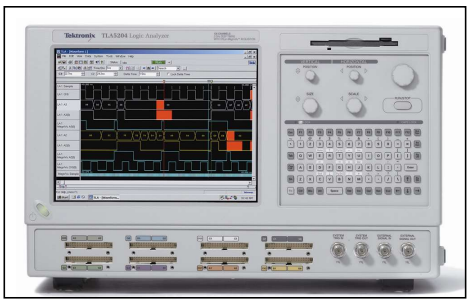




Summary

Test and Measurement Instruments

The logic analyzer can display multiple channels of digital information or show data in tabular form.

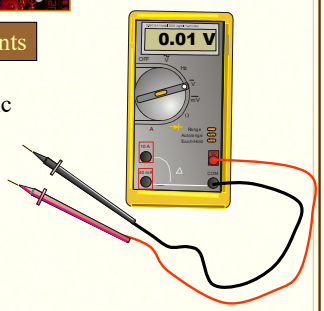


Summary

Test and Measurement Instruments

The DMM can make three basic electrical measurements.

- Voltage
- Resistance
- Current



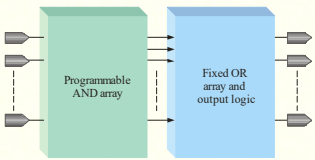
In digital work, DMMs are useful for checking power supply voltages, verifying resistors, testing continuity, and occasionally making other measurements.

Summary

Programmable Logic

Programmable logic devices (PLDs) are an alternative to fixed function devices. The logic can be programmed for a specific purpose. In general, they cost less and use less board space that fixed function devices.

A PAL device is a form of PLD that uses a combination of a programmable AND array and a fixed OR array:



Selected Key Terms

Analog Being continuous or having continuous values.

Digital Related to digits or discrete quantities; having a set of discrete values.

Binary Having two values or states; describes a number system that has a base of two and utilizes 1 and 0 as its digits.

Bit A binary digit, which can be a 1 or a 0.

Pulse A sudden change from one level to another, followed after a time, called the pulse width, by a sudden change back to the original level.

Selected Key Terms

Clock A basic timing signal in a digital system; a periodic waveform used to synchronize actions.

Gate A logic circuit that performs a basic logic operations such as AND or OR.

NOT A basic logic function that performs inversion.

AND A basic logic operation in which a true (HIGH) output occurs only when all input conditions are true (HIGH).

OR A basic logic operation in which a true (HIGH) output occurs when when one or more of the input conditions are true (HIGH).

Selected Key Terms

Fixed-function logic A category of digital integrated circuits having functions that cannot be altered.

Programmable logic A category of digital integrated circuits capable of being programmed to perform specified functions.

Quiz

1. Compared to analog systems, digital systems
   a. are less prone to noise
   b. can represent an infinite number of values
   c. can handle much higher power
   d. all of the above

© 2008 Pearson Education

Quiz

2. The number of values that can be assigned to a bit are
   a. one
   b. two
   c. three
   d. ten

© 2008 Pearson Education

Quiz

3. The time measurement between the 50% point on the leading edge of a pulse to the 50% point on the trailing edge of the pulse is called the
   a. rise time
   b. fall time
   c. period
   d. pulse width

© 2008 Pearson Education

Quiz

4. The time measurement between the 90% point on the trailing edge of a pulse to the 10% point on the trailing edge of the pulse is called the
   a. rise time
   b. fall time
   c. period
   d. pulse width

© 2008 Pearson Education

Quiz

5. The reciprocal of the frequency of a clock signal is the
   a. rise time
   b. fall time
   c. period
   d. pulse width

© 2008 Pearson Education

Quiz

6. If the period of a clock signal is 500 ps, the frequency is

a. 20 MHz

b. 200 MHz

c. 2 GHz

d. 20 GHz

© 2008 Pearson Education

Quiz

7. AND, OR, and NOT gates can be used to form

a. storage devices

b. comparators

c. data selectors

d. all of the above

© 2008 Pearson Education

Quiz

8. A shift register is an example of a

a. storage device

b. comparator

c. data selector

d. counter

© 2008 Pearson Education

Quiz

9. A device that is used to switch one of several input lines to a single output line is called a

a. comparator

b. decoder

c. counter

d. multiplexer

© 2008 Pearson Education

Quiz

10. For most digital work, an oscilloscope should be coupled to the signal using

a. ac coupling

b. dc coupling

c. GND coupling

d. none of the above

© 2008 Pearson Education

Quiz

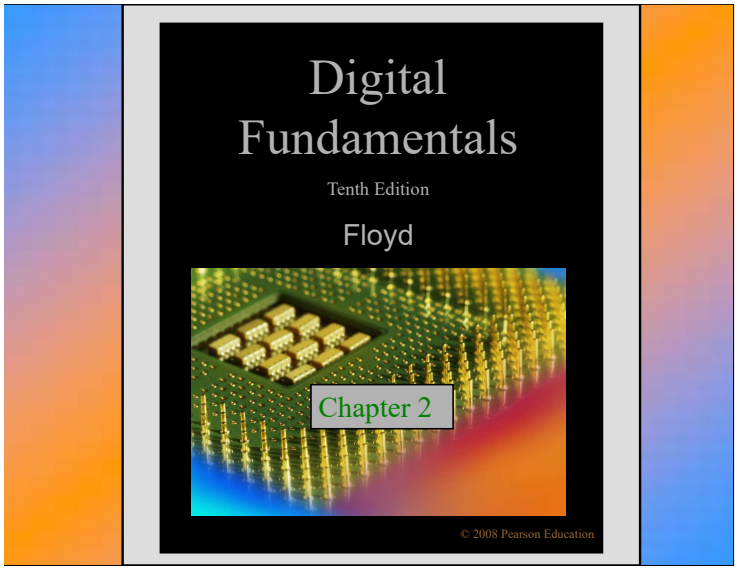
Answers:

1. a	6. c
2. b	7. d
3. d	8. a
4. b	9. d
5. c	10. b

Homework

Page 12

1,5,12,13



Summary

Decimal Numbers

The position of each digit in a **weighted number system** is assigned a weight based on the **base** or **radix** of the system. The radix of decimal numbers is ten, because only ten symbols (0 through 9) are used to represent any number.

The column weights of decimal numbers are powers of ten that increase from right to left beginning with $10^0 = 1$:

$\ldots 10^5\ 10^4\ 10^3\ 10^2\ 10^1\ 10^0$.

For fractional decimal numbers, the column weights are negative powers of ten that decrease from left to right:

$10^2\ 10^1\ 10^0.\ 10^{-1}\ 10^{-2}\ 10^{-3}\ 10^{-4}\ \ldots$

Summary

Decimal Numbers

Decimal numbers can be expressed as the sum of the products of each digit times the column value for that digit. Thus, the number 9240 can be expressed as

$$(9 \times 10^3) + (2 \times 10^2) + (4 \times 10^1) + (0 \times 10^0)$$

or

$$9 \times 1{,}000 + 2 \times 100 + 4 \times 10 + 0 \times 1$$

Example Express the number 480.52 as the sum of values of each digit.

Solution

$480.52 = (4 \times 10^2) + (8 \times 10^1) + (0 \times 10^0) + (5 \times 10^{-1}) + (2 \times 10^{-2})$

Summary

Counting in Binary

For digital systems, the binary number system is used. Binary has a radix of two and uses the digits 0 and 1 to represent quantities.

In general, with n bits you can count up to a number equal to $2^n - 1$.

TABLE 2–1

Decimal Number	Binary Number
0	0 0 0 0
1	0 0 0 1
2	0 0 1 0
3	0 0 1 1
4	0 1 0 0
5	0 1 0 1
6	0 1 1 0
7	0 1 1 1
8	1 0 0 0
9	1 0 0 1
10	1 0 1 0
11	1 0 1 1
12	1 1 0 0
13	1 1 0 1
14	1 1 1 0
15	1 1 1 1

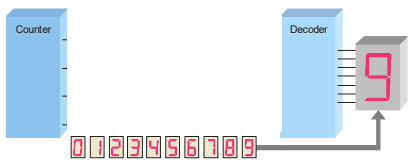
Summary

Binary Numbers

A binary counting sequence for numbers from zero to fifteen is shown.

Notice the pattern of zeros and ones in each column.

Digital counters frequently have this same pattern of digits:



Decimal Number	Binary Number
0	0 0 0 0
1	0 0 0 1
2	0 0 1 0
3	0 0 1 1
4	0 1 0 0
5	0 1 0 1
6	0 1 1 0
7	0 1 1 1
8	1 0 0 0
9	1 0 0 1
10	1 0 1 0
11	1 0 1 1
12	1 1 0 0
13	1 1 0 1
14	1 1 1 0
15	1 1 1 1

Summary

Binary Numbers

The column weights of binary numbers are powers of two that increase from right to left beginning with $2^0 = 1$:

$$\ldots 2^5\ 2^4\ 2^3\ 2^2\ 2^1\ 2^0.$$

For fractional binary numbers, the column weights are negative powers of two that decrease from left to right:

$$2^2\ 2^1\ 2^0.\ 2^{-1}\ 2^{-2}\ 2^{-3}\ 2^{-4}\ \ldots$$

The weight structure of a binary number is

$$2^{n-1} \ldots 2^3\ 2^2\ 2^1\ 2^0\ .\ 2^{-1}\ 2^{-2} \ldots 2^{-n}$$

Binary point

Summary

Binary Conversions

The decimal equivalent of a binary number can be determined by adding the column values of all of the bits that are 1 and discarding all of the bits that are 0.

Example Convert the binary number 100101.01 to decimal.

Solution Start by writing the column weights; then add the weights that correspond to each 1 in the number.

2^5	2^4	2^3	2^2	2^1	2^0.	2^{-1}	2^{-2}	
32	16	8	4	2	1 .	½	¼	
1	0	0	1	0	1.	0	1	
32			+4		+1		+¼ =	37¼

Summary

Binary Conversions

You can convert a decimal whole number to binary by reversing the procedure. Write the decimal weight of each column and place 1's in the columns that sum to the decimal number.

Example Convert the decimal number 49 to binary.

Solution The column weights double in each position to the right. Write down column weights until the last number is larger than the one you want to convert.

2^6	2^5	2^4	2^3	2^2	2^1	2^0.
64	32	16	8	4	2	1.
0	1	1	0	0	0	1.

Summary

Binary Conversions

Example Convert the following decimal numbers to binary:

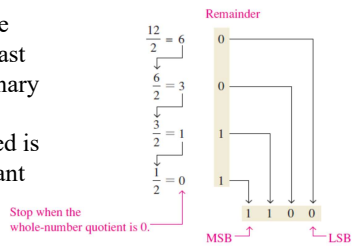
(a) 12 **(b)** 25
(c) 58 **(d)** 82

Summary

Binary Conversions

A systematic method of converting whole numbers from decimal to binary is the *repeated division-by-2* process. You can convert decimal to any other base by repeatedly dividing by the base.

The first remainder to be produced is the LSB (least significant bit) in the binary number, and the last remainder to be produced is the MSB (most significant bit).



Summary

Binary Conversions

Example Convert the following decimal numbers to binary:

(a) 19 **(b)** 45

Summary

Binary Conversions

You can convert a decimal fraction to binary by repeatedly multiplying the fractional results of successive multiplications by 2. The carries form the binary number.

Example Convert the decimal fraction 0.188 to binary by repeatedly multiplying the fractional results by 2.

Solution

0.188 x 2 = 0.376 carry = 0
0.376 x 2 = 0.752 carry = 0
0.752 x 2 = 1.504 carry = 1
0.504 x 2 = 1.008 carry = 1
0.008 x 2 = 0.016 carry = 0

MSB

Answer = .00110 (for five significant digits)

Summary

Binary Conversions

Example

1. Convert each decimal number to binary by using the sum-of-weights method:
(a) 23 **(b)** 57 **(c)** 45.5
2. Convert each decimal number to binary by using the repeated division-by-2 method(repeated multiplication-by-2 for fractions):
(a) 14 **(b)** 21 **(c)** 0.375

Octal Numbers--Base 8

- "base" 8
- with 8 digits: 0..7

Eg.

$231.25_8 = 2\times8^2 + 3\times8^1 + 1\times8^0 + 2\times8^{-1} + 5\times8^{-2}$

(in decimal) $= 153.328125_{10}$

Hexadecimal Numbers--Base 16

- $r = 16$
- Digits (convention): 0..9, A, B, C, D, E, F
- A=10, B=11, … , F = 15

Hexadecimal	Decimal
A	10
B	11
C	12
D	13
E	14
F	15

Floyd, Digital Fundamentals, 10th ed © 2009 Pearson Education, Upper Saddle River, NJ 07458. All Rights Reserved

e.g1.
$(3FB)_{16} = 3\times16^2 + 15\times16^1 + \mathbf{11}\times16^0$
(in decimal) = 768 + 240 + 11
= (1019)10

e.g2.

$\mathbf{A59C.3A_{16} = (A\times16^3)+(5\times16^2)+(9\times16^1)+(C\times16^0)+(3\times16^{-1})+(A\times16^{-2})}$

Binary ↔ Octal

$$(011\,|\,010\,|\,101\,|\,000\,.\,|\,111\,|\,101\,|\,011\,|\,100)_2$$
$$(\,3\quad 2\quad 5\quad 0\,.\,7\quad 5\quad 3\quad 4\,)_8$$

E.g. Convert the binary number

010011110111.110101010_2 to octal

Solution $010,011,110,111.110,101,010_2 = 2367.652_8$

Binary ↔ Hex

$$(0110|1010|1000|.|1111|0101|1100)_2$$

$$(6 \quad A \quad 8 \; . \; F \quad 5 \quad C)_{16}$$

E.g. Convert the binary number

010011110111.110101010_2 to hexadecimal

Solution $0100,1111,0111.1101,0101,0000_2=4F7.D50_{16}$

Number System Conversion

◆ **hexadecimal and octal to Binary Conversion**

E.g. Convert the numbers $F37A.B2_{16}$,735.5_8 to binary

Solution $F37A.B2=1111,0011,0111,1010.1011,0010_2$

$735.5_8=111,011,101.101_2$

Practice problem: convert 367.236_8 to binary then hexadecimal

Solution $367.236_8=011,110,111.010,011,110_2$

$011,110,111.010,011,110_2=0,1111,0111.0100,1111,0_2=F7.4F_{16}$

Number System Conversion

◆**Any Radix to Decimal**

e.g. Convert 324.2_5 to decimal

Solution $3\cdot5^2+2\cdot5^1+4\cdot5^0+2\cdot5^{-1}$

$=3(25)+2(5)+4(1)+2(1/5)$

$=75+10+4+2/5$

$=89.4_{10}$

Number System Conversion

Practice problem:

1. Convert 345.2_6 to decimal
2. Convert 34.8_{10} to base 5
3. Convert 47.5_{10} to hexadecimal

Solution:

1. **$137.333\ldots\ldots_{10}$**
2. **114.4_5**
3. **$2F.8_{16}$**

Summary

Binary Addition

The rules for binary addition are

0 + 0 = 0	Sum = 0, carry = 0
0 + 1 = 0	Sum = 1, carry = 0
1 + 0 = 0	Sum = 1, carry = 0
1 + 1 = 10	Sum = 0, carry = 1

When an input carry = 1 due to a previous result, the rules are

1 + 0 + 0 = 01	Sum = 1, carry = 0
1 + 0 + 1 = 10	Sum = 0, carry = 1
1 + 1 + 0 = 10	Sum = 0, carry = 1
1 + 1 + 1 = 10	Sum = 1, carry = 1

Summary

Binary Addition

Example Add the binary numbers 00111 and 10101 and show the equivalent decimal addition.

Solution

$$\begin{array}{r} {\scriptstyle 0\,1\,1\,1} \\ 00111 \\ 10101 \\ \hline 11100 \end{array} \quad \begin{array}{r} 7 \\ 21 \\ \hline =28 \end{array}$$

Summary

Binary Subtraction

The rules for binary subtraction are

$0 - 0 = 0$
$1 - 1 = 0$
$1 - 0 = 1$
$10 - 1 = 1$ with a borrow of 1

Example Subtract the binary number 00111 from 10101 and show the equivalent decimal subtraction.

Solution

```
  1 1 1
  10101    21
  00111     7
  01110  = 14
```

Binary Codes

◆ Signed Number Binary

Binary signed magnitude number convention uses the most significant bit position to indicate sign and the remaining lesser significant bits to represent magnitude

- ✓**Sign magnitude**
- ✓**2s complement**
- ✓**1s complement**

Sign magnitude and complement

Decimal	Sign magnitude	2s complement	1s complement
+15	01111	01111	01111
+14	01110	01110	01110
+13	01101	01101	01101
+12	01100	01100	01100
+11	01011	01011	01011
+10	01010	01010	01010
+9	01001	01001	01001
+8	01000	01000	01000
+7	00111	00111	00111
+6	00110	00110	00110
+5	00101	00101	00101
+4	00100	00100	00100
+3	00011	00011	00011
+2	00010	00010	00010
+1	00001	00001	00001
+0	00000	00000	00000
−0	10000	00000	11111
−1	10001	11111	11110
−2	10010	11110	11101
−3	10011	11101	11100
−4	10100	11100	11011
−5	10101	11011	11010
−6	10110	11010	11001
−7	10111	11001	11000
−8	11000	11000	10111
−9	11001	10111	10110
−10	11010	10110	10101
−11	11011	10101	10100
−12	11100	10100	10011
−13	11101	10011	10010
−14	11110	10010	10001
−15	11111	10001	10000
−16		10000	

Signed Number Representation

- Signed Magnitude Method

  $N = (sa_{n-1} \ldots a_0.a_{-1} \ldots a_{-m})_{rsm}$,

  where s = 0 if N is positive and s = r -1 otherwise.

  – **E.g.** $N = -(15)_{10}$

  In binary: $N = -(15)_{10} = -(1111)_2 = (1, 1111)_{2sm}$

  In decimal: $N = -(15)_{10} = (9, 15)_{10sm}$

- Complementary Number(补数) Systems

  – **Radix complements** (r's complements)

  $[N]_r = r^n - (N)_r$

  where n is the number of digits in $(N)_r$.

  – **Diminished radix complements** (r-1's complements)

  $[N]_{r-1} = r^n - (N)_r - 1$

Radix Complement Number Systems

◆ **2's complement of $(N)_2 = (101001)_2$**

$[N]_2 = 2^6 - (101001)_2 = (1000000)_2 - (101001)_2 = (010111)_2$

$(N)_2 + [N]_2 = (101001)_2 + (010111)_2 = (1000000)_2$

If we discard the carry, $(N)_2 + [N]_2 = 0$.

Hence, $[N]_2$ can be used to represent $-(N)_2$.

$[\,[N]_2\,]_2 = [(010111)_2]_2 = (1000000)_2 - (010111)_2 = (101001)_2 = (N)_2$.

◆ **2's complement of $(N)_2 = (1010)_2$ for n = 6**

$[N]_2 = (1000000)_2 - (1010)_2 = (110110)_2$.

◆ **10's complement of $(N)_{10} = (72092)_{10}$**

$[N]_{10} = (100000)_{10} - (72092)_{10} = (27908)_{10}$.

Radix Complement Number Systems

- Algorithm： Find $[N]_r$ given $(N)_r$.

  1.Copy the digits of N, beginning with the LSB and proceeding toward the MSB until the first nonzero digit, a_j, has been reached

  2.Replace a_j with $r - a_j$.

  3.Replace each remaining digit a_j , of N by $(r - 1) - a_j$ until the MSB has been replaced.

  – **Example**: 10's complement of $(56700)_{10}$ is $(43300)_{10}$

  – **Example**: 2's complement of $(10100)_2$ is $(01100)_2$.

  – **Example**: 2's complement of $N = (10110)_2$ for n = 8.

  - Put three zeros in the MSB position and apply algorithm
  - N = 00010110
  - $[N]_2 = (11101010)_2$

Radix Complement Number Systems

- Algorithm2： Find $[N]_r$ given $(N)_r$.

  First replace each digit, a_k , of $(N)_r$ by $(r - 1) - a_k$ and then add 1 to the resultant.

- **Example**: Find 2's complement of $N = (01100101)_2$.

  N = 01100101
  10011010 Complement the bits
  +1 Add 1

  $[N]_2 = (10011011)_2$

- **Example**: Find 10's complement of $N = (40960)_{10}$

  N = 40960
  59039 Complement the bits
  +1 Add 1

  $[N]_2 = (59040)_{10}$

Radix Complement Number Systems

- **2's complement code system**
  - Positive number :
    - $N = +(a_{n-2}, \ldots, a_0)_2 = (0, a_{n-2}, \ldots, a_0)_{2cns}$, where $0 \le N \le 2^{n-1}-1$.
  - Negative number:
    - $N = (a_{n-1}, a_{n-2}, \ldots, a_0)_2$
    - $-N = [a_{n-1}, a_{n-2}, \ldots, a_0]_2$ (two's complement of N), where $-1 \ge N \ge -2^{n-1}$.
  - ***Example***: Two's complement code system representation of $\pm(N)_2$ when $(N)_2 = (1011001)_2$ for $n = 8$:
    - $+(N)_2 = (0, 1011001)_{2cns}$
    - $-(N)_2 = [+(N)_2]_2 = [0, 1011001]_2 = (1, 0100111)_{2cns}$

Diminished Radix Complement Number systems

- **Diminished radix complement** $[N]_{r-1}$ of a number $(N)_r$ is:
  $[N]_{r-1} = r^n - (N)_r - 1$
- **One's complement** (r = 2):
  $[N]_{2-1} = 2^n - (N)_2 - 1$
- **Example**: One's complement of $(01100101)_2$
  $[N]_{2-1} = 2^8 - (01100101)_2 - 1$
  $= (100000000)_2 - (01100101)_2 - (00000001)_2$
  $= (10011011)_2 - (00000001)_2$
  $= (10011010)_2$

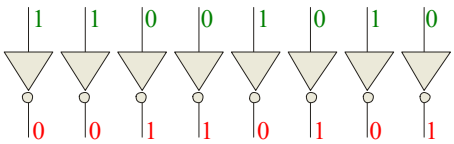
Summary

1's Complement

The 1's complement of a binary number is just the inverse of the digits. To form the 1's complement, change all 0's to 1's and all 1's to 0's.

For example, the 1's complement of 11001010 is 00110101

In digital circuits, the 1's complement is formed by using inverters:



Summary

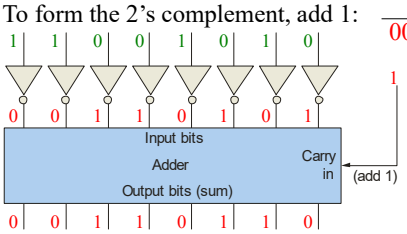
2's Complement

The 2's complement of a binary number is found by adding 1 to the LSB of the 1's complement.

Recall that the 1's complement of 11001010 is 00110101 (1's complement)

To form the 2's complement, add 1: $+1$

00110110 (2's complement)



Radix Complement Number Systems

Quzi:

- ◆**Find the 2s complement of the binary number 110011**
- ◆**Find the 1s complement code for -12_{10}**
- ◆**Find the 2s complement code for -18_{10}**

◆**Answer: 001101**

10011

101110

Summary

Signed Binary Numbers

An easy way to read a signed number that uses this notation is to assign the sign bit a column weight of –128 (for an 8-bit number). Then add the column weights for the 1's.

Example Assuming that the sign bit = –128, show that 11000110 = –58 as a 2's complement signed number:

Solution Column weights: –128 64 32 16 8 4 2 1.

1 1 0 0 0 1 1 0

–128 +64 +4 +2 = –58

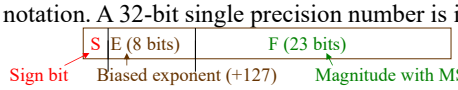
For 2's complement signed numbers, the range of values for n-bit numbers is

Range = $-(2^{n-1})$ to $+(2^{n-1} - 1)$

Summary

Floating Point Numbers

Floating point notation is capable of representing very large or small numbers by using a form of scientific notation. A 32-bit single precision number is illustrated.



Example Express the speed of light, c, in single precision floating point notation. ($c = 0.2998 \times 10^9$)

Solution In binary, $c = 0001\ 0001\ 1101\ 1110\ 1001\ 0101\ 1100\ 0000_2$.

In scientific notation, $c = 1.001\ 1101\ 1110\ 1001\ 0101\ 1100\ 0000 \times 2^{28}$.

S = 0 because the number is positive. $E = 28 + 127 = 155_{10} = 1001\ 1011_2$.

F is the next 23 bits after the first 1 is dropped.

In floating point notation, $c =$

0	10011011	001 1101 1110 1001 0101 1100

Summary

Arithmetic Operations with Signed Numbers

Using the signed number notation with negative numbers in 2's complement form simplifies addition and subtraction of signed numbers.

Rules for **addition**: Add the two signed numbers. Discard any final carries. The result is in signed form.

Examples:

00011110 = +30	00001110 = +14	11111111 = –1
00001111 = +15	11101111 = –17	11111000 = –8
00101101 = +45	11111101 = –3	11110111 = –9

Discard carry

Summary

Arithmetic Operations with Signed Numbers

Note that if the number of bits required for the answer is exceeded, overflow will occur. This occurs only if both numbers have the same sign. If the sign bit of the result is different than the sign bit of the numbers that are added, overflow is indicated

Two examples are:

01000000 = +128	10000001 = –127
01000001 = +129	10000001 = –127
10000001 = –126	00000010 = +2

Discard carry

Wrong! The answer is incorrect and the sign bit has changed.

Summary

Arithmetic Operations with Signed Numbers

Rules for **subtraction**: 2's complement the subtrahend and add the numbers. Discard any final carries. The result is in signed form.

Repeat the examples done previously, but subtract:

00011110 (+30)	00001110 (+14)	11111111 (–1)
– 00001111 –(+15)	– 11101111 –(–17)	– 11111000 –(–8)

2's complement subtrahend and add:

00011110 = +30	00001110 = +14	11111111 = –1
11110001 = –15	00010001 = +17	00001000 = +8
00001111 = +15	00011111 = +31	00000111 = +7

Discard carry

Discard carry

Arithmetic Using Complement Codes

- Exe：perform the following operations by finding the radix complement of the subtrahend(减数) and adding the result to the minuend(被减数):

23.4_{10}	135.7_8	321.2_4
$- 19.8_{10}$	$- 67.7_8$	$- 33.3_4$

Summary

Arithmetic Operations with Signed Numbers

Rules for **Multiplication**:

Multiply the signed binary numbers: 01001101 (multiplicand) and 00000100 (multiplier) using the direct addition method.

Solution

Since both numbers are positive, they are in true form, and the product will be positive. The decimal value of the multiplier is 4, so the multiplicand is added to itself four times as follows:

Summary

Arithmetic Operations with Signed Numbers

01001101 1st time
+ 01001101 2nd time
10011010 Partial sum
+ 01001101 3rd time
11100111 Partial sum
+ 01001101 4th time
100110100 Product

Since the sign bit of the multiplicand is 0, it has no effect on the outcome. All of the bits in the product are magnitude bits.

Related Problem

Multiply 01100001 by 00000110 using the direct addition method.

Summary

Arithmetic Operations with Signed Numbers

The basic steps in the partial products method of binary multiplication are as follows:

Step 1: Determine if the signs of the multiplicand and multiplier are the same or different. This determines what the sign of the product will be.

Step 2: Change any negative number to true (uncomplemented) form. Because most computers store negative numbers in 2's complement, a 2's complement operation is required to get the negative number into true form.

Summary

Arithmetic Operations with Signed Numbers

Step 3: Starting with the least significant multiplier bit, generate the partial products. When the multiplier bit is 1, the partial product is the same as the multiplicand. When the multiplier bit is 0, the partial product is zero. Shift each successive partial product one bit to the left.

Step 4: Add each successive partial product to the sum of the previous partial products to get the final product.

Step 5: If the sign bit that was determined in step 1 is negative, take the 2's complement of the product. If positive, leave the product in true form. Attach the sign bit to the product.

Example: Multiply the signed binary numbers: 01010011 (multiplicand) and 11000101 (multiplier).

Summary

Arithmetic Operations with Signed Numbers

Rules for **Division**:

Step 1: Determine if the signs of the dividend and divisor are the same or different. This determines what the sign of the quotient will be. Initialize the quotient to zero.

Step 2: Subtract the divisor from the dividend using 2's complement addition to get the first partial remainder and add 1 to the quotient. If this partial remainder is positive, go to step 3. If the partial remainder is zero or negative, the division is complete.

Step 3: Subtract the divisor from the partial remainder and add 1 to the quotient. If the result is positive, repeat for the next partial remainder. If the result is zero or negative, the division is complete.

Example: Divide 01100100 by 00011001.

Summary

BCD

Binary coded decimal (BCD) is a weighted code that is commonly used in digital systems when it is necessary to show decimal numbers such as in clock displays.

The table illustrates the difference between straight binary and BCD. BCD represents each decimal digit with a 4-bit code. Notice that the codes 1010 through 1111 are not used in BCD.

Decimal	Binary	BCD
0	0000	0000
1	0001	0001
2	0010	0010
3	0011	0011
4	0100	0100
5	0101	0101
6	0110	0110
7	0111	0111
8	1000	1000
9	1001	1001
10	1010	0001 0000
11	1011	0001 0001
12	1100	0001 0010
13	1101	0001 0011
14	1110	0001 0100
15	1111	0001 0101

Summary

BCD

You can think of BCD in terms of column weights in groups of four bits. For an 8-bit BCD number, the column weights are: 80 40 20 10 8 4 2 1.

Question: What are the column weights for the BCD number 1000 0011 0101 1001?

Answer:

8000 4000 2000 1000 800 400 200 100 80 40 20 10 8 4 2 1

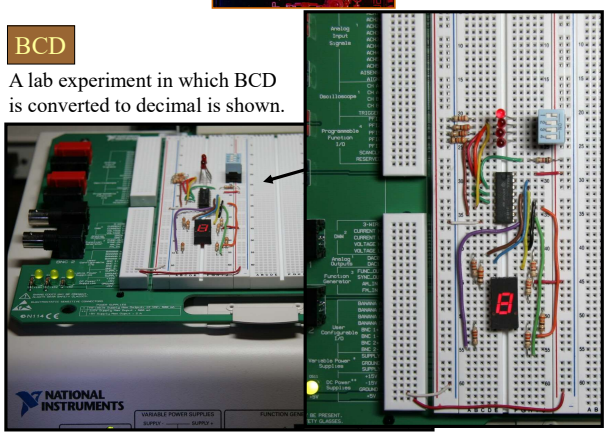
Note that you could add the column weights where there is a 1 to obtain the decimal number. For this case:

$8000 + 200 + 100 + 40 + 10 + 8 + 1 = 8359_{10}$

Summary

BCD

A lab experiment in which BCD is converted to decimal is shown.



Summary

Gray code

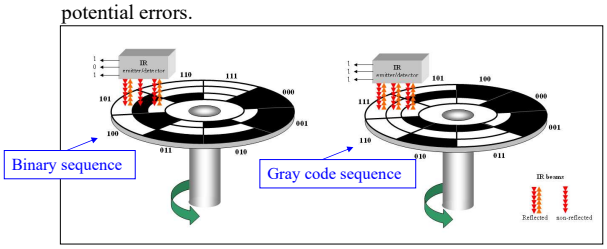
Gray code is an unweighted code that has a single bit change between one code word and the next in a sequence. Gray code is used to avoid problems in systems where an error can occur if more than one bit changes at a time.

Decimal	Binary	Gray code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Summary

Gray code

A shaft encoder is a typical application. Three IR emitter/detectors are used to encode the position of the shaft. The encoder on the left uses binary and can have three bits change together, creating a potential error. The encoder on the right uses gray code and only 1-bit changes, eliminating potential errors.



Summary

ASCII

ASCII is a code for alphanumeric characters and control characters. In its original form, ASCII encoded 128 characters and symbols using 7-bits. The first 32 characters are control characters, that are based on obsolete teletype requirements, so these characters are generally assigned to other functions in modern usage.

In 1981, IBM introduced extended ASCII, which is an 8-bit code and increased the character set to 256. Other extended sets (such as Unicode) have been introduced to handle characters in languages other than English.

Summary

Parity Method

The parity method is a method of error detection for simple transmission errors involving one bit (or an odd number of bits). A parity bit is an "extra" bit attached to a group of bits to force the number of 1's to be either even (even parity) or odd (odd parity).

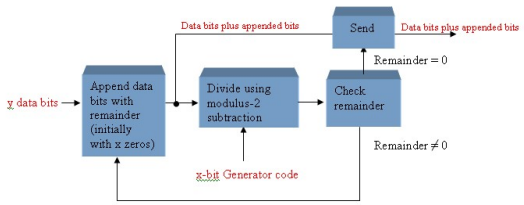
Example The ASCII character for "a" is 1100001 and for "A" is 1000001. What is the correct bit to append to make both of these have odd parity?

Solution The ASCII "a" has an odd number of bits that are equal to 1; therefore the parity bit is 0. The ASCII "A" has an even number of bits that are equal to 1; therefore the parity bit is 1.

Summary

Cyclic Redundancy Check

The cyclic redundancy check (CRC) is an error detection method that can detect multiple errors in larger blocks of data. At the sending end, a checksum is appended to a block of data. At the receiving end, the check sum is generated and compared to the sent checksum. If the check sums are the same, no error is detected.



Selected Key Terms

Byte	A group of eight bits
Floating-point number	A number representation based on scientific notation in which the number consists of an exponent and a mantissa.
Hexadecimal	A number system with a base of 16.
Octal	A number system with a base of 8.
BCD	Binary coded decimal; a digital code in which each of the decimal digits, 0 through 9, is represented by a group of four bits.

Selected Key Terms

Alphanumeric	Consisting of numerals, letters, and other characters
ASCII	American Standard Code for Information Interchange; the most widely used alphanumeric code.
Parity	In relation to binary codes, the condition of evenness or oddness in the number of 1s in a code group.
Cyclic redundancy check (CRC)	A type of error detection code.

Quiz

1. For the binary number 1000, the weight of the column with the 1 is

   a. 4

   b. 6

   c. 8

   d. 10

© 2008 Pearson Education

Quiz

2. The 2's complement of 1000 is

   a. 0111

   b. 1000

   c. 1001

   d. 1010

© 2008 Pearson Education

Quiz

3. The fractional binary number 0.11 has a decimal value of

   a. ¼

   b. ½

   c. ¾

   d. none of the above

© 2008 Pearson Education

Quiz

4. The hexadecimal number 2C has a decimal equivalent value of

   a. 14

   b. 44

   c. 64

   d. none of the above

© 2008 Pearson Education

Quiz

5. Assume that a floating point number is represented in binary. If the sign bit is 1, the

a. number is negative

b. number is positive

c. exponent is negative

d. exponent is positive

© 2008 Pearson Education

Quiz

6. When two positive signed numbers are added, the result may be larger that the size of the original numbers, creating overflow. This condition is indicated by

a. a change in the sign bit

b. a carry out of the sign position

c. a zero result

d. smoke

© 2008 Pearson Education

Quiz

7. The number 1010 in BCD is

a. equal to decimal eight

b. equal to decimal ten

c. equal to decimal twelve

d. invalid

© 2008 Pearson Education

Quiz

8. An example of an unweighted code is

a. binary

b. decimal

c. BCD

d. Gray code

© 2008 Pearson Education

Quiz

9. An example of an alphanumeric code is

a. hexadecimal

b. ASCII

c. BCD

d. CRC

© 2008 Pearson Education

Quiz

10. An example of an error detection method for transmitted data is the

a. parity check

b. CRC

c. both of the above

d. none of the above

© 2008 Pearson Education

Quiz

Answers:

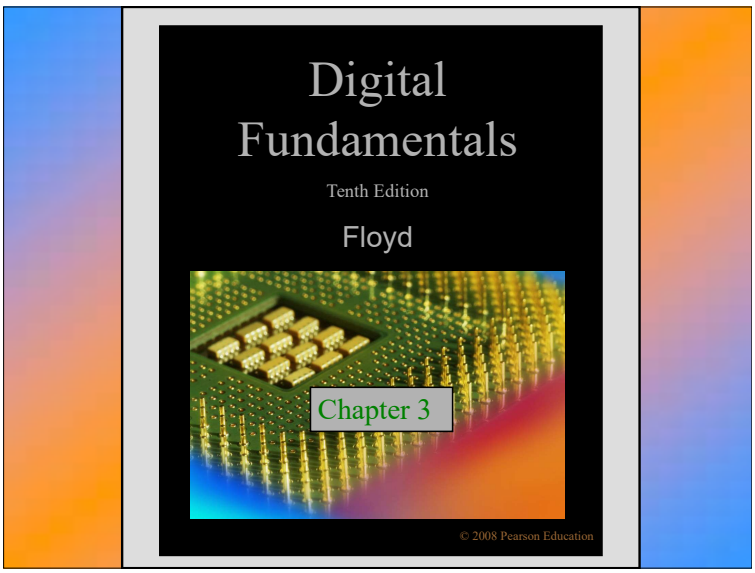
1. c	6. a
2. b	7. d
3. c	8. d
4. b	9. b
5. a	10. c

Homework

Problems

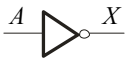
P100: 6(a,f,g), 7(b,g), 14, 18.b, 40(ae), 43(ag), 44(ab)

23(a,b), 24(a,b), 25(a,b), 26(a,b), 27(a,b), 28(a,b), 31(bcd), 49(ab), 58



Summary

The Inverter



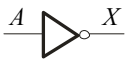
The inverter performs the Boolean **NOT** operation. When the input is LOW, the output is HIGH; when the input is HIGH, the output is LOW.

Input	Output
A	X
LOW (0)	HIGH (1)
HIGH (1)	LOW(0)

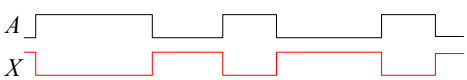
The **NOT** operation (complement) is shown with an overbar. Thus, the Boolean expression for an inverter is $X = \overline{A}$.

Summary

The Inverter



Example waveforms:



A group of inverters can be used to form the 1's complement of a binary number:

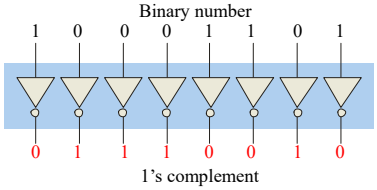


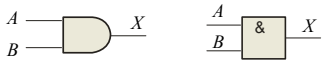
Figure 3.1 Standard logic symbols for the inverter (ANSI/IEEE Std. 91-1984).

(a) Distinctive shape symbols with negation indicators

(b) Rectangular outline symbols with polarity indicators

Summary

The AND Gate



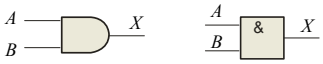
The **AND gate** produces a HIGH output when all inputs are HIGH; otherwise, the output is LOW. For a 2-input gate, the truth table is

Inputs		Output
A	*B*	*X*
0	0	0
0	1	0
1	0	0
1	1	1

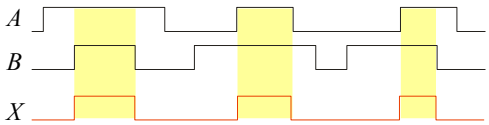
The **AND** operation is usually shown with a dot between the variables but it may be implied (no dot). Thus, the AND operation is written as $X = A \cdot B$ or $X = AB$.

Summary

The AND Gate



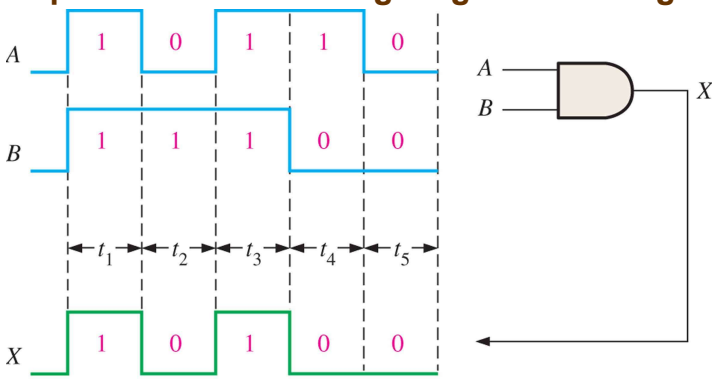
Example waveforms:



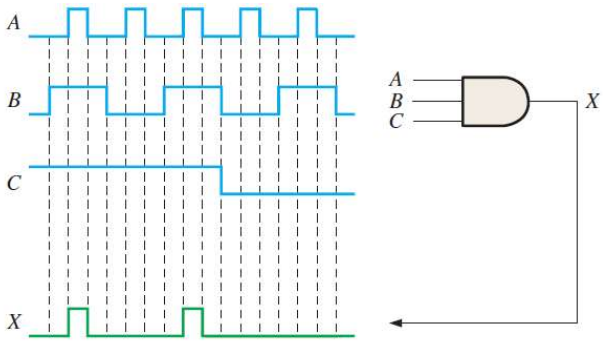
The AND operation is used in computer programming as a selective mask. If you want to retain certain bits of a binary number but reset the other bits to 0, you could set a mask with 1's in the position of the retained bits.

Example If the binary number 10100011 is ANDed with the mask 00001111, what is the result? 00000011

Figure 3.10 Example of AND gate operation with a timing diagram showing



Example of 3-input AND gate operation with a timing diagram



Logic Expressions for an AND Gate

•Boolean multiplication follows the same basic rules governing binary multiplication

0 * 0 = 0
0 * 1 = 0
1 * 0 = 0
1 * 1 = 1

•The operation of a 2-input AND gate can be expressed in equation form as follows

$X = AB$

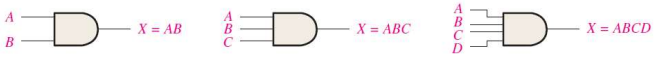


Figure 3.16 The AND Gate as an Enable/Inhibit Device.

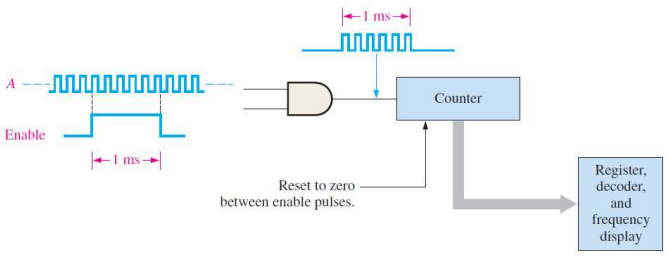
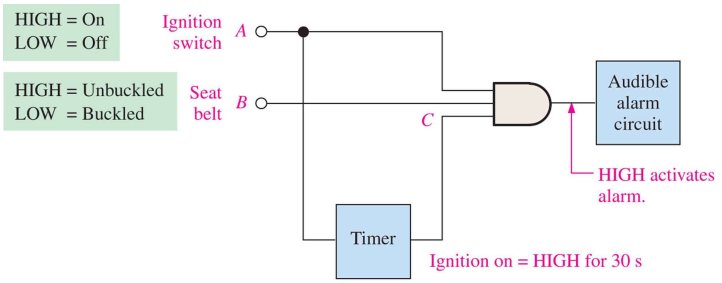


FIGURE 3–16 An AND gate performing an enable/inhibit function for a frequency counter.

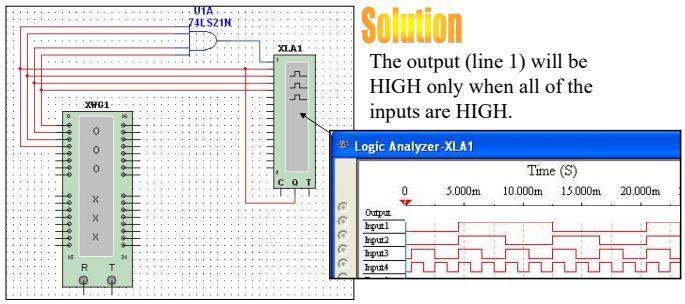
Figure 3.17 A simple seat belt alarm circuit using an AND gate.



Summary

The AND Gate

Example A Multisim circuit is shown. XWG1 is a word generator set in the count down mode. XLA1 is a logic analyzer with the output of the AND gate connected to first (upper) line of the analyzer. What signal do you expect to on this line?



Solution The output (line 1) will be HIGH only when all of the inputs are HIGH.

Summary

The OR Gate

The **OR gate** produces a HIGH output if any input is HIGH; if all inputs are LOW, the output is LOW. For a 2-input gate, the truth table is

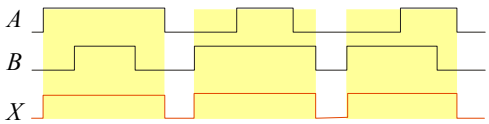
Inputs		Output
A	*B*	*X*
0	0	0
0	1	1
1	0	1
1	1	1

The **OR** operation is shown with a plus sign (+) between the variables. Thus, the OR operation is written as $X = A + B$.

Summary

The OR Gate

Example waveforms:

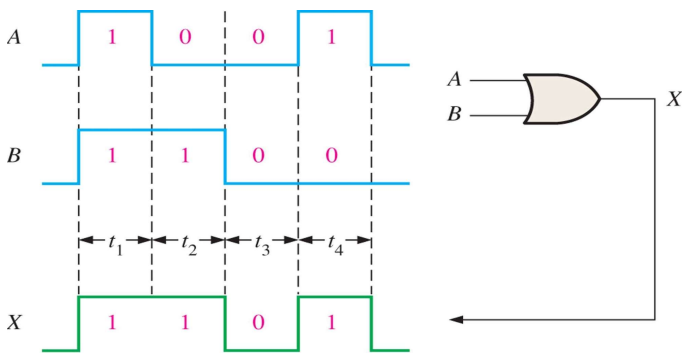


The OR operation can be used in computer programming to set certain bits of a binary number to 1.

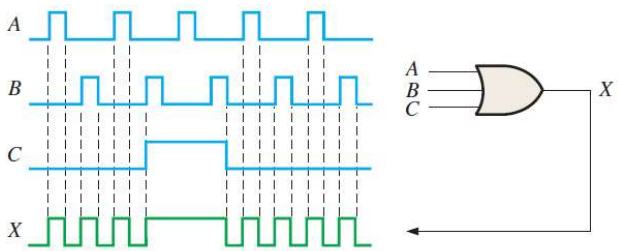
Example ASCII letters have a 1 in the bit 5 position for lower case letters and a 0 in this position for capitals. (Bit positions are numbered from right to left starting with 0.) What will be the result if you OR an ASCII letter with the 8-bit mask 00100000?

Solution The resulting letter will be lower case.

Figure 3.20 Example of OR gate operation with a timing diagram



Example of an 3-input OR gate operation with a timing diagram



Logic Expressions for an OR Gate

•The basic rules for **Boolean addition are as follows:**

$$0 + 0 = 0$$
$$0 + 1 = 1$$
$$1 + 0 = 1$$
$$1 + 1 = 1$$

•Boolean addition differs from binary addition in the case where two 1s are added. There is no carry in Boolean addition

•The operation of a 2-input OR gate can be expressed as follows

$$X = A + B$$

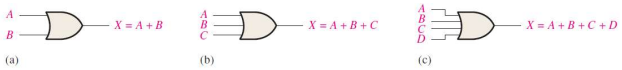
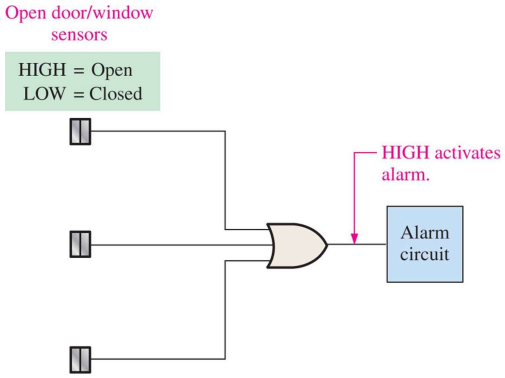


Figure 3.25 A simplified intrusion detection system using an OR gate.



Summary

The NAND Gate

The **NAND gate** produces a LOW output when all inputs are HIGH; otherwise, the output is HIGH. For a 2-input gate, the truth table is

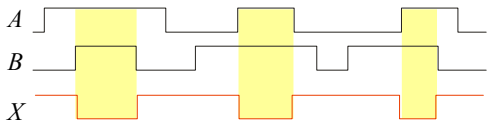
Inputs		Output
A	*B*	*X*
0	0	1
0	1	1
1	0	1
1	1	0

The **NAND** operation is shown with a dot between the variables and an overbar covering them. Thus, the NAND operation is written as $X = \overline{A \cdot B}$ (Alternatively, $X = \overline{AB}$.)

Summary

The NAND Gate

Example waveforms:



The NAND gate is particularly useful because it is a "universal" gate – all other basic gates can be constructed from NAND gates.

Question How would you connect a 2-input NAND gate to form a basic inverter?

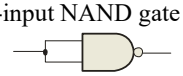


Figure 3.28

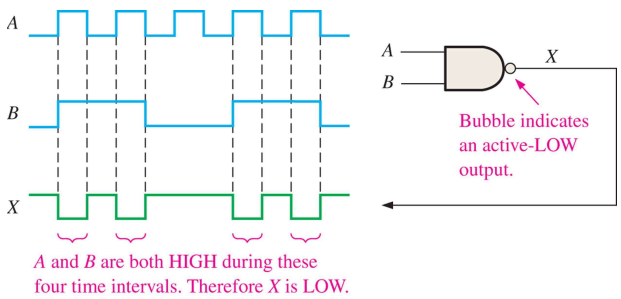


Figure 3.29

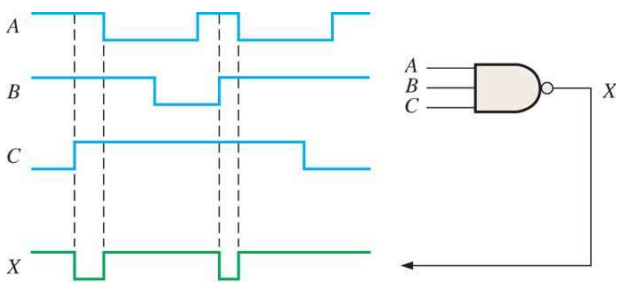
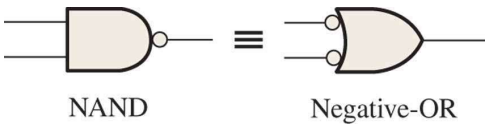


Figure 3.30 Standard symbols representing the two equivalent operations of a NAND gate.



Example:

Figure 3–31 shows a NAND gate with its two inputs connected to the tank level sensors and its output connected to the indicator panel. The operation can be stated as follows: If tank *A and tank B are above one-quarter full, the LED is on.*

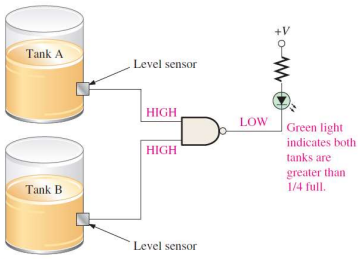


FIGURE 3–31

Example:

Figure 3–32 shows a NAND gate operating as a negative-OR gate to detect the occurrence of at least one LOW on its inputs

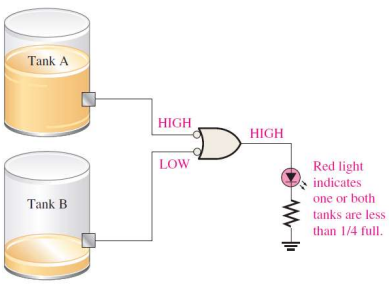


FIGURE 3–32

Example:

For the 4-input NAND gate in Figure 3–33, operating as a negative-OR gate, determine the output with respect to the inputs

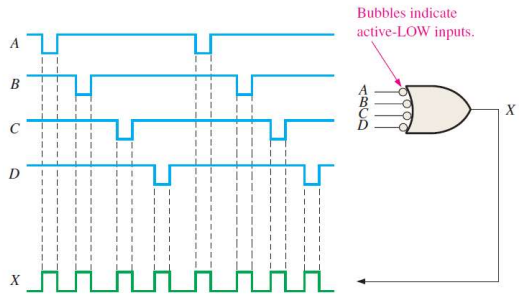


FIGURE 3–33

Summary

The NOR Gate

The **NOR gate** produces a LOW output if any input is HIGH; if all inputs are HIGH, the output is LOW. For a 2-input gate, the truth table is

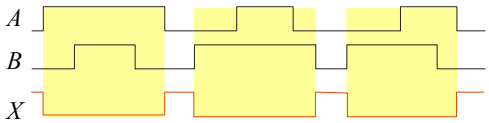
Inputs		Output
A	*B*	*X*
0	0	1
0	1	0
1	0	0
1	1	0

The **NOR** operation is shown with a plus sign (+) between the variables and an overbar covering them. Thus, the NOR operation is written as $X = \overline{A + B}$.

Summary

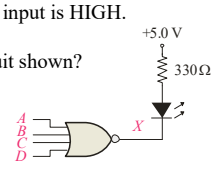
The NOR Gate

Example waveforms:



The NOR operation will produce a LOW if any input is HIGH.

Example When is the LED is ON for the circuit shown?



Solution The LED will be on when any of the four inputs are HIGH.

Example:

Show the output waveform for the 3-input NOR gate in Figure 3–37 with the proper time relation to the inputs.

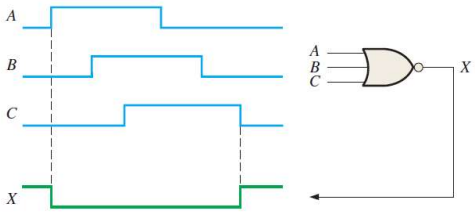
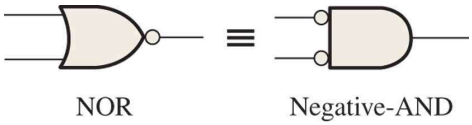


FIGURE 3–37

Figure 3.38 Standard symbols representing the two equivalent operations of a NOR gate.



Example:

As part of an aircraft's functional monitoring system, a circuit is required to indicate the status of the landing gears prior to landing

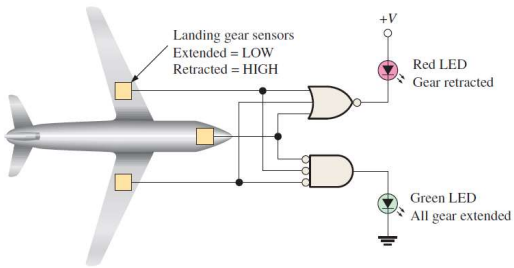


FIGURE 3–40

Example:

For the 4-input NOR gate operating as a negative-AND in Figure 3–41, determine the output relative to the inputs.

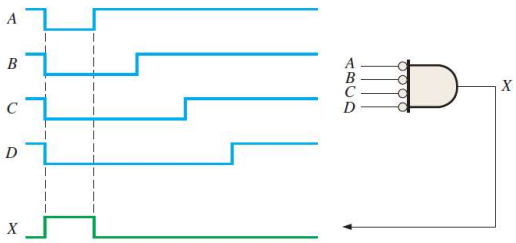


FIGURE 3–41

Summary

The XOR Gate

The **XOR gate** produces a HIGH output only when both inputs are at opposite logic levels. The truth table is

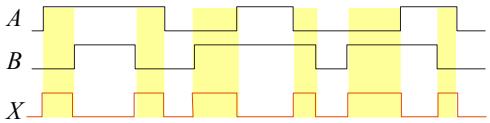
Inputs		Output
A	*B*	*X*
0	0	0
0	1	1
1	0	1
1	1	0

The **XOR** operation is written as $X = \overline{A}B + A\overline{B}$. Alternatively, it can be written with a circled plus sign between the variables as $X = A \oplus B$.

Summary

The XOR Gate

Example waveforms:



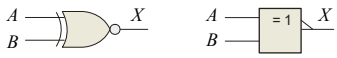
Notice that the XOR gate will produce a HIGH only when exactly one input is HIGH.

Question If the *A* and *B* waveforms are both inverted for the above waveforms, how is the output affected?

There is no change in the output.

Summary

The XNOR Gate



The **XNOR gate** produces a HIGH output only when both inputs are at the same logic level. The truth table is

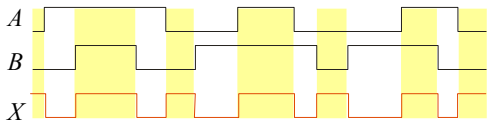
Inputs		Output
A	*B*	*X*
0	0	1
0	1	0
1	0	0
1	1	1

The **XNOR** operation shown as $X = \overline{A}\overline{B} + AB$. Alternatively, the XNOR operation can be shown with a circled dot between the variables. Thus, it can be shown as $X = A \odot B$.

Summary

The XNOR Gate

Example waveforms:

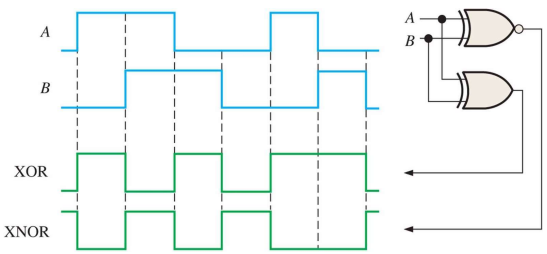


Notice that the XNOR gate will produce a HIGH when both inputs are the same. This makes it useful for comparison functions.

Question If the *A* waveform is inverted but *B* remains the same, how is the output affected?

The output will be inverted.

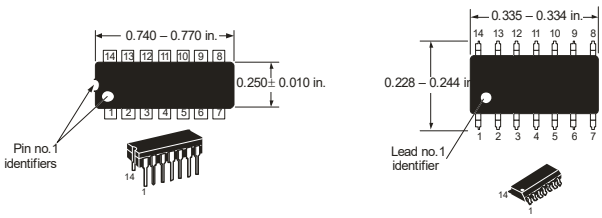
Figure 3.48



Summary

Fixed Function Logic

Two major fixed function logic families are TTL and CMOS. A third technology is BiCMOS, which combines the first two. Packaging for fixed function logic is shown.

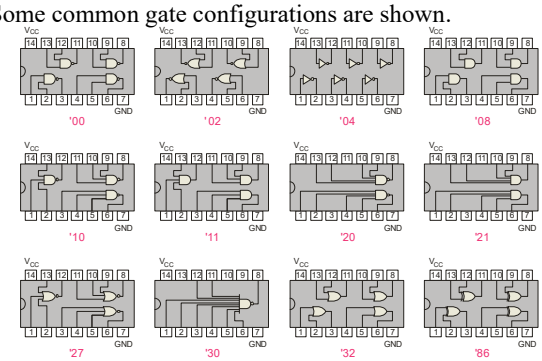


DIP package SOIC package

Summary

Fixed Function Logic

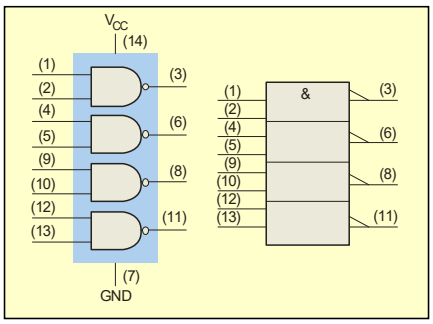
Some common gate configurations are shown.



Summary

Fixed Function Logic

Logic symbols show the gates and associated pin numbers.



Summary

Fixed Function Logic

Data sheets include limits and conditions set by the manufacturer as well as DC and AC characteristics. For example, some maximum ratings for a 74HC00A are:

MAXIMUM RATINGS

Symbol	Parameter	Value	Unit
V_{CC}	DC Supply Voltage (Referenced to GND)	−0.5 to + 7.0 V	V
V_{in}	DC InputVoltage (Referenced to GND)	−0.5 to V_{CC} +0.5 V	V
V_{out}	DC Output Voltage (Referenced to GND)	−0.5 to V_{CC} +0.5 V	V
I_{in}	DC Input Current, per pin	±20	mA
I_{out}	DC Output Current, per pin	±25	mA
I_{CC}	DC Supply Current, V_{CC} and GND pins	±50	mA
P_D	Power Dissipation in StillAir, Plastic or Ceramic DIP † SOIC Package † TSSOP Package †	750 500 450	mW
T_{stg}	Storage Temperature	−65 to + 150	°C
T_L	Lead Temperature, 1 mm from Case for 10 Seconds Plastic DIP, SOIC, or TSSOP Package Ceramic DIP	 260 300	°C

Summary

Fixed Function Logic

RECOMMENDED OPERATING CONDITIONS for a 74HC00A are:

RECOMMENDED OPERATING CONDITIONS

Symbol	Parameter		in	Max	Unit
V_{CC}	DC Supply Voltage (Referenced to GND)		2.0	6.0	V
V_{in}, V_{out}	DC Input Voltage, Output Voltage (Referenced to GND)		0	V_{CC}	V
T_A	Operating Temperature, All Package Types		−55	+125	°C
t_r, t_f	Input Rise and Fall Time	V_{CC} = 2.0 V	0	1000	ns
		V_{CC} = 4.5 V	0	500	
		V_{CC} = 6.0 V	0	400	

Summary

Fixed Function Logic

DC CHARACTERISTICS for a 74HC00A are:

DC CHARACTERISTICS (Voltages Referenced to GND) MC54/74HC00A

Symbol	Parameter	Condition	V_{CC} V	Guaranteed Limit −55 to 25°C	≤ 85°C	≤125°C	Unit
V_{IH}	Minimum High-Level Input Voltage	V_{out} = 0.1V or V_{CC} − 0.1V $\|I_{out}\| \le 20\mu A$	2.0 3.0 4.5 6.0	1.50 2.10 3.15 4.20	1.50 2.10 3.15 4.20	1.50 2.10 3.15 4.20	V
V_{IL}	Maximum Low-Level Input Voltage	V_{out} = 0.1V or V_{CC} − 0.1V $\|I_{out}\| \le 20\mu A$	2.0 3.0 4.5 6.0	0.50 0.90 1.35 1.80	0.50 0.90 1.35 1.80	0.50 0.90 1.35 1.80	V
V_{OH}	Minimum High-Level Output Voltage	V_{in} = V_{IH} or V_{IL} $\|I_{out}\| \le 20\mu A$	2.0 4.5 6.0	1.9 4.4 5.9	1.9 4.4 5.9	1.9 4.4 5.9	V
		V_{in} = V_{IH} or V_{IL} $\|I_{out}\| \le 2.4mA$ $\|I_{out}\| \le 4.0mA$ $\|I_{out}\| \le 5.2mA$	3.0 4.5 6.0	2.48 3.98 5.48	2.34 3.84 5.34	2.20 3.70 5.20	
V_{OL}	Maximum Low-Level Output Voltage	V_{in} = V_{IH} or V_{IL} $\|I_{out}\| \le 20\mu A$	2.0 4.5 6.0	0.1 0.1 0.1	0.1 0.1 0.1	0.1 0.1 0.1	V
		V_{in} = V_{IH} or V_{IL} $\|I_{out}\| \le 2.4mA$ $\|I_{out}\| \le 4.0mA$ $\|I_{out}\| \le 5.2mA$	3.0 4.5 6.0	0.26 0.26 0.26	0.33 0.33 0.33	0.40 0.40 0.40	
I_{in}	Maximum Input Leakage Current	V_{in} = V_{CC} or GND	6.0	±0.1	±1.0	±1.0	μA
I_{CC}	Maximum Quiescent Supply Current (per Package)	V_{in} = V_{CC} or GND I_{out} = 0μA	6.0	1.0	10	40	μA

Summary

Fixed Function Logic

AC CHARACTERISTICS for a 74HC00A are:

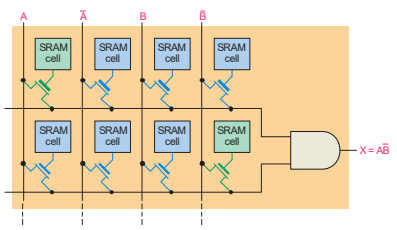
AC CHARACTERISTICS (T_A = 25°C)

Symbol	Parameter	Limits Min	Typ	Max	Unit	Test Conditions
t_{PLH}	Turn-Off Delay, Input to Output		9.0	15	ns	V_{CC} = 5.0 V C_L = 15 pF
t_{PHL}	Turn-On Delay, Input to Output		10	15	ns	

Summary

Programmable Logic

A Programmable Logic Device (PLD) can be programmed to implement logic. There are various technologies available for PLDs. Many use an internal array of AND gates to form logic terms. Many PLDs can be programmed multiple times.



Summary

Programmable Logic

In general, the required logic for a PLD is developed with the aid of a computer. The logic can be entered using a Hardware Description Language (HDL) such as VHDL. Logic can be specified to the HDL as a text file, a schematic diagram, or a state diagram.

Example A text entry for a programming a PLD in VHDL as a 2-input NAND gate is shown for reference in the following slide. In this case, the inputs and outputs are first specified. Then the signals are described. Although you are probably not familiar with VHDL, you can see that the program is simple to read.

Summary

Programmable Logic

```
entity NandGate is
        port(A, B: in bit;
        LED: out bit);
end entity NandGate;
architecture GateBehavior of NandGate is
signal A, B: bit;
begin
        X <= A nand B;
        LED <= X;
end architecture GateBehavior;
```

Selected Key Terms

Inverter A logic circuit that inverts or complements its inputs.

Truth table A table showing the inputs and corresponding output(s) of a logic circuit.

Timing diagram A diagram of waveforms showing the proper time relationship of all of the waveforms.

Boolean algebra The mathematics of logic circuits.

AND gate A logic gate that produces a HIGH output only when all of its inputs are HIGH.

Selected Key Terms

OR gate A logic gate that produces a HIGH output when one or more inputs are HIGH.

NAND gate A logic gate that produces a LOW output only when all of its inputs are HIGH.

NOR gate A logic gate that produces a LOW output when one or more inputs are HIGH.

Exclusive-OR gate A logic gate that produces a HIGH output only when its two inputs are at opposite levels.

Exclusive-NOR gate A logic gate that produces a LOW output only when its two inputs are at opposite levels.

Quiz

1. The truth table for a 2-input AND gate is

a.

Inputs		Output
A	*B*	*X*
0	0	0
0	1	1
1	0	1
1	1	0

b.

Inputs		Output
A	*B*	*X*
0	0	1
0	1	0
1	0	0
1	1	0

c.

Inputs		Output
A	*B*	*X*
0	0	0
0	1	0
1	0	0
1	1	1

d.

Inputs		Output
A	*B*	*X*
0	0	0
0	1	1
1	0	1
1	1	1

© 2008 Pearson Education

Quiz

2. The truth table for a 2-input NOR gate is

a.

Inputs		Output
A	*B*	*X*
0	0	0
0	1	1
1	0	1
1	1	0

b.

Inputs		Output
A	*B*	*X*
0	0	1
0	1	0
1	0	0
1	1	0

c.

Inputs		Output
A	*B*	*X*
0	0	0
0	1	0
1	0	0
1	1	1

d.

Inputs		Output
A	*B*	*X*
0	0	0
0	1	1
1	0	1
1	1	1

© 2008 Pearson Education

Quiz

3. The truth table for a 2-input XOR gate is

a.

Inputs		Output
A	*B*	*X*
0	0	0
0	1	1
1	0	1
1	1	0

b.

Inputs		Output
A	*B*	*X*
0	0	1
0	1	0
1	0	0
1	1	0

c.

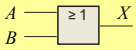
Inputs		Output
A	*B*	*X*
0	0	0
0	1	0
1	0	0
1	1	1

d.

Inputs		Output
A	*B*	*X*
0	0	0
0	1	1
1	0	1
1	1	1

© 2008 Pearson Education

Quiz

4. The symbol 

is for a(n)

a. OR gate

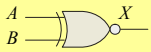
b. AND gate

c. NOR gate

d. XOR gate

© 2008 Pearson Education

Quiz

5. The symbol 

is for a(n)

a. OR gate

b. AND gate

c. NOR gate

d. XOR gate

© 2008 Pearson Education

Quiz

6. A logic gate that produces a HIGH output only when all of its inputs are HIGH is a(n)

a. OR gate

b. AND gate

c. NOR gate

d. NAND gate

© 2008 Pearson Education

Quiz

7. The expression $X = A \oplus B$ means

a. *A* OR *B*

b. *A* AND *B*

c. *A* XOR *B*

d. *A* XNOR *B*

© 2008 Pearson Education

Quiz

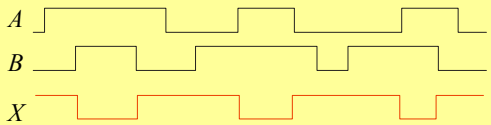
8. A 2-input gate produces the output shown. (*X* represents the output.) This is a(n)

a. OR gate

b. AND gate

c. NOR gate

d. NAND gate



© 2008 Pearson Education

Quiz

9. A 2-input gate produces a HIGH output only when the inputs agree. This type of gate is a(n)

a. OR gate

b. AND gate

c. NOR gate

d. XNOR gate

© 2008 Pearson Education

Quiz

10. The required logic for a PLD can be specified in an Hardware Description Language by

a. text entry

b. schematic entry

c. state diagrams

d. all of the above

© 2008 Pearson Education

Quiz

Answers:

1. c	6. b
2. b	7. c
3. a	8. d
4. a	9. d
5. d	10. d

Summary

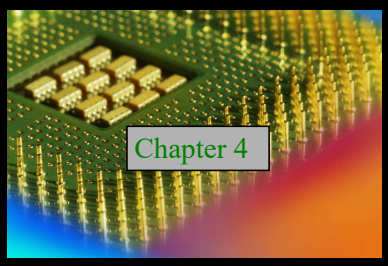
Homework

P93: 8, 13, 18, 20, 24, 28

Digital Fundamentals

Tenth Edition

Floyd



© 2008 Pearson Education

Summary

Boolean Addition

In Boolean algebra, a **variable** is a symbol used to represent an action, a condition, or data. A single variable can only have a value of 1 or 0.

The **complement** represents the inverse of a variable and is indicated with an overbar. Thus, the complement of A is $\overline{A}$.

A **literal** is a variable or its complement.

Addition is equivalent to the OR operation. The sum term is 1 if one or more if the literals are 1. The sum term is zero only if each literal is 0.

Example Determine the values of A, B, and C that make the sum term of the expression $\overline{A} + B + \overline{C} = 0$?

Solution Each literal must = 0; therefore $A = 1$, $B = 0$ and $C = 1$.

Summary

Boolean Multiplication

In Boolean algebra, multiplication is equivalent to the AND operation. The product of literals forms a product term. The product term will be 1 only if all of the literals are 1.

Example What are the values of the A, B and C if the product term of $A\overline{B}\overline{C} = 1$?

Solution Each literal must = 1; therefore $A = 1$, $B = 0$ and $C = 0$.

Summary

Commutative Laws

The **commutative laws** are applied to addition and multiplication. For addition, the commutative law states

In terms of the result, the order in which variables are ORed makes no difference.

$$A + B = B + A$$

For multiplication, the commutative law states

In terms of the result, the order in which variables are ANDed makes no difference.

$$AB = BA$$

Summary

Associative Laws

The **associative laws** are also applied to addition and multiplication. For addition, the associative law states

When ORing more than two variables, the result is the same regardless of the grouping of the variables.

$$A + (B + C) = (A + B) + C$$

For multiplication, the associative law states

When ANDing more than two variables, the result is the same regardless of the grouping of the variables.

$$A(BC) = (AB)C$$

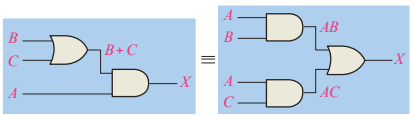
Summary

Distributive Law

The **distributive law** is the factoring law. A common variable can be factored from an expression just as in ordinary algebra. That is

$$AB + AC = A(B + C)$$

The distributive law can be illustrated with equivalent circuits:



$A(B + C)$ $\quad$ $AB + AC$

Summary

Rules of Boolean Algebra

1. $A + 0 = A$
2. $A + 1 = 1$
3. $A \cdot 0 = 0$
4. $A \cdot 1 = 1$
5. $A + A = A$
6. $A + \overline{A} = 1$
7. $A \cdot A = A$
8. $A \cdot \overline{A} = 0$
9. $\overline{\overline{A}} = A$
10. $A + AB = A, \quad A(A+B) = A$
11. $A + \overline{A}B = A + B$
12. $(A + B)(A + C) = A + BC$

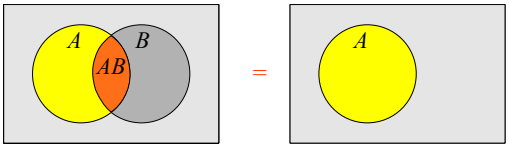
Summary

Rules of Boolean Algebra

Rules of Boolean algebra can be illustrated with *Venn* diagrams. The variable A is shown as an area.

The rule $A + AB = A$ can be illustrated easily with a diagram. Add an overlapping area to represent the variable B.

The overlap region between A and B represents AB.



The diagram visually shows that $A + AB = A$. Other rules can be illustrated with the diagrams as well.

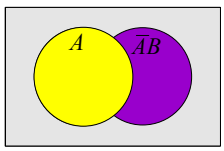
Summary

Rules of Boolean Algebra

Example Illustrate the rule $A + \overline{A}B = A + B$ with a Venn diagram.

Solution

This time, $\overline{A}$ is represented by the blue area and B again by the red circle. The intersection represents $\overline{A}B$. Notice that $A + \overline{A}B = A + B$



Summary

Rules of Boolean Algebra

Rule 12, which states that $(A + B)(A + C) = A + BC$, can be proven by applying earlier rules as follows:

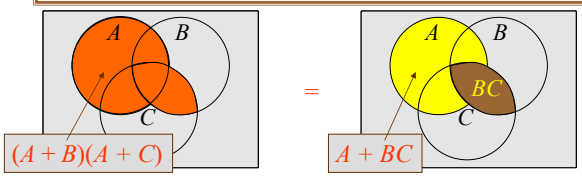
$$\begin{aligned}(A + B)(A + C) &= AA + AC + AB + BC \\ &= A + AC + AB + BC \\ &= A(1 + C + B) + BC \\ &= A \cdot 1 + BC \\ &= A + BC\end{aligned}$$

This rule is a little more complicated, but it can also be shown with a Venn diagram, as given on the following slide…

Summary

Three areas represent the variables A, B, and C.
The area representing $A + B$ is shown in yellow.
The area representing $A + C$ is shown in red.
The overlap of red and yellow is shown in orange.

The overlapping area between B and C represents BC.
ORing with A gives the same area as before.



Summary

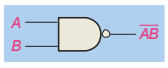
DeMorgan's Theorem

DeMorgan's 1st Theorem

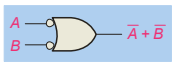
The complement of a product of variables is equal to the sum of the complemented variables.

$$\overline{AB} = \overline{A} + \overline{B}$$

Applying DeMorgan's first theorem to gates:



NAND



Negative-OR

Inputs		Output	
A	B	$\overline{AB}$	$\overline{A} + \overline{B}$
0	0	1	1
0	1	1	1
1	0	1	1
1	1	0	0

Summary

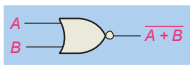
DeMorgan's Theorem

DeMorgan's 2nd Theorem

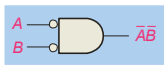
The complement of a sum of variables is equal to the product of the complemented variables.

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

Applying DeMorgan's second theorem to gates:



NOR



Negative-AND

Inputs		Output	
A	B	$\overline{A + B}$	$\overline{AB}$
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0

Summary

DeMorgan's Theorem

Example Apply DeMorgan's theorem to remove the overbar covering both terms from the expression $X = \overline{\overline{C} + D}$.

Solution To apply DeMorgan's theorem to the expression, you can break the overbar covering both terms and change the sign between the terms. This results in $X = \overline{\overline{C}} \cdot \overline{D}$. Deleting the double bar gives $X = C \cdot \overline{D}$.

Summary

DeMorgan's Theorem

Example Apply DeMorgan's theorem to the following expressions

(a) $\overline{(A + B + C)D}$
(b) $\overline{ABC + DEF}$
(c) $\overline{A\overline{B} + \overline{C}D + EF}$

(a) $\overline{\overline{(A + B)} + \overline{C}}$
(b) $\overline{(\overline{A} + B) + CD}$
(c) $\overline{(A + B)\overline{C}\,\overline{D} + E + F}$

(a) $\overline{ABC} + (\overline{\overline{D} + E})$ **(b)** $\overline{(A + B)C}$ **(c)** $\overline{A + B + C} + \overline{\overline{D}E}$

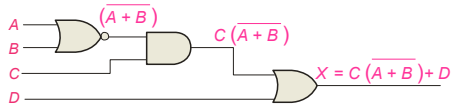
Summary

Boolean Analysis of Logic Circuits

Combinational logic circuits can be analyzed by writing the expression for each gate and combining the expressions according to the rules for Boolean algebra.

Example Apply Boolean algebra to derive the expression for *X*.

Solution Write the expression for each gate:



Applying DeMorgan's theorem and the distribution law:

$$X = C(\overline{A}\ \overline{B}) + D = \overline{A}\,\overline{B}\,C + D$$

Summary

Boolean Analysis of Logic Circuits

Example Generate the truth table for the circuit in the previous example.

1. Replace the AND gates with OR gates and the OR gate with an AND gate in Figure 4–16. Determine the Boolean expression for the output.
2. Construct a truth table for the circuit in Question 1.

Summary

Boolean Analysis of Logic Circuits

Construct a truth table for A(B+CD) , and draw the logic diagram

There are three ways to describe the switching functions

1. Switching equations
2. Truth table
3. Logic diagram

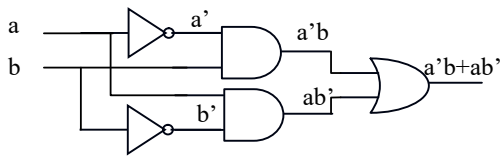
TABLE 4–5
Truth table for the logic circuit in Figure 4–18.

Inputs				Output
A	*B*	*C*	*D*	*A(B + CD)*
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Summary

Boolean Analysis of Logic Circuits

Draw the logic diagram for
T=a'b+ab'



Summary

Simplification using Boolean algebra

- Why reduction?
  to reduce cost of the circuit
- "And-Or" expression (sum of products)
- "Or-And" expression (product of sums)

Using Boolean algebra techniques, simplify this expression:

$AB + A(B + C) + B(B + C)$

Summary

Simplification using Boolean algebra

- Simplify the following Boolean expressions:

$$[A\overline{B}(C + BD) + \overline{A}\,\overline{B}]C$$

$$\overline{A}BC + A\overline{B}\,\overline{C} + \overline{A}\,\overline{B}\,\overline{C} + A\overline{B}C + ABC$$

$$\overline{AB + AC} + \overline{A}\,\overline{B}C$$

Simplification using Boolean algebra

- "And-Or" expression

Exe. 1、 $F=\bar{A}\bar{C}(\bar{B}+BD)+\bar{A}CD$

2、 $F=(A\oplus B)\overline{AB+\bar{A}\bar{B}}+AB$

ANS: 1 $\bar{A}\bar{B}\bar{C}+\bar{A}D$

2 A+B

Simplification using Boolean algebra

"Or-And" expression (product of sums)

E.g. S=A(B'+C)'(BC)'

Solution: **S=A(BC')(B'+C')** DEMO

=(ABC')(B'+C') Distribution

= ABB'C'+ABC'C' ..

=ABC' complement, idempotence

Simplification using Boolean algebra

- "Or-And" expression (product of sums)

Exe. 1、 $F=(A+B)(A+\overline{B})(B+C)(B+C+D)$

2、 $F=(A+\overline{B})(\overline{A}+B)(B+C)(\overline{A}+C)$

ANS: 1、A(B+C)

2、 $(A+\overline{B})(\overline{A}+B)C$

Homework

P141:

2, 6(a,d,e),10, 15(b,c),18(d,c),20(b,c,e), 22

Chapter 4: Bool algebra and Logic Simplification

Section 4.6-4.9

Summary

SOP and POS forms

Boolean expressions can be written in the **sum-of-products** form (**SOP**) or in the **product-of-sums** form (**POS**). These forms can simplify the implementation of combinational logic, particularly with PLDs. In both forms, an overbar cannot extend over more than one variable.

An expression is in SOP form when two or more product terms are summed as in the following examples:

$\bar{A}\bar{B}\bar{C}+AB$ $AB\bar{C}+\bar{C}\bar{D}$ $CD+\bar{E}$

An expression is in POS form when two or more sum terms are multiplied as in the following examples:

$(A+B)(\bar{A}+C)$ $(A+B+\bar{C})(B+D)$ $(\bar{A}+B)C$

Summary

SOP Standard form

In **SOP standard form**, every variable in the domain must appear in each term. This form is useful for constructing truth tables or for implementing logic in PLDs.

You can expand a nonstandard term to standard form by multiplying the term by a term consisting of the sum of the missing variable and its complement.

Example Convert $X = \overline{A}\,\overline{B} + A\,B\,C$ to standard form.

Solution The first term does not include the variable C. Therefore, multiply it by the $(C + \overline{C})$, which = 1:

$X = \overline{A}\,\overline{B}\,(C + \overline{C}) + A\,B\,C$

$= \overline{A}\,\overline{B}\,C + \overline{A}\,\overline{B}\,\overline{C} + A\,B\,C$

Summary

POS Standard form

In **POS standard form**, every variable in the domain must appear in each sum term of the expression.

You can expand a nonstandard POS expression to standard form by adding the product of the missing variable and its complement and applying rule 12, which states that $(A + B)(A + C) = A + BC$.

Example Convert $X = (\overline{A} + \overline{B})(A + B + C)$ to standard form.

Solution The first sum term does not include the variable C. Therefore, add $C\,\overline{C}$ and expand the result by rule 12.

$X = (\overline{A} + \overline{B} + C\,\overline{C})(A + B + C)$

$= (\overline{A} + \overline{B} + C)(\overline{A} + \overline{B} + \overline{C})(A + B + C)$

Convert switching equations to canonical form

- Expand *non-canonical* terms by inserting equivalent of 1 in each missing variable x:
  $(x + x') = 1$
- Remove duplicate minterms
- $f_1(a,b,c) = a'b'c + bc' + ac'$
  $= a'b'c + (a+a')bc' + a(b+b')c'$
  $= a'b'c + abc' + a'bc' + abc' + ab'c'$
  $= a'b'c + abc' + a'bc + ab'c'$

Convert switching equations to canonical form

- SOP

E.g. P=f(a,b,c)=ab'+ac'+bc

=ab'(c+c')+ac' (b+b')+bc(a+a')

=ab'c+ab'c'+abc'+ab'c'+abc+a'bc

=ab'c+ab'c'+abc'+abc+a'bc

Convert switching equations to canonical form

- Expand noncanonical terms by adding 0 in terms of missing variables (*e.g.*, $xx' = 0$) and using the distributive law
- Remove duplicate maxterms
- $f_1(a,b,c) = (a+b+c)\cdot(b'+c')\cdot(a'+c')$
  $= (a+b+c)\cdot(aa'+b'+c')\cdot(a'+bb'+c')$
  $= (a+b+c)\cdot(a+b'+c')\cdot(a'+b'+c')\cdot$
  $(a'+b+c')\cdot(a'+b'+c')$
  $= (a+b+c)\cdot(a+b'+c')\cdot(a'+b'+c')\cdot(a'+b+c')$

Convert switching equations to canonical form

- POS

E.g. T=f(a,b,c)=(a+b')(b'+c)

=(a+b'+cc')(b'+c+aa')

=(a+b'+c) (a+b'+c')(a+b'+c)(a'+b'+c)

=(a+b'+c) (a+b'+c')(a'+b'+c)

Convert switching equations to canonical form

- Exe:

  P=f(w,x,y,z)=w'x+yz'

  T=f(a,b,c,d)=(a+b'+c)(a'+d)

- **Ans:**

  P=f(w,x,y,z)=wxyz'+wx'yz'+w'xyz+w'xyz'+w'xy'z
  +w'xy'z'+w'x'yz'

  T=f(a,b,c,d)= (a+b'+c+d) (a+b'+c+d') (a'+b+c+d)
  (a'+b'+c+d) (a'+b+c'+d) (a'+b'+c'+d)

Boolean Expressions and Truth Tables

- Converting SOP expressions to truth table
  - Develop a truth table for the standard SOP expression A'B'C + AB' C' + ABC.

TABLE 4–6

Inputs			Output	
A	*B*	*C*	*X*	Product Term
0	0	0	0	
0	0	1	1	$\overline{A}\,\overline{B}C$
0	1	0	0	
0	1	1	0	
1	0	0	1	$A\overline{B}\,\overline{C}$
1	0	1	0	
1	1	0	0	
1	1	1	1	ABC

Boolean Expressions and Truth Tables

- Converting POS Expressions to Truth Table Format
  - Determine the truth table for the following standard POS expression:

$(A + B + C)(A + \overline{B} + C)(A + \overline{B} + \overline{C})(\overline{A} + B + \overline{C})(\overline{A} + \overline{B} + C)$

TABLE 4–7

Inputs			Output	
A	*B*	*C*	*X*	Sum Term
0	0	0	0	$(A + B + C)$
0	0	1	1	
0	1	0	0	$(A + \overline{B} + C)$
0	1	1	0	$(A + \overline{B} + \overline{C})$
1	0	0	1	
1	0	1	0	$(\overline{A} + B + \overline{C})$
1	1	0	0	$(\overline{A} + \overline{B} + C)$
1	1	1	1	

Boolean Expressions and Truth Tables

- Determining Standard Expressions from a Truth
  - From the truth table in Table 4–8, determine the standard SOP expression and the equivalent standard POS expression

TABLE 4–8

Inputs			Output
A	*B*	*C*	*X*
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

- Definitions
  - *Product term:* literals connected by •
  - *Sum term:* literals connected by +
  - *Minterm:* a product term in which all the variables appear exactly once, either complemented or uncomplemented
  - *Maxterm:* a sum term in which all the variables appear exactly once, either complemented or uncomplemented

Minterm

- Represents exactly one combination in the truth table.
- Denoted by m_j, where *j* is the decimal equivalent of the minterm's corresponding binary combination (b_j).
- A variable in m_j is complemented if its value in b_j is 0, otherwise is uncomplemented.
- M(a,b,m,s)=a'bms+ab'ms+abms

  (m_7) (m_{11}) (m_{15})

Maxterm

- Represents exactly one combination in the truth table.
- Denoted by M_j, where j is the decimal equivalent of the maxterm's corresponding binary combination (b_j).
- A variable in M_j is complemented if its value in b_j is 1, otherwise is uncomplemented.
- F(a,b,c)=(a'+b+c')(a+b+c')(a+b'+c')

  101 001 011

  M_5 M_1 M_3

Truth Table notation for Minterms and Maxterms

Input variables			Minterm		Maxterm	
a	b	c	Term	Designation	Term	Designation
0	0	0	$a'b'c'$	m_0	$a+b+c$	M_0
0	0	1	$a'b'c$	m_1	$a+b+c'$	M_1
0	1	0	$a'bc'$	m_2	$a+b'+c$	M_2
0	1	1	$a'bc$	m_3	$a+b'+c'$	M_3
1	0	0	$ab'c'$	m_4	$a'+b+c$	M_4
1	0	1	$ab'c$	m_5	$a'+b+c'$	M_5
1	1	0	abc'	m_6	$a'+b'+c$	M_6
1	1	1	abc	m_7	$a'+b'+c'$	M_7

3 variables x,y,z (order is fixed)

Example

- Truth table for $f_1(a,b,c)$ at right
- The standard sum-of-products form for f_1 is
  $f_1(a,b,c) = m_1 + m_2 + m_4 + m_6$
  $= a'b'c + a'bc' + ab'c' + abc'$
- The standard product-of-sums form for f_1 is
  $f_1(a,b,c) = M_0 \bullet M_3 \bullet M_5 \bullet M_7$
  $= (a+b+c)\bullet(a+b'+c')\bullet$
  $(a'+b+c')\bullet(a'+b'+c')$.
- Observe that: $m_j = M_j'$

a	b	c		f_1
0	0	0		0
0	0	1		1
0	1	0		1
0	1	1		0
1	0	0		1
1	0	1		0
1	1	0		1
1	1	1		0

Shorthand: ∑ and ∏

- $f_1(a,b,c) = \sum m(1,2,4,6)$, where ∑ indicates that this is a sum-of-products form, and m(1,2,4,6) indicates that the minterms to be included are m_1, m_2, m_4, and m_6.
- $f_1(a,b,c) = \prod M(0,3,5,7)$, where ∏ indicates that this is a product-of-sums form, and M(0,3,5,7) indicates that the maxterms to be included are M_0, M_3, M_5, and M_7.
- $\sum m(1,2,4,6) = \prod M(0,3,5,7) = f_1(a,b,c)$

Karnaugh Maps

- Why Karnaugh maps?
  to simplify boolean equations
- Karnaugh maps (K-maps) are *graphical* representations of boolean functions.
- One *map cell* corresponds to a row in the truth table.
- one map cell corresponds to a minterm or a maxterm in the boolean expression

Karnaugh Maps

B \ A	0	1
0	m_0	m_2
1	m_1	m_3

C \ AB	00	01	11	10
0	m_0	m_2	m_6	m_4
1	m_1	m_3	m_7	m_5

CD \ AB	00	01	11	10
00	m_0	m_4	m_{12}	m_8
01	m_1	m_5	m_{13}	m_9
11	m_3	m_7	m_{15}	m_{11}
10	m_2	m_6	m_{14}	m_{10}

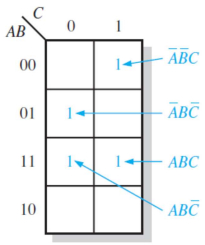
- Logic adjacency : Any two adjacent cells in the map differ by ONLY one.
- Example:
  m_0 (=$x_1'x_2'$) is adjacent to m_1 (=$x_1'x_2$) and m_2 (=x_1x_2') but NOT m_3 (=x_1x_2)

Mapping a Standard SOP Expression

- Map the following standard SOP expression on a Karnaugh map:

$\overline{A}\,\overline{B}C + \overline{A}B\overline{C} + AB\overline{C} + ABC$

$\overline{A}\,\overline{B}C + \overline{A}B\overline{C} + AB\overline{C} + ABC$
001 010 110 111

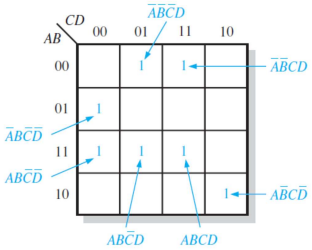


Mapping a Standard SOP Expression

- Map the following standard SOP expression on a Karnaugh map:

$\overline{A}\,\overline{B}CD + \overline{A}B\overline{C}\,\overline{D} + AB\overline{C}D + ABCD + AB\overline{C}\,\overline{D} + \overline{A}\,\overline{B}\,\overline{C}D + A\overline{B}C\overline{D}$

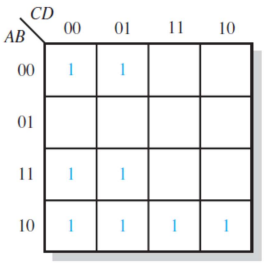
$\overline{A}\,\overline{B}CD + \overline{A}B\overline{C}\,\overline{D} + AB\overline{C}D + ABCD + AB\overline{C}\,\overline{D} + \overline{A}\,\overline{B}\,\overline{C}D + A\overline{B}C\overline{D}$
0011 0100 1101 1111 1100 0001 1010



Mapping a Nonstandard SOP Expression

- Map the following SOP expression on a Karnaugh map:

$\overline{B}\,\overline{C} + A\overline{B} + AB\overline{C} + A\overline{B}C\overline{D} + \overline{A}\,\overline{B}\,\overline{C}D + A\overline{B}CD$



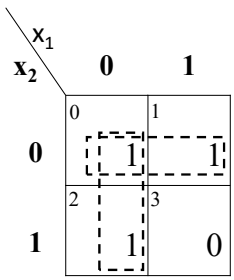
2-Variable Map -- Example

- E.g.

$f(x_1,x_2) = x_1'x_2' + x_1'x_2 + x_1x_2'$
$= m_0 + m_1 + m_2$

- What (simpler) function is represented by each dashed rectangle?
  - $x_1'x_2' + x_1'x_2 = x_1'(x_2' + x_2) = x_1'$
  - $x_1'x_2' + x_1x_2' = x_2'(x_1' + x_1) = x_2'$

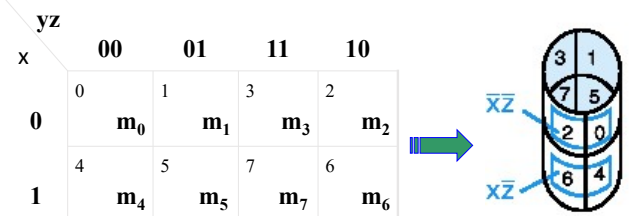
$f(x_1,x_2) = x_1' + x_2'$



Minimization as SOP using K-map

- Enter 1s in the K-map for each product term in the function
- Group *adjacent* K-map cells containing 1s to obtain a product with fewer variables. Groups must be in power of 2 (2, 4, 8, …)
- Handle "boundary wrap" for K-maps of 3 or more variables.

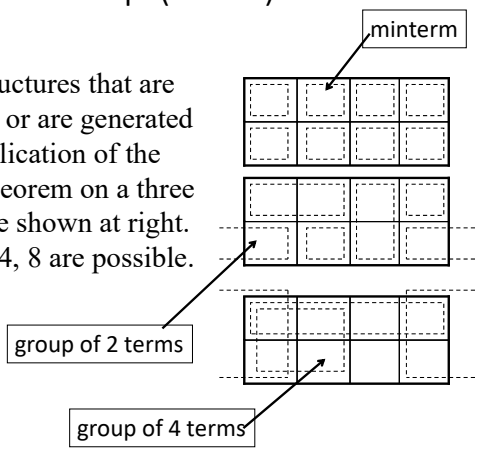
Three-Variable Map



-Note: variable ordering is (x,y,z); yz specifies column, x specifies row.
-Each cell is adjacent to *three* other cells (left or right or top or bottom or edge wrap)

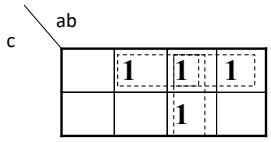
Three-Variable Map (cont.)

The types of structures that are either minterms or are generated by repeated application of the minimization theorem on a three variable map are shown at right. Groups of 1, 2, 4, 8 are possible.



Simplification

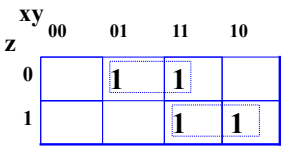
- Example: f(a,b,c) = ac' + abc + bc'
  =a(b+b')c'+abc+(a+a')bc'
  =abc'+ab'c'+abc+abc'+a'bc'
  =abc'+ab'c'+abc+a'bc'
- Result: f(a,b,c) = ac'+ab+bc'



More Examples

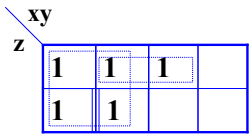
- $f_1(x, y, z) = \sum m(2,3,5,7)$

$f_1(x, y, z) = yz' + xz$



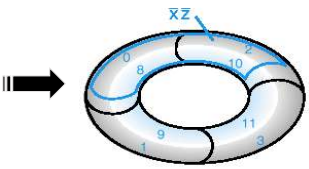
- $f_2(x, y, z) = \sum m(0,1,2,3,6)$

$f_2(x, y, z) = x'+yz'$



Four-Variable Maps

WX \ YZ	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}



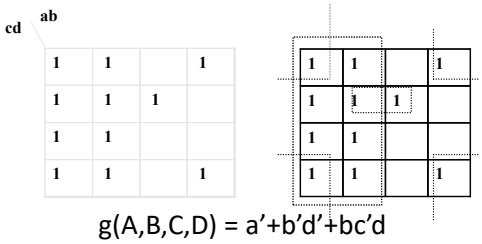
- Top cells are adjacent to bottom cells. Left-edge cells are adjacent to right-edge cells.
- Note variable ordering (WXYZ).

Four-variable Map Simplification

- One square represents a minterm of 4 literals.
- A rectangle of 2 adjacent squares represents a product term of 3 literals.
- A rectangle of 4 squares represents a product term of 2 literals.
- A rectangle of 8 squares represents a product term of 1 literal.
- A rectangle of 16 squares produces a function that is equal to logic 1.

Example

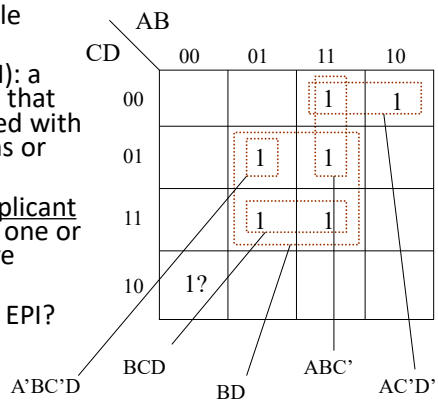
- Simplify the following Boolean function (A,B,C,D) = ∑m(0,1,2,3,4,5,6,7,8,10,13).
- First put the function g( ) into the map, and then group as many 1s as possible.



g(A,B,C,D) = a'+b'd'+bc'd

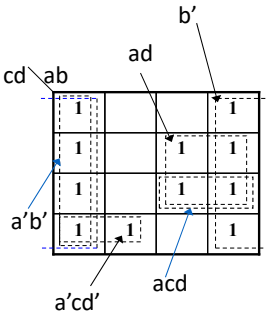
Implicants ,Prime Implicants (PIs) and Essential Prime Implicant (EPI)

- Implicant: any single mintern
- Prime Implicant (PI): a group of minterms that cannot be combined with any other minterms or groups
- Essential Prime Implicant (EPI): a PI in which one or more miniterms are unique
- How to find PI and EPI?



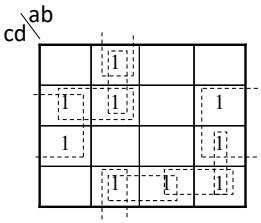
Example

- *a'b'* is not a prime implicant because it is contained in *b'.*
- *acd* is not a prime implicant because it is contained in *ad*.
- b', ad, and a'cd' are prime implicants.
- b', ad, and a'cd' are essential prime implicants.



Another Example

- Consider $f_2(a,b,c,d)$, whose K-map is shown below.
- The only essential PI is b'd.

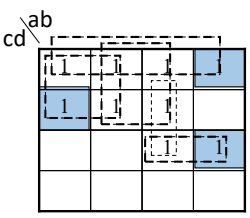


Systematic Procedure for Simplifying Boolean Functions

1. Load the minterms into the K-map by placing a 1 in the appropriate square.
2. Look for groups of minterms (PI)
   1. The group size must be a power of 2
   2. Find the largest groups of minterms first, then smaller collections until all groups are found.
3. Find all essential PIs.
4. For remaining min-terms not included in the essential PIs, select a set of other PIs to cover them.
5. The resulting simplified function is the logical OR of the product terms selected above.

Example

- f(a,b,c,d) = ∑m(0,1,4,5,8,11,12,13,15).
- F(a,b,c,d) = c'd' + a'c' + bc' + acd



Don't Care Conditions

- There may be a combination of input values which
  - will **never** occur
  - if they do occur, the output is of no concern.
- The function value for such combinations is called a *don't care*.
- They are denoted with **x** or **–**. Each **x** may be arbitrarily assigned the value 0 or 1 in an implementation.
- Don't cares can be used to ***further*** simplify a function

Minimization using Don't Cares

- Treat don't cares as if they are 1s to generate PIs.
- Delete PI's that cover only don't care minterms.
- Treat the covering of remaining don't care minterms as optional in the selection process (*i.e.* they may be, but need not be, covered).

Example

ab \ cd	00	01	11	10
00	0	1	0	1
01	1	1	0	1
11	0	0	d	d
10	1	1	d	d

- Simplify the function f(a,b,c,d) whose K-map is shown at the right.
- f = a'c'd+ab'+cd'+a'bc'

  or
- f = a'c'd+ab'+cd'+a'bd'
- The middle two terms are EPIs, while the first and last terms are selected to cover the minterms m_1, m_4, and m_5.

0	1	0	1
1	1	0	1
0	0	d	d
1	1	d	d

0	1	0	1
1	1	0	1
0	0	d	d
1	1	d	d

Another Example

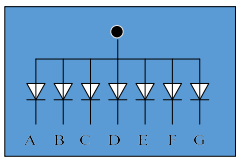
- Simplify the function g(a,b,c,d) whose K-map is shown at right.
- g = a'c'+ ab
  or
- g = a'c'+b'd

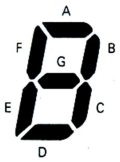
ab \ cd				
	d	1	0	0
	1	d	0	d
	1	d	d	1
	0	d	d	0

d	1	0	0
1	d	0	d
1	d	d	1
0	d	d	0

d	1	0	0
1	d	0	d
1	d	d	1
0	d	d	0

Application Example

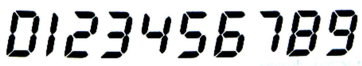




TIL 312
TIL 314
TIL 315

Pin 1	Cathode A
Pin 2	Cathode F
Pin 3	Anode Digit and Decimal
Pin 4	Omitted
Pin 5	Omitted
Pin 6	Cathode Left Decimal
Pin 7	Cathode E
Pin 8	Cathode D
Pin 9	Cathode Right Decimal
Pin 10	Cathode C
Pin 11	Cathode G
Pin 12	Omitted
Pin 13	Cathode B
Pin 14	Anode Digit and Decimal

Pin 3 is internally connected to Pin 14



Application Example

TRUTH TABLE

2421 Decimal	Min-max Term Decimal	Inputs w	x	y	z	Outputs A	B	C	D	E	F	G
0	0	0	0	0	0	0	0	0	0	0	0	1
1	1	0	0	0	1	1	0	0	1	1	1	1
2	8	1	0	0	0	0	0	1	0	0	1	0
3	9	1	0	0	1	0	0	0	0	1	1	0
4	10	1	0	1	0	1	0	0	1	1	0	0
5	11	1	0	1	1	0	1	0	0	1	0	0
6	12	1	1	0	0	0	1	0	0	0	0	0
7	13	1	1	0	1	0	0	0	1	1	1	1
8	14	1	1	1	0	0	0	0	0	0	0	0
9	15	1	1	1	1	0	0	0	0	1	0	0

Application Example

A=∑(1,10)
B=∑(11,12)
C=∑(8)
D=∑(1,10,13)
E=∑(1,9,10,11,13,15)
F=∑(1,8,9,13)
G=∑(0,1,13)

→

A=w'z+x'yz'
B=xy'z'+x'yz
C=wx'y'z'
D=xy'z+x'yz'+w'z
E=x'y+z
F=wx'y'+y'z
G=w'+xy'z

→

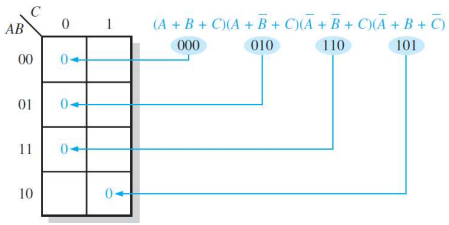
A=[(w'z)'(x'yz')]'
B=[(xy'z')'(x'yz)']
C=(wx'y'z')'
D=[(xy'z)'(x'yz')'(w'z)']'
E=[(x'y)'(z)']'
F=[(wx'y')'(y'z)']'
G=[(w)(xy'z)']'

Karnaugh Map POS Minimization

- Loading and grouping maxterms is exactly the same as loading and grouping minterms, except that 0s are loaded into the map and grouped to form PI and EPI groups.

Step 1: Determine the binary value of each sum term in the standard POS expression. This is the binary value that makes the term equal to 0.

Step 2: As each sum term is evaluated, place a 0 on the Karnaugh map in the corresponding cell.



Karnaugh Map POS Minimization

- Examples:
  - Map the following standard POS expression on a Karnaugh map:
  - 1.

$$(\overline{A} + \overline{B} + C + D)(\overline{A} + B + \overline{C} + \overline{D})(A + B + \overline{C} + D)(\overline{A} + \overline{B} + \overline{C} + \overline{D})(A + B + \overline{C} + \overline{D})$$

  - 2.

$$(A + \overline{B} + \overline{C} + D)(A + B + C + \overline{D})(A + B + C + D)(\overline{A} + B + \overline{C} + D)$$

Karnaugh Map POS Minimization

- The process for minimizing a POS expression is basically the same as for an SOP expression, except that you group 0s to produce minimum sum terms instead of grouping 1s to produce minimum product terms.
- Use a Karnaugh map to minimize the following standard POS expression
- 1.

$$(A + B + C)(A + B + \overline{C})(A + \overline{B} + C)(A + \overline{B} + \overline{C})(\overline{A} + \overline{B} + C)$$

- 2.

$$(B + C + D)(A + B + \overline{C} + D)(\overline{A} + B + C + \overline{D})(A + \overline{B} + C + D)(\overline{A} + \overline{B} + C + D)$$

Exp. F(a,b,c,d)= Π(0,4,5,7,8,9,11,12,13)

POS Example

F(x,y,z) = Π(0,3,4,7)

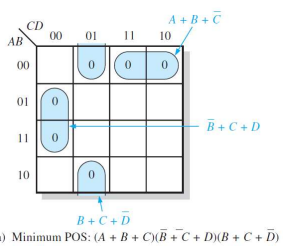
1. Using POS simplification
2. Using SOP simplification

F(a,b,c,d) = Π(1,5,7,8,10,13) + Πd(9, 11, 15)

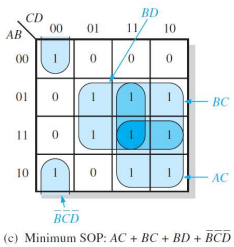
Converting Between POS and SOP Using the Karnaugh Map

- Using a Karnaugh map, convert the following standard POS expression into a minimum POS expression, a standard SOP expression, and a minimum SOP expression

$$(\overline{A} + \overline{B} + C + D)(A + \overline{B} + C + D)(A + B + C + \overline{D})(A + B + \overline{C} + \overline{D})(\overline{A} + B + C + \overline{D})(A + B + \overline{C} + D)$$



(a) Minimum POS: $(A + B + C)(\overline{B} + \overline{C} + D)(B + C + \overline{D})$



(c) Minimum SOP: $AC + BC + BD + \overline{B}\overline{C}\overline{D}$

Converting Between POS and SOP Using the Karnaugh Map

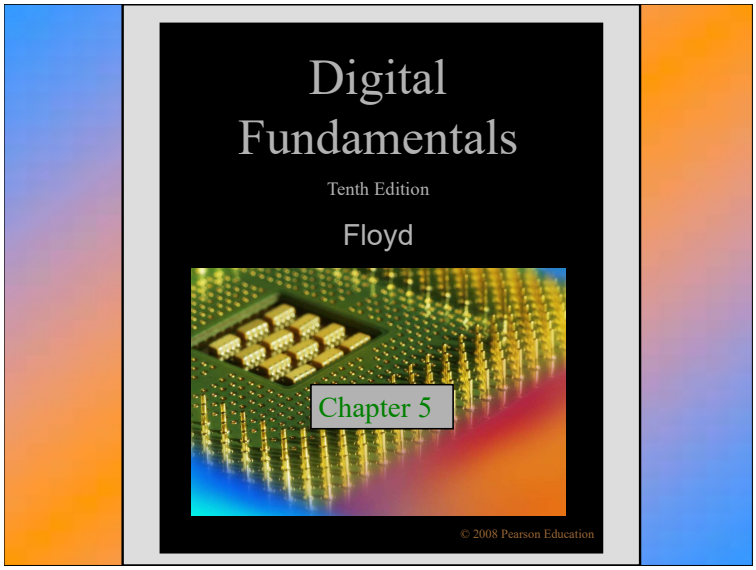
- Exp. Use a Karnaugh map to convert the following expression to minimum SOP form:

$$(W + \overline{X} + Y + \overline{Z})(\overline{W} + X + \overline{Y} + \overline{Z})(\overline{W} + \overline{X} + \overline{Y} + Z)(\overline{W} + \overline{X} + \overline{Z})$$

Homework

Page 143:
24, 32, 35,41,

46,47,44(a,c,e),52



Combinational Circuits

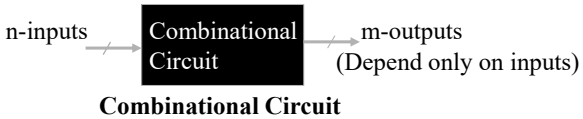
- A combinational circuit consists of logic gates whose outputs, at any time, are determined by combining the values of the inputs.
- For *n* input variables, there are 2^n possible binary input combinations.
- For each binary combination of the input variables, there is one possible binary value on each output.

Combinational Circuits (cont.)

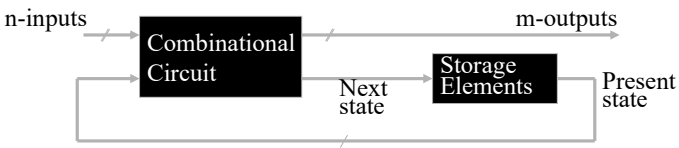
- Hence, a combinational circuit can be described by:

$$Y_1=f_1(x_1,x_2,\ldots,x_n)$$
$$Y_2=f_2(x_1,x_2,\ldots,x_n)$$
$$\vdots$$
$$Y_m=f_m(x_1,x_2,\ldots,x_n)$$

Combinational vs. Sequential Circuits



Combinational Circuit

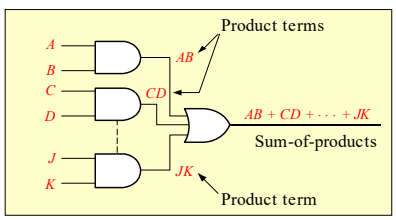


Sequential Circuit

Summary

Combinational Logic Circuits

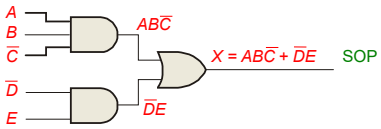
In Sum-of-Products (SOP) form, basic combinational circuits can be directly implemented with AND-OR combinations if the necessary complement terms are available.



Summary

Combinational Logic Circuits

An example of an SOP implementation is shown. The SOP expression is an AND-OR combination of the input variables and the appropriate complements.



Summary

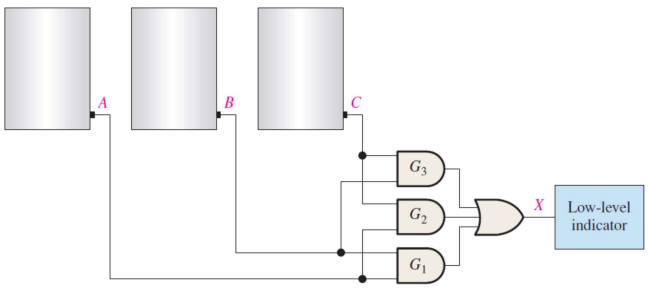
Example 5.1

In a certain chemical-processing plant, a liquid chemical is used in a manufacturing process. The chemical is stored in three different tanks. A level sensor in each tank produces a HIGH voltage when the level of chemical in the tank drops below a specified point.

Design a circuit that monitors the chemical level in each tank and indicates when the level in any two of the tanks drops below the specified point

Summary

Example 5.1

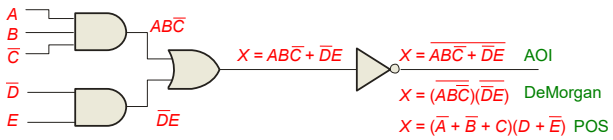


Summary

Combinational Logic Circuits

When the output of a SOP form is inverted, the circuit is called an AND-OR-Invert circuit. The AOI configuration lends itself to product-of-sums (POS) implementation.

An example of an AOI implementation is shown. The output expression can be changed to a POS expression by applying DeMorgan's theorem twice.



Summary

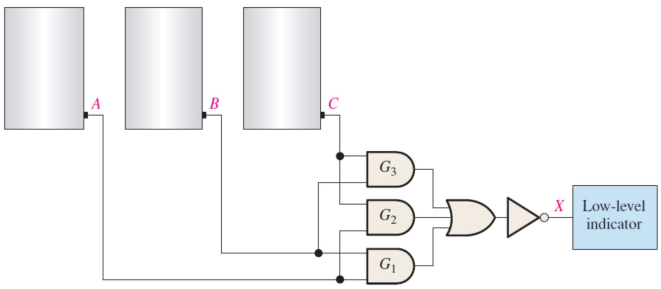
Example 5.2

The sensors in the chemical tanks of Example 5–1 are being replaced by a new model that produces a LOW voltage instead of a HIGH voltage when the level of the chemical in the tank drops below a critical point.

Modify the circuit in Figure 5–2 to operate with the different input levels and still produce a HIGH output to activate the indicator when the level in any two of the tanks drops below the critical point. Show the logic diagram

Summary

Example 5.2



Summary

Exclusive-OR Logic

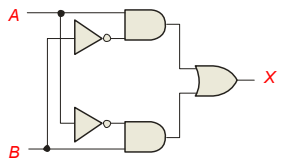
The truth table for an exclusive-OR gate is

Inputs		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

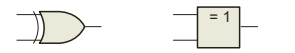
Notice that the output is HIGH whenever A and B disagree.

The Boolean expression is $X = \overline{A}B + A\overline{B}$

The circuit can be drawn as



Symbols:



Distinctive shape — Rectangular outline

Summary

Exclusive-NOR Logic

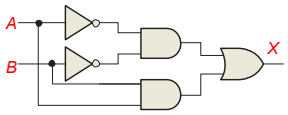
The truth table for an exclusive-NOR gate is

Inputs		Output
A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

Notice that the output is HIGH whenever A and B agree.

The Boolean expression is $X = \overline{A}\overline{B} + AB$

The circuit can be drawn as



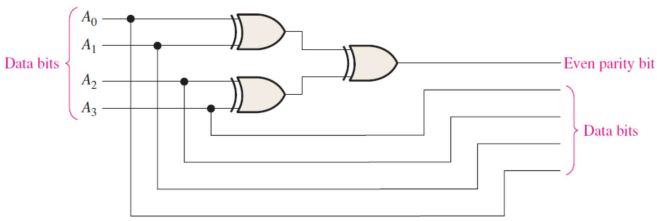
Symbols:

Distinctive shape — Rectangular outline

Summary

Example 5.3

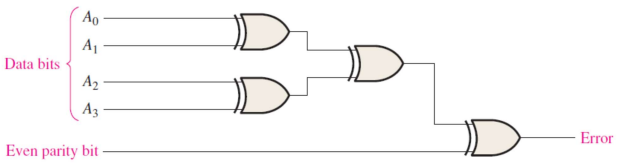
Use exclusive-OR gates to implement an even-parity code generator for an original 4-bit code



Summary

Example 5.4

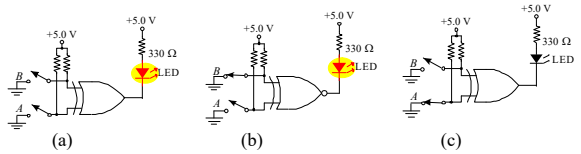
Use exlusive-OR gates to implement an even-parity checker for the 5-bit code generated by the circuit in Example 5–3.



Summary

Example

For each circuit, determine if the LED should be on or off.



Solution

Circuit (a): XOR, inputs agree, output is LOW, LED is ON.

Circuit (b): XNOR, inputs disagree, output is LOW, LED is ON.

Circuit (c): XOR, inputs disagree, output is HIGH, LED is OFF.

Summary

Implementing Combinational Logic

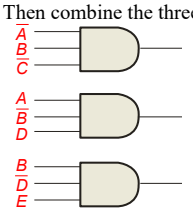
Implementing a SOP expression is done by first forming the AND terms; then the terms are ORed together.

Example

Show the circuit that will implement the Boolean expression $X = \overline{A}B\overline{C} + A\overline{B}D + B\overline{D}E$. (Assume that the variables and their complements are available.)

Solution

Start by forming the terms using three 3-input AND gates. Then combine the three terms using a 3-input OR gate.



$X = \overline{A}B\overline{C} + A\overline{B}D + B\overline{D}E$

Summary

Example 5.5

Design a logic circuit to implement the operation specified in the truth table of Table 5–4.

TABLE 5–4

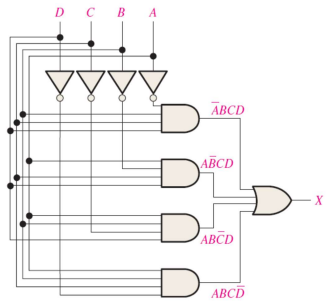
Inputs			Output	
A	*B*	*C*	*X*	Product Term
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	1	$\overline{A}BC$
1	0	0	0	
1	0	1	1	$A\overline{B}C$
1	1	0	1	$AB\overline{C}$
1	1	1	0	

Summary

Example 5.6

Develop a logic circuit with four input variables that will only produce a 1 output when exactly three input variables are 1s.

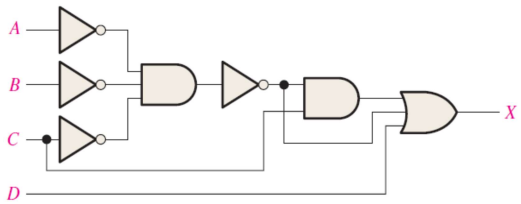
Determine if the logic circuit can be simplified



Summary

Example 5.7

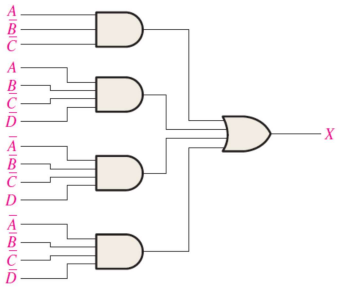
Reduce the combinational logic circuit in Figure 5–14 to a minimum form



Summary

Example 5.7

Minimize the combinational logic circuit in Figure 5–16. Inverters for the complemented variables are not shown.

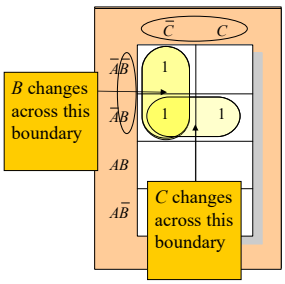


Summary

Karnaugh Map Implementation

For basic combinational logic circuits, the Karnaugh map can be read and the circuit drawn as a minimum SOP.

Example A Karnaugh map is drawn from a truth table. Read the minimum SOP expression and draw the circuit.



Solution

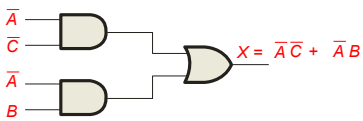
1. Group the 1's into two overlapping groups as indicated.
2. Read each group by eliminating any variable that changes across a boundary.
3. The vertical group is read $\overline{A}\,\overline{C}$.
4. The horizontal group is read $\overline{A}B$.

The circuit is on the next slide:

Summary

Solution *continued…*

Circuit:



The result is shown as a sum of products.

It is a simple matter to implement this form using only NAND gates as shown in the text and following example.

Functionally Complete Operation Sets(完备运算集)

- Functionally Complete Operation Sets is a set of logic functions from which any combinational logic expression can be realized.
- FC1={AND,OR,NOT}
- FC2={NOR}
- FC3={NAND}
- FC4={EXOR,AND}

- $xy=\overline{\overline{xy}}$, $x+y=\overline{\overline{x}\,\overline{y}}$, $\overline{x}=\overline{xx}$ (NAND)
- $xy=\overline{\overline{x}+\overline{y}}$, $x+y=\overline{\overline{x+y}}$, $\overline{x}=\overline{x+0}$ (NOR)
- $\overline{x}=1\oplus x$, $x+y=x\oplus y\oplus xy$ (EXOR, AND)

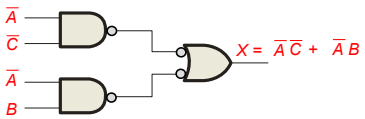
Summary

NAND Logic

Example Convert the circuit in the previous example to one that uses only NAND gates.

Solution

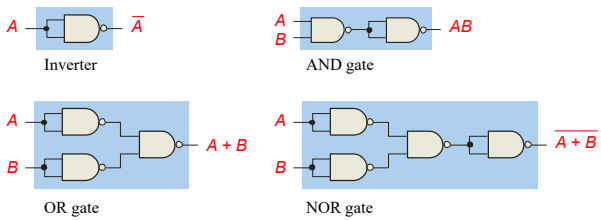
Recall from Boolean algebra that double inversion cancels. By adding inverting bubbles to above circuit, it is easily converted to NAND gates:



Summary

Universal Gates

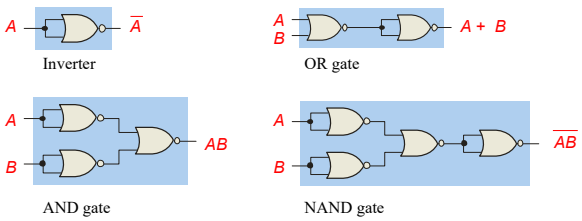
NAND gates are sometimes called **universal** gates because they can be used to produce the other basic Boolean functions.



Summary

Universal Gates

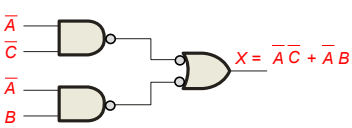
NOR gates are also **universal** gates and can form all of the basic gates.



Summary

Combinational Logic Using NAND and NOR Gates

Recall from DeMorgan's theorem that $\overline{AB} = \overline{A} + \overline{B}$. By using equivalent symbols, it is simpler to read the logic of SOP forms. The earlier example shows the idea:

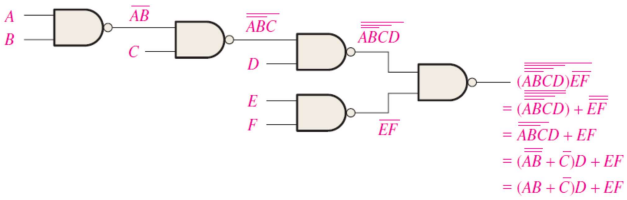


The logic is easy to read if you (mentally) cancel the two connected bubbles on a line.

Summary

Combinational Logic Using NAND and NOR Gates

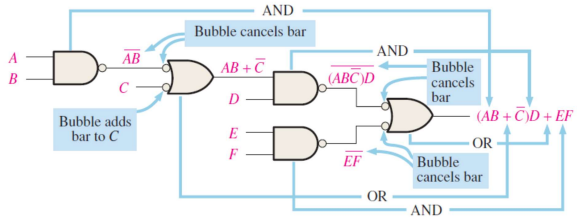
- The NAND symbol and the **negative-OR** symbol are called *dual symbols*.
- When drawing a NAND logic diagram, a bubble output should not be connected to a nonbubble input or vice versa



Summary

Combinational Logic Using NAND and NOR Gates

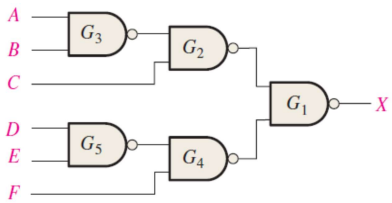
Although using all NAND symbols is correct, the diagram in the following is much easier to "read" and is the preferred method.



Summary

Example5.10

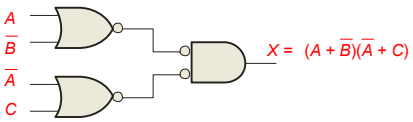
Redraw the logic diagram and develop the output expression for the circuit using the appropriate dual symbols.



Summary

NOR Logic Diagram Using Dual Symbols

Alternatively, DeMorgan's theorem can be written as $\overline{A + B} = \overline{A}\,\overline{B}$. By using equivalent symbols, it is simpler to read the logic of POS forms. For example,

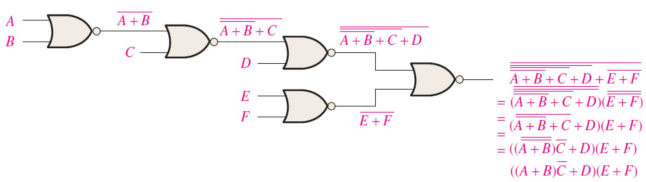


Again, the logic is easy to read if you cancel the two connected bubbles on a line.

Summary

NOR Logic Diagram Using Dual Symbols

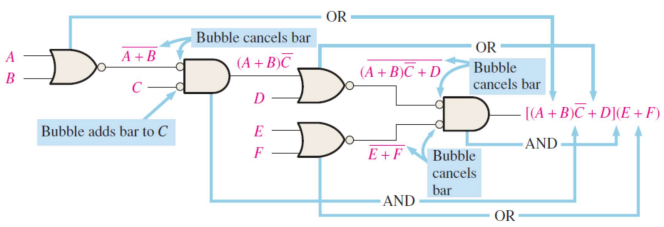
As with NAND logic, the purpose for using the dual symbols is to make the logic diagram easier to read and analyze



Summary

NOR Logic Diagram Using Dual Symbols

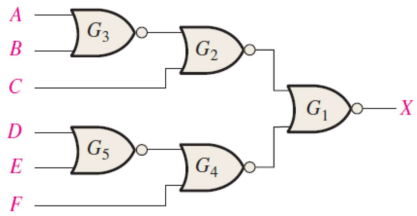
Output expression can be obtained directly from the function of each gate symbol in the diagram



Summary

Example5.11

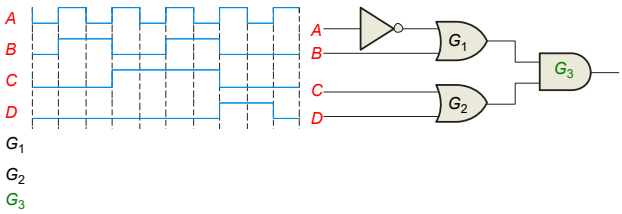
Using appropriate dual symbols, redraw the logic diagram and develop the output expression for the circuit



Summary

Pulsed Waveforms

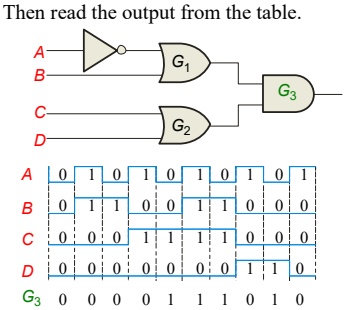
For combinational circuits with pulsed inputs, the output can be predicted by developing intermediate outputs and combining the result. For example, the circuit shown can be analyzed at the outputs of the OR gates:



Summary

Pulsed Waveforms

Alternatively, you can develop the truth table for the circuit and enter 0's and 1's on the waveforms. Then read the output from the table.



Inputs				Output
A	*B*	*C*	*D*	*X*
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Summary

Homework

P169

12, 14, 24(b,d,g), 25(b,d,g)

Quiz

1. Assume an AOI expression is $\overline{AB + CD}$. The equivalent POS expression is

a. $(A + B)(C + D)$

b. $(\overline{A} + \overline{B})(\overline{C} + \overline{D})$

c. $\overline{(A + B)(C + D)}$

d. none of the above

© 2008 Pearson Education

Quiz

2. The truth table shown is for

a. a NAND gate

b. a NOR gate

c. an exclusive-OR gate

d. an exclusive-NOR gate

Inputs		Output
A	*B*	*X*
0	0	1
0	1	0
1	0	0
1	1	1

© 2008 Pearson Education

Quiz

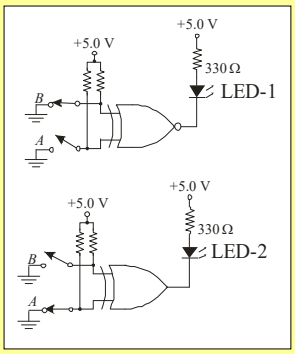
3. An LED that should be ON is

a. LED-1

b. LED-2

c. neither

d. both



© 2008 Pearson Education

Quiz

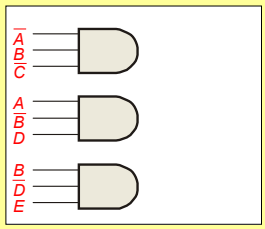
4. To implement the SOP expression $X = \overline{A}B\overline{C} + A\overline{B}D + B\overline{D}E$, the type of gate that is needed is a

a. 3-input AND gate

b. 3-input NAND gate

c. 3-input OR gate

d. 3-input NOR gate



© 2008 Pearson Education

Quiz

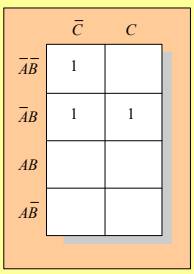
5. Reading the Karnaugh map, the logic expression is

a. $\overline{A}\,\overline{C} + \overline{A}B$

b. $\overline{A}B + A\overline{C}$

c. $A\overline{B} + B\overline{C}$

d. $\overline{A}B + \overline{A}\,\overline{C}$



© 2008 Pearson Education

Quiz

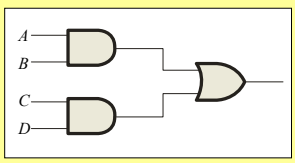
6. The circuit shown will have identical logic out if all gates are changed to

a. AND gates

b. OR gates

c. NAND gates

d. NOR gates



© 2008 Pearson Education

Quiz

7. The two types of gates which are called *universal gates* are

a. AND/OR

b. NAND/NOR

c. AND/NAND

d. OR/NOR

© 2008 Pearson Education

Quiz

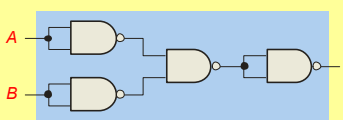
8. The circuit shown is equivalent to an

a. AND gate

b. XOR gate

c. OR gate

d. none of the above



© 2008 Pearson Education

Quiz

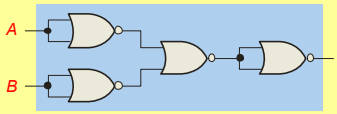
9. The circuit shown is equivalent to

a. an AND gate

b. an XOR gate

c. an OR gate

d. none of the above



© 2008 Pearson Education

Quiz

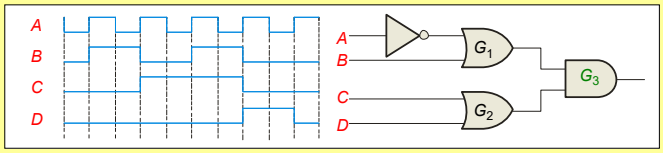
10. During the first *three* intervals for the pulsed circuit shown, the output of

a. G_1 is LOW and G_2 is LOW

b. G_1 is LOW and G_2 is HIGH

c. G_1 is HIGH and G_2 is LOW

d. G_1 is HIGH and G_2 is HIGH



© 2008 Pearson Education

Quiz

Answers:

1. b	6. c
2. d	7. b
3. a	8. c
4. c	9. a
5. d	10. c

CHAPTER 6

Functions of Combinational Logic

(Section 6.1-6.4)

Overview

- Binary Addition
  - Half Adder
  - Full Adder
  - Ripple Carry Adder(并行加法器)
  - Carry Look ahead Adder(超前进位加法器)
- Decimal Addition
- Binary Comparators (Section 6.4)

1-bit Adder

- Performs the addition of two binary bits.
- Four possible operations:
  - 0+0=0
  - 0+1=1
  - 1+0=1
  - 1+1=10
- Circuit implementation requires 2 outputs; one to indicate the ***sum*** and another to indicate the ***carry***.

Half Adder

- Performs 1-bit addition.
- Inputs: A_0, B_0
- Outputs: S_0, C_1
- Index indicates significance, 0 is for LSB and 1 is for the next higher significant bit.
- Boolean equations:
  - $S_0 = A_0B_0' + A_0'B_0 = A_0 \oplus B_0$
  - $C_1 = A_0B_0$

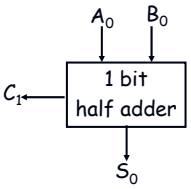
Truth Table

A_0	B_0	S_0	C_1
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

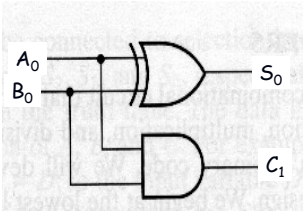
Half Adder (cont.)

- $S_0 = A_0B_0' + A_0'B_0 = A_0 \oplus B_0$
- $C_1 = A_0B_0$

Block Diagram



Logic Diagram



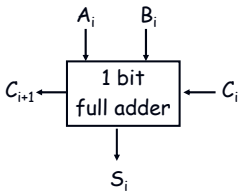
n-bit Addition

- Design an n-bit binary adder which performs the addition of two n-bit binary numbers and generates a n-bit sum and a carry out.
- Example: Let n=4

C_{out}	C_3	C_2	C_1	C_0	1	1	0	1	0
	A_3	A_2	A_1	A_0		1	1	0	1
+	B_3	B_2	B_1	B_0		+1	1	0	1
	S_3	S_2	S_1	S_0		1	0	1	0

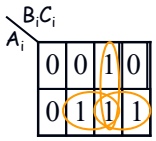
Full Adder

- Full adder (for higher-order bit addition)
- Combinational circuit that performs the additions of 3 bits (two bits and a carry-in bit)

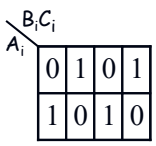


Full Adder (cont.)

- The K-maps for
  - C_{i+1}:



  - S_i:



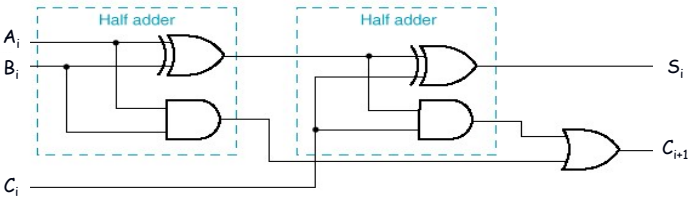
A_i	B_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Full Adder (cont.)

- Boolean equations:
  - $C_{i+1} = A_iB_i + A_iC_i + B_iC_i$
  - $S_i = A_iB_i'C_i' + A_i'B_i'C_i + A_i'B_iC_i' + A_iB_iC_i$
    $= A_i \oplus B_i \oplus C_i$
- You can design full adder circuit ***directly*** from the above equations (requires 3 ANDs and 1 OR for C_{i+1} and 2 XORs for S_i)

Full Adder using 2 Half Adders

- A full adder can also be realized with two half adders and an OR gate, since C_{i+1} can also be expressed as:
- $C_{i+1} = A_iB_i + A_iB_i'C_i + A_i'B_iC_i$
  $= A_iB_i + (A_iB_i' + A_i'B_i)C_i$
  $= A_iB_i + (Ai \oplus B_i)C_i$
- and $S_i = A_i \oplus B_i \oplus C_i$



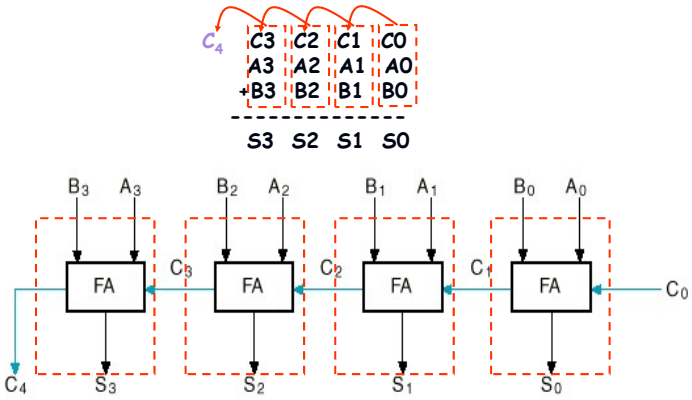
n-bit Combinational Adders

- Perform ***parallel*** multi-bit addition
- Ripple Carry Adder
  - Simple design
  - Time consuming. Why? (you'll see in a bit!)
- Carry Look ahead Adder
  - More complex than ripple-carry adder
  - Reduces circuit delay

n-bit Ripple Carry Adder

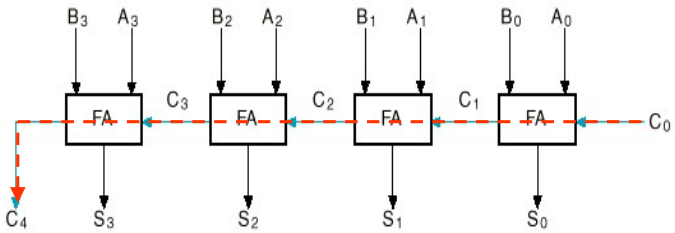
- Constructed using ***n*** 1-bit full adder blocks in parallel.
- Cascade the full adders so that the carry out from one becomes the carry in to the next higher bit position.

Example: 4-bit Ripple Carry Adder



Ripple Carry Adder Delay

- Circuit delay in an n-bit ripple carry adder is determined by the delay on the carry path from the LSB (C_0) to the MSB (C_n).
- Let the delay in a 1-bit FA be Δ. Then, the delay of an n-bit ripple carry adder is nΔ.

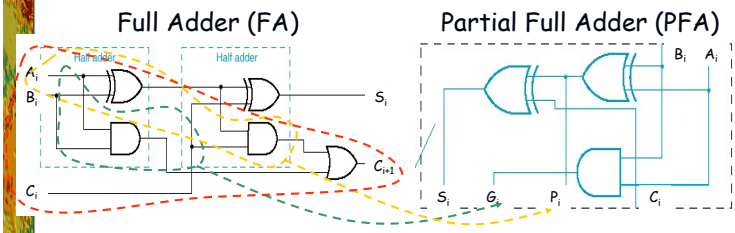


Carry Look ahead Adder

- Alternative design for a combinational n-bit adder.
- Practical design with reduced delay at the expense of more complex hardware.
- Derived from a transformation of the ripple carry adder design.

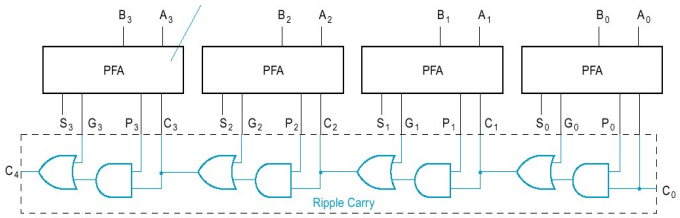
Carry Lookahead Adder Design

- From a FA, separate between carry ***generation*** (a new carry signal is generated, i.e. C_{out}=1) and carry ***propagation*** (an existing C_{in} is propagated to C_{out})
- Generate: $G_i = A_iB_i$: if 1, C_{i+1}=1
- Propagate: $P_i = A_i \oplus B_i$: if 1, $C_{i+1} = C_i$



Carry Lookahead Adder Design (cont.)

- $C_{i+1} = G_i + P_iC_i$
- PFA design breaks S functionality apart from G/P functionality



Carry Look ahead Adder (cont.)

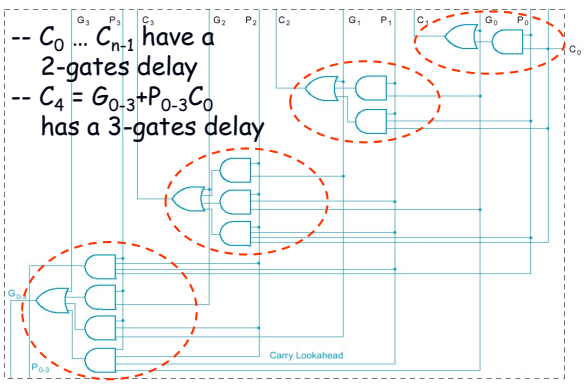
- Does this (design in previous slide) solve the long delay problem?
- No, carry out still "ripples" !
- Idea: use two levels of logic to generate carry out of any block C_i in terms of carry in C_0 and addend bits A_i and B_i

Block CLA

- Implement:
  - $C_1 = G_0 + P_0 C_0$
  - $C_2 = G_1 + P_1C_1 = G_1 + P_1(G_0 + P_0C_0) = G_1 + P_1G_0 + P_1P_0C_0$
  - $C_3 = G_2 + P_2C_2 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$
  - $C_4 = \boxed{G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0} + \boxed{P_3P_2P_1P_0}C_0$
    $= G_{0\text{-}3} + P_{0\text{-}3}C_0$

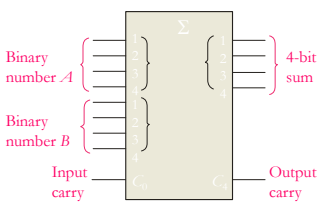
Group carry generate *Group carry propagate*

Generate/Propagate logic of a 4-bit CLA



Using MSI Adders as Subtractors

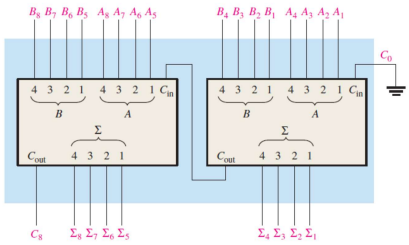
- 74LS283 : 4Bits carry look ahead adder device
  - The logic symbol for a 4-bit parallel adder is shown. This 4-bit adder includes a carry in (labeled (C_0) and a Carry out (labeled C_4).



  - The 74LS283 is an example. It features look-ahead carry, which adds logic to minimize the output carry delay. For the 74LS283, the maximum delay to the output carry is 17 ns.

Using MSI Adders as Subtractors

- Cascading four 74LS283 two-bit adders to form 1 8bit adder (Fig. 6-12)

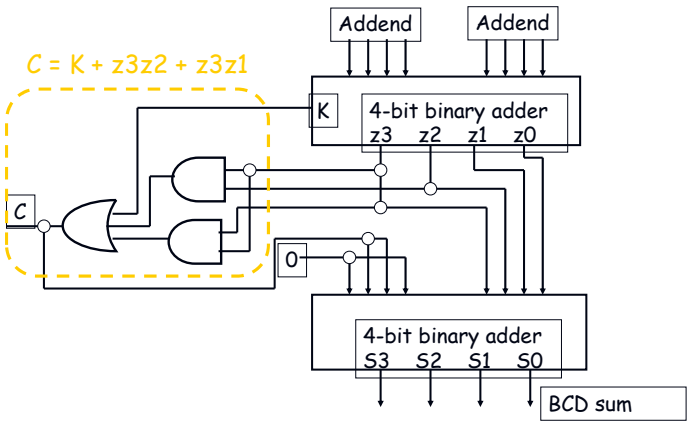


- Subtractor
  - Using 2s complement to realize subtraction

BCD Adder

- BCD Decimal Adder:
  - Requires 8 inputs (4 bits per decimal number)
  - 5 outputs indicate the decimal sum and the carry
  - Remember BCD addition rules: Add 0110 to the sum if it is greater than 1010 to correct the carry bit

Binary Coded Decimal (BCD) Adder



Binary Comparators

- Single bit comparator
  - A EQ B = A′B′ + AB
  - A > B = AB′
  - A< B = A′B
- Two-bit comparator
- Iterative Design of multiple-bit comparators
  - Basic iterative circuit model
  - Truth table for a single cell of a binary comparator

- Homework

  P345: 6

CHAPTER 6

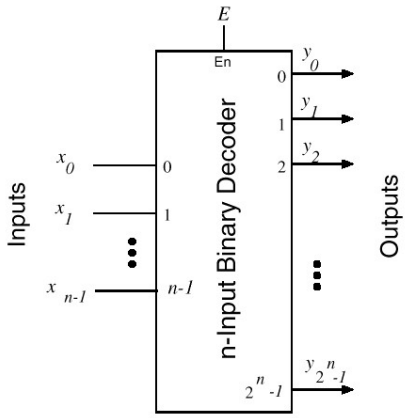
Combinational Logic– Encoders/Decoders

(Sections 4.5 – 4.6)

Decoders

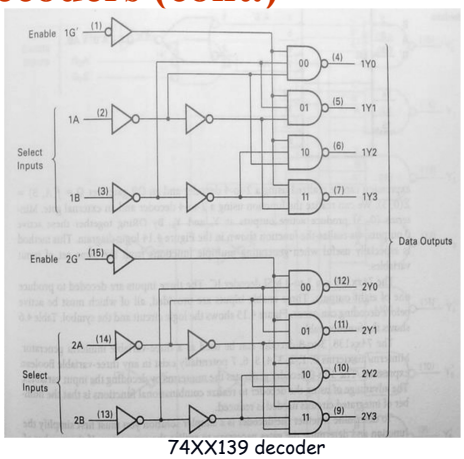
- A combinational circuit that converts binary information from *n* coded inputs to a maximum 2^n coded outputs
  → *n-to-* 2^n decoder
- *n-to-m* decoder, $m \leq 2^n$
- Examples: BCD-to-7-segment decoder, where *n=4* and *m=10*

Decoders (cont.)



Decoders (cont.)

$Y_0=(G'B'A')'=G+B+A$
$Y_1=(G'B'A)'=G+B+A'$
$Y_2=(G'BA')'=G+B'+A$
$Y_3=(G'BA)'=G+B'+A'$



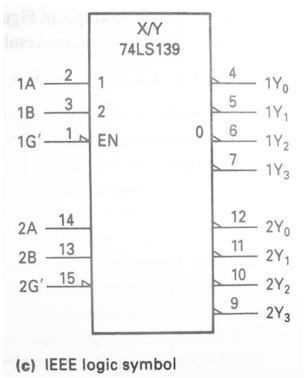
74XX139 decoder

Decoders (cont.)

$Y_0=(G'B'A')'=G+B+A$
$Y_1=(G'B'A)'=G+B+A'$
$Y_2=(G'BA')'=G+B'+A$
$Y_3=(G'BA)'=G+B'+A'$

Inputs			Outputs			
Enable G′	Select B	A	Y_0	Y_1	Y_2	Y_3
H	X	X	H	H	H	H
L	L	L	L	H	H	H
L	L	H	H	L	H	H
L	H	L	H	H	L	H
L	H	H	H	H	H	L

Decoders (cont.)

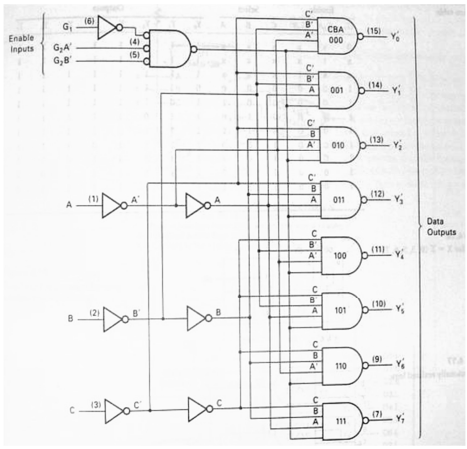


(c) IEEE logic symbol

3-to-8 Decoder (cont.)

- Three inputs, A_0, A_1, A_2, are decoded into eight outputs, Y_0 through Y_7
- Each output Y_i represents one of the minterms of the 3 input variables.
- $Y_i = 1$ when the binary number $A_2A_1A_0 = i$
- Shorthand: $Y_i = m_i$
- The output variables are *mutually exclusive*; exactly one output has the value 1 at any time, and the other seven are 0.

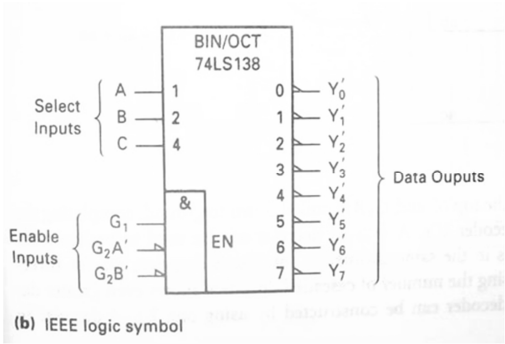
Decoders (cont.)



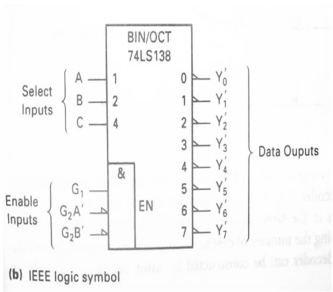
Decoders (cont.)

Inputs						Outputs							
Enable			Select										
G_1	G_2A'	G_2B'	C	B	A	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7
0	x	x	x	x	x	1	1	1	1	1	1	1	1
x	1	x	x	x	x	1	1	1	1	1	1	1	1
x	x	1	x	x	x	1	1	1	1	1	1	1	1
1	0	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	0	1	1	0	1	1	1	1	1	1
1	0	0	0	1	0	1	1	0	1	1	1	1	1
1	0	0	0	1	1	1	1	1	0	1	1	1	1
1	0	0	1	0	0	1	1	1	1	0	1	1	1
1	0	0	1	0	1	1	1	1	1	1	0	1	1
1	0	0	1	1	0	1	1	1	1	1	1	0	1
1	0	0	1	1	1	1	1	1	1	1	1	1	0

Decoders (cont.)



Implementing Boolean functions using decoders



$Y_0'=(C'B'A')'=m_0'$

$Y_1'=(C'B'A)'=m_1'$

$Y_2'=(C'BA')'=m_2'$

$Y_3'=(C'BA)'=m_3'$

$Y_4'=(CB'A')'=m_4'$

$Y_5'=(CB'A)'=m_5'$

$Y_6'=(CBA')'=m_6'$

$Y_7'=(CBA)'=m_7'$

Implementing Boolean functions using decoders

- E.g. use 74XX138,3-to-8 decoder to implement boolean functions

$Y_1=A'B'+AC+A'C'$

$Y_2=A'C+AC'$

$Y_3=B'C+BC'$

Implementing Boolean functions using decoders

$Y_1=A'B'+AC+A'C'=m_0+m_1+m_2+m_5+m_7$

$= (m_0'm_1'm_2'm_5'm_7')'$

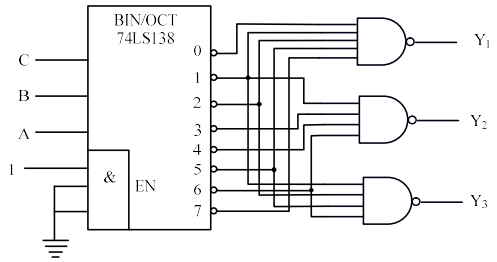
$Y_2=A'C+AC'= m_1+m_3+m_4+m_6$

$= (m_1'm_3'm_4'm_6')'$

$Y_3=B'C+BC'=m_1+m_2+m_5+m_6$

$= (m_1'm_2'm_5'm_6')'$

Implementing Boolean functions using decoders



$Y_1 = (m_0'm_1'm_2'm_5'm_7')'$

$Y_2 = (m_1'm_3'm_4'm_6')'$

$Y_3 = (m_1'm_2'm_5'm_6')'$

Implementing Boolean functions using decoders

$Y_1 = A'B' + AC + A'C' = m_0 + m_1 + m_2 + m_5 + m_7$

$= \sum(0,1,2,5,7)$

$= \prod(3,4,6) = M_3M_4M_6 = m_3'm_4'm_6'$

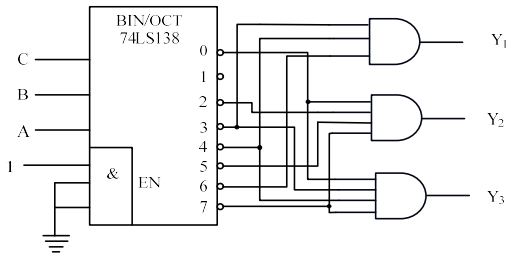
$Y_2 = A'C + AC' = m_1 + m_3 + m_4 + m_6$

$= m_0'm_2'm_5'm_7'$

$Y_3 = B'C + BC' = m_1 + m_2 + m_5 + m_6$

$= m_0'm_3'm_4'm_7'$

Implementing Boolean functions using decoders



$Y_1 = m_3'm_4'm_6'$

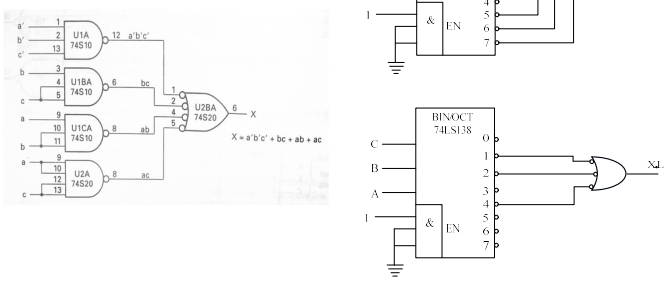
$Y_2 = m_0'm_2'm_5'm_7'$

$Y_3 = m_0'm_3'm_4'm_7'$

Implementing Boolean functions using decoders

- e.g. $X = f(a,b,c) = \sum(0,3,5,6,7)$

  $= a'b'c' + ab + bc + ac$



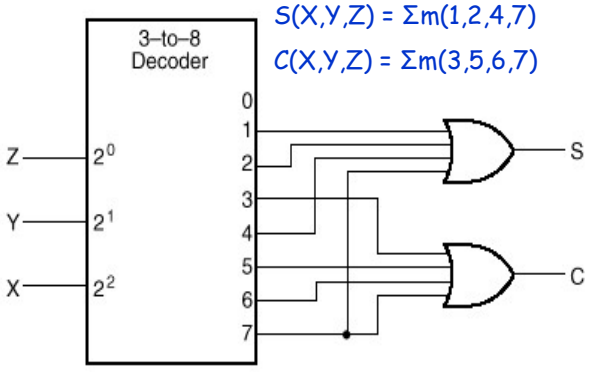
Implementing Boolean functions using decoders

- Here is another example:
  Recall full adder equations, and let X, Y, and Z be the inputs:
  - S(X,Y,Z) = Σm(1,2,4,7)
  - C (X,Y,Z) = Σm(3, 5, 6, 7).
- Since there are 3 inputs and a total of 8 minterms, we need a 3-to-8 decoder.

Implementing a Binary Adder Using a Decoder

$S(X,Y,Z) = \Sigma m(1,2,4,7)$

$C(X,Y,Z) = \Sigma m(3,5,6,7)$



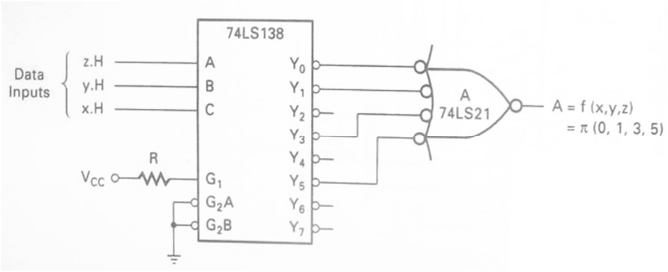
Implementing a Binary Adder Using a Decoder

- Exe. Using decoders realize functions

  $A=f(x,y,z)=\prod(0,1,3,5)$

  $G=f(x,y,z)=\sum(0,1,2,4,5,6,7)$

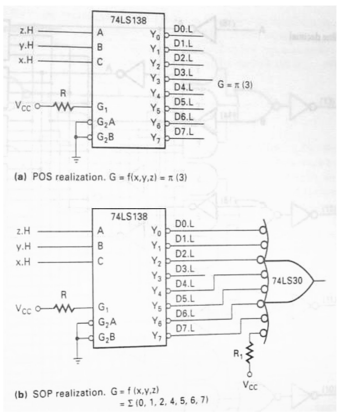
Implementing a Binary Adder Using a Decoder



$A=f(x,y,z)=\prod(0,1,3,5)$

Implementing a Binary Adder Using a Decoder

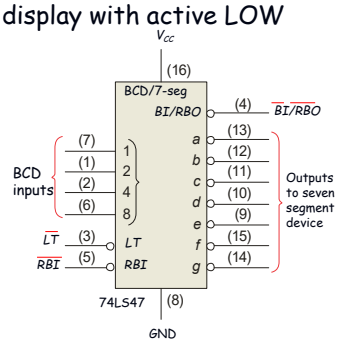
$G=f(x,y,z)=\sum(0,1,2,4,5,6,7)$
$=\prod(3)$



BCD Decoder/Driver

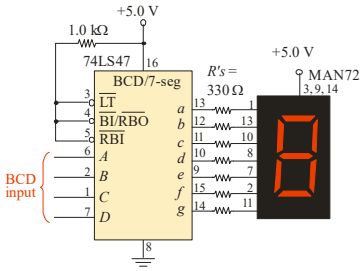
Another useful decoder is the 74LS47. This is a BCD-to-seven segment display with active LOW outputs.

The *a-g* outputs are designed for much higher current than most devices (hence the word driver in the name).



BCD Decoder/Driver

Here the 7447A is an connected to an LED seven segment display. Notice the current limiting resistors, required to prevent overdriving the LED display.

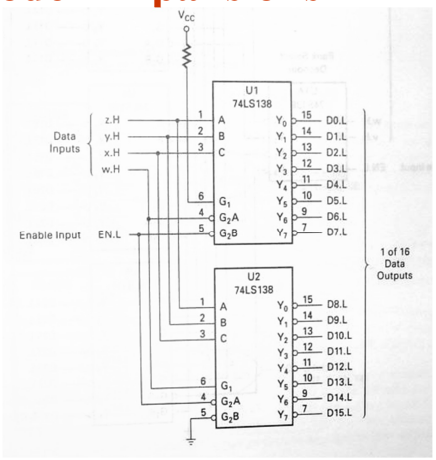


Decoder Expansions

- Larger decoders can be constructed using a number of smaller ones.
- HIERARCHICAL design!
- Example:
  A 6-to-64 decoder can be designed using four 4-to-16 and one 2-to-4 decoders. How? (Hint: Use the 2-to-4 decoder to generate the enable signals to the four 4-to-16 decoders).

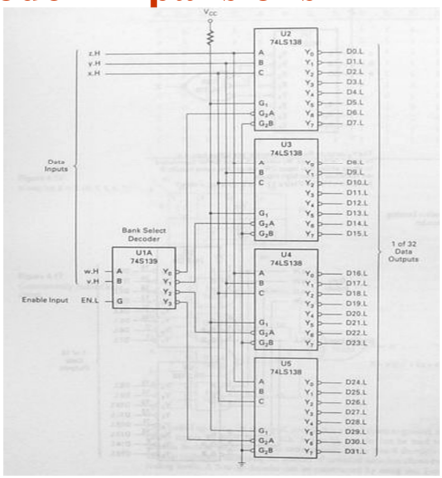
Decoder Expansions

Two 74XX138 decoders forming a single 4-to-16 decoder(P177)



Decoder Expansions

A 5-to-32 decoder using one 2-to-4 and four 3-to-8 decoder ICs(P178)

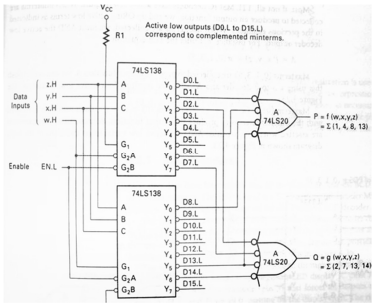


Decoder Expansions

- E.g. Using two 74XX138 decoders to realize a four-variable multiple output function(P179)

P=f(w,x,y,z)=Σ(1,4,8,13)

Q=f(w,x,y,z)=Σ(2,7,13,14)



Encoders

- An encoder is a digital circuit that performs the inverse operation of a decoder. An encoder has 2^n input lines and n output lines.
- The output lines generate the binary equivalent of the input line whose value is 1.

Encoders (cont.)

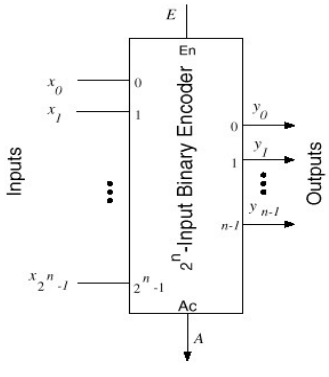


Figure 9.12: 2ⁿ-input binary encoder.

Encoder Example

- Example: 8-to-3 binary encoder (octal-to-binary)

Inputs								Outputs		
D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	A_2	A_1	A_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

$$A_0 = D_1 + D_3 + D_5 + D_7$$
$$A_1 = D_2 + D_3 + D_6 + D_7$$
$$A_2 = D_4 + D_5 + D_6 + D_7$$

Encoder Example (cont.)

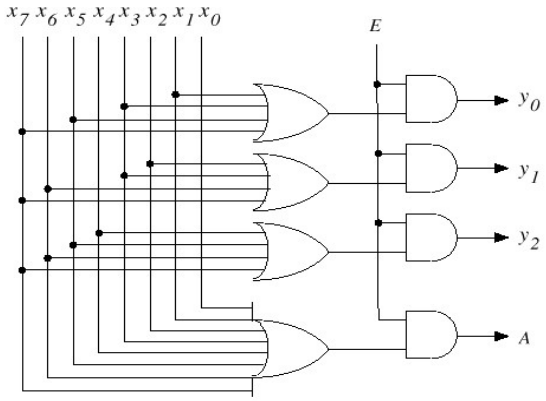


Figure mplementation of an 8-input binary encoder.

Priority Encoders

- Multiple asserted inputs are allowed; one has priority over all others.

Example: 4-to-2 Priority Encoder Truth Table

Inputs				Outputs		
D_3	D_2	D_1	D_0	A_1	A_0	V
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1

8-to-3 Priority Encoder

A priority encoder

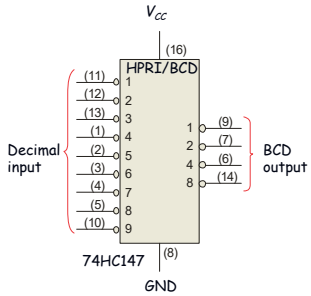
A_0	A_1	A_2	A_3	A_4	A_5	A_6	A_7	Z_0	Z_1	Z_2	NR
0	0	0	0	0	0	0	0	X	X	X	1
X	X	X	X	X	X	X	1	1	1	1	0
X	X	X	X	X	X	1	0	1	1	0	0
X	X	X	X	X	1	0	0	1	0	1	0
X	X	X	X	1	0	0	0	1	0	0	0
X	X	X	1	0	0	0	0	0	1	1	0
X	X	1	0	0	0	0	0	0	1	0	0
X	1	0	0	0	0	0	0	0	0	1	0
1	0	0	0	0	0	0	0	0	0	0	0

Encoders

The 74HC147 is an example of an IC encoder. It is has ten active-LOW inputs and converts the active input to an active-LOW BCD output.

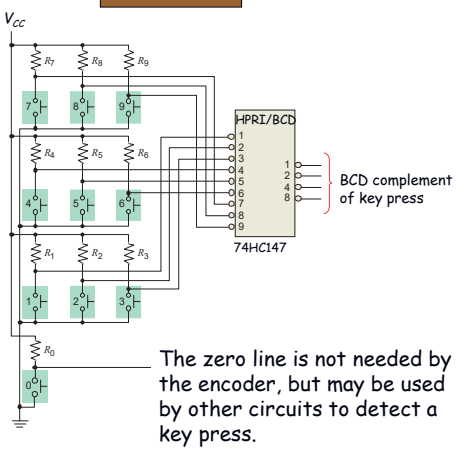
This device is offers additional flexibility in that it is a **priority encoder**. This means that if more than one input is active, the one with the highest order decimal digit will be active.

The next slide shows an application …



Encoders

Keyboard encoder



The zero line is not needed by the encoder, but may be used by other circuits to detect a key press.

Application of priority encoders (cont.)

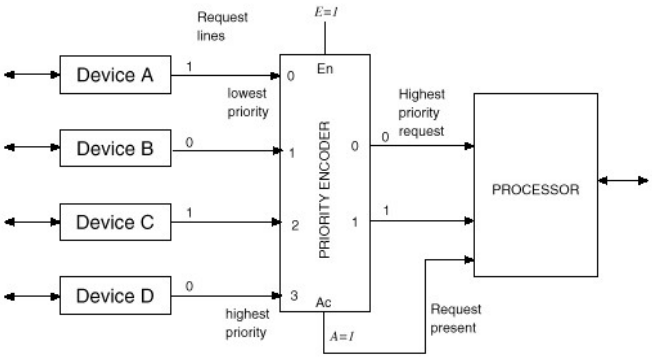


Figure Resolving interrupt requests using a priority encoder.

Homework

Using 3-8 decoders to implement following expressions

(a) Z = f(A,B,C) = A'B+BC'+BC+AB'C'

(b) Z = f(A,B,C) =A'•B' •C'+A'•B+A•B•C'+A•C

TO BE CONTINUED

CHAPTER 6

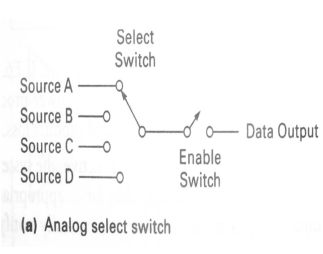
Combinational Logic Design – Multiplexers/Demultiplexers

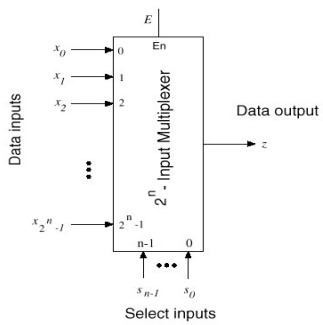
(Sections 6.8-6.10)

Multiplexer

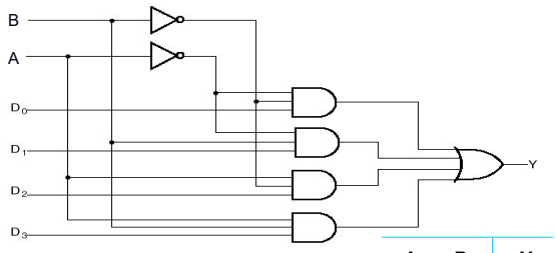
- **"Selects" binary information from one of many input lines and directs it to a single output line.**
- **Also know as the "selector" circuit,**
- **Selection is controlled by a particular set of inputs lines whose output depends on the combination of the data input lines.**
- **For a 2^n-to-1 multiplexer, there are 2^n data input lines and n selection lines whose bit combination determines which input is selected.**

Multiplexer (cont.)





4-to-1 MUX



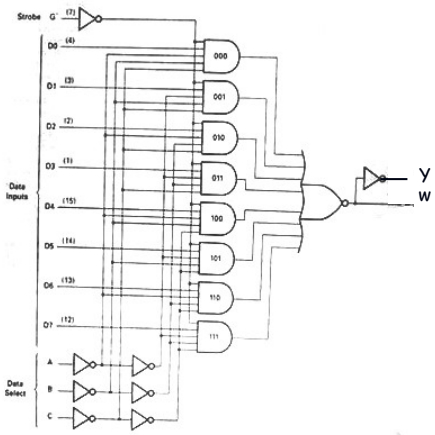
A	B	Y
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

$Y=A'B'D_0+A'BD_1+AB'D_2+ABD_3$

$=\sum_{i=0}^{2^n-1} m_i D_i$

8-to-1 MUX

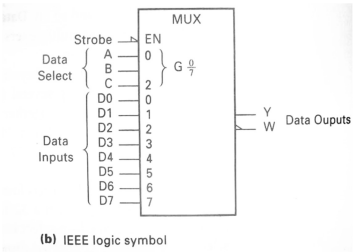
74LS155



8-to-1 MUX

$Y= C'B'A'D_0+ C'B'AD_1+ C'BA'D_2+C'BAD_3+ CB'A'D_4+ CB'AD_5+ CBA'D_6+ CBAD_7$

$=\sum_{i=0}^{i=2^n-1} m_i D_i$



(b) IEEE logic symbol

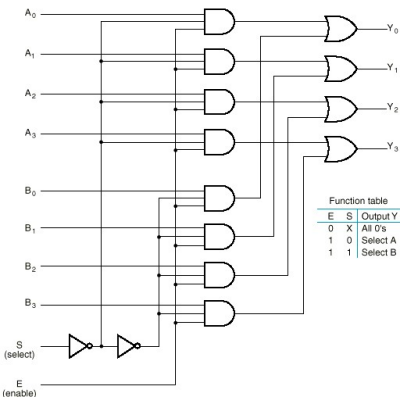
Inputs				Outputs	
Select			Strobe		
C	B	A	G′	Y	W
x	x	x	1	0	1
0	0	0	0	D0	D0′
0	0	1	0	D1	D1′
0	1	0	0	D2	D2′
0	1	1	0	D3	D3′
1	0	0	0	D4	D4′
1	0	1	0	D5	D5′
1	1	0	0	D6	D6′
1	1	1	0	D7	D7′

Multiplexer Expansions

- **Until now, we have examined single-bit data selected by a MUX. What if we want to select m-bit data/words?**
  → Combine MUX blocks in parallel with common select and enable signals
- **Example: Construct a logic circuit that selects between 2 sets of 4-bit inputs (see next slide for solution).**

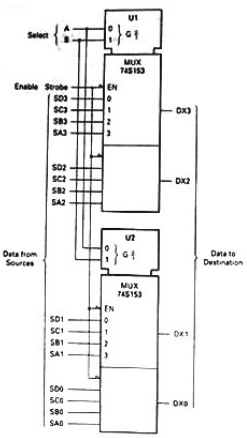
Example: Quad 2-to-1 MUX

- **Uses four 2-to-1 MUXs with common select (S) and enable (E).**
- **Select line chooses between A_i's and B_i's. The selected four-wire digital signal is sent to the Y_i's**
- **Enable line turns MUX on and off (E=1 is on).**



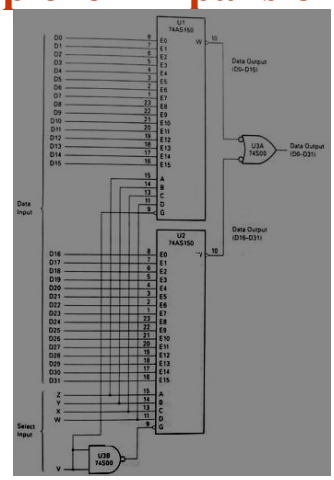
Example: Quad 4-to-1 MUX

74LS153(P194)



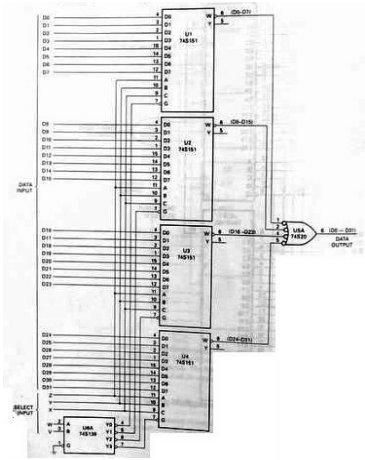
Multiplexer Expansions

A32-to-1multiplexer using two 74xx150ICs(P195)



Multiplexer Expansions

A 32-to-1 multiplexer using four 8-to-1 multiplexers and a 2-to-4 decoder(P196)



Implementing Boolean functions with Multiplexers

- **E.g. Using an 8-to-1 multiplexer to realize the Boolean function F=f(x,y,z)=∑(1,2,4,5,7)**

$$Y= \underline{C'B'A'}D_0+ \underline{C'B'A}D_1+ \underline{C'BA'}D_2+\underline{C'BA}D_3+ \underline{CB'A'}D_4+ \underline{CB'A}D_5+ \underline{CBA'}D_6+ \underline{CBA}D_7$$

$$=\sum_{i=0}^{i=2^n-1} m_i D_i$$

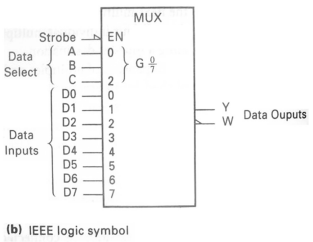
F=f(x,y,z)=∑(1,2,4,5,7)

=x'y'z+x'yz'+xy'z'+xy'z+xyz

C=x,B=y,A=z

$D_0=D_3=D_6=0$

$D_1= D_2= D_4= D_5= D_7=1$



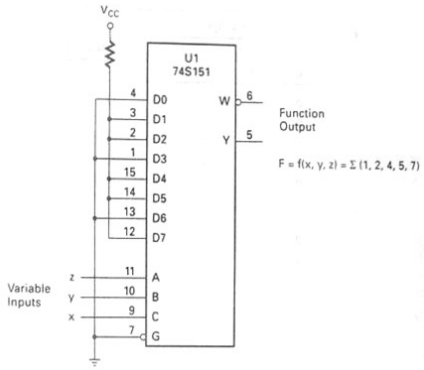
(b) IEEE logic symbol

Implementing Boolean functions with Multiplexers

C=x,B=y,A=z

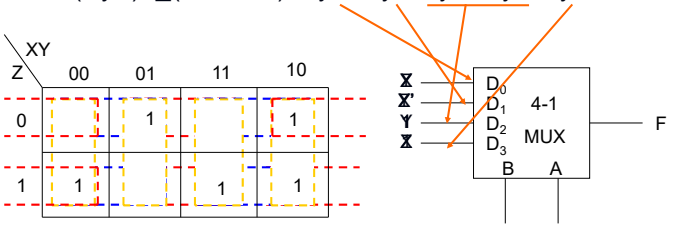
$D_0=D_3=D_6=0$

$D_1= D_2= D_4= D_5= D_7=1$



Using an 4-to-1 multiplexer to realize the Boolean function F=f(x,y,z)=∑(1,2,4,5,7)

F=f(x,y,z)=∑(1,2,4,5,7)=x'y'z+x'yz'+xy'z'+xy'z+xyz



$D_0=X$

$D_1=X'$

$D_2=Y'$

$D_3=X$

$F=X'Y'D_0+X'YD_1+XY'D_2+XYD_3$

Implementing Boolean functions with Multiplexers

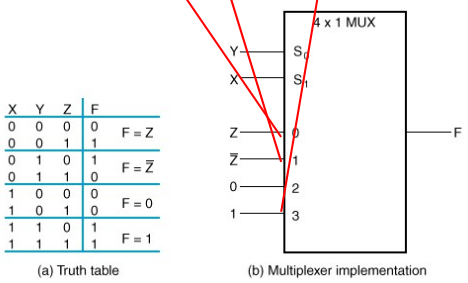
- Exe. implement function using a 4-to-1 MUX

F(X,Y,Z) = Σm(1,2,6,7)

F(A,B,C) = ∑m(1,3,5,6).

Implementing Boolean functions with Multiplexers

- F(X,Y,Z) = X'Y'Z + X'YZ' + XYZ' + XYZ = Σm(1,2,6,7)
- There are n=3 inputs, thus we need a 2^2-to-1 MUX
- The first n-1 (=2) inputs serve as the selection lines



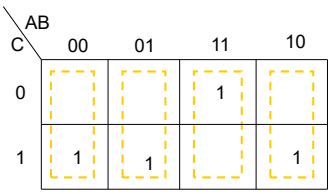
X	Y	Z	F	
0	0	0	0	F = Z
0	0	1	1	
0	1	0	1	F = $\bar{Z}$
0	1	1	0	
1	0	0	0	F = 0
1	0	1	0	
1	1	0	1	F = 1
1	1	1	1	

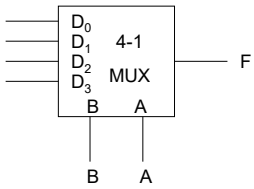
(a) Truth table

(b) Multiplexer implementation

Implementing Boolean functions with Multiplexers

F(A,B,C) = ∑m(1,3,5,6).





When A=B=0, $F=D_0=C$

When A=0, B=1, $F=D_1=C$

When A=1, B=0, $F=D_2=C$

When A=B=1, $F=D_3=C'$

Implementing Boolean functions with Multiplexers

When A=B=0, F=C

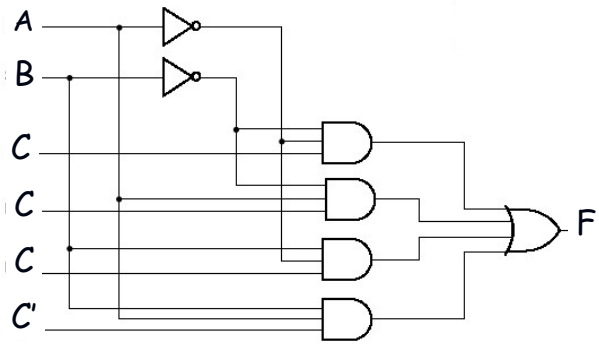
When A=0, B=1, F=C

When A=1, B=0, F=C

When A=B=1, F=C'

A	B	C		F
0	0	0		0
0	0	1		1
0	1	0		0
0	1	1		1
1	0	0		0
1	0	1		1
1	1	0		1
1	1	1		0

MUX implementation of F(A,B,C) = ∑m(1,3,5,6)



Implementing Boolean functions with Multiplexers

E.g. Consider the following Boolean expression given in sum-of-product form:

$F(x1,x2,x3)=x_1'x_2'+x_1x_2'+x_1x_3$

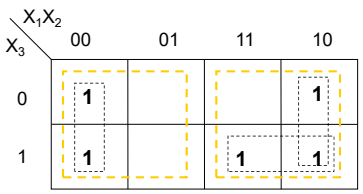
Derive a circuit for using only 2-to-1 multiplexers.

Implementing Boolean functions with Multiplexers

$F(x1,x2,x3)=x_1'x_2'+x_1x_2'+x_1x_3$

$= x_1'x_2'x_3' + x_1'x_2'x_3 + x_1x_2'x_3' + x_1x_2'x_3 + x_1x_2x_3$

$=\sum(0,1,4,5,7)$

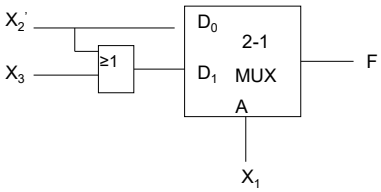


How to derive it only from equation?

Implementing Boolean functions with Multiplexers

$D_0 = X_2'$

$D1 = X_2' + X_3$



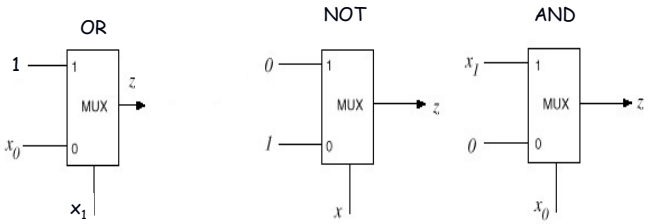
Implementing Boolean functions with Multiplexers

Exe. F(A,B,C,D)=∑m(1,2,4,9,10,11,12,14,15)

Derive a circuit for using 4-to-1 multiplexers.

MUX as a Universal Gate

- **We can construct OR, AND, and NOT gates using 2-to-1 MUXs. Thus, 2-to-1 MUX is a universal gate.**



$z = x_1 + x_1'x_0$
$= x_1x_0' + x_1x_0 + x_1'x_0 = x_1 + x_0$

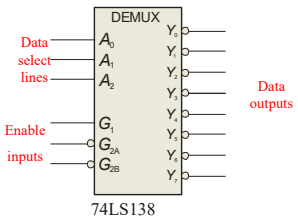
$z = 0x + 1x' = x'$

$z = x_1x_0 + 0x_0' = x_1x_0$

Demultiplexers

A demultiplexer (DEMUX) performs the opposite function from a MUX. It switches data from one input line to two or more data lines depending on the select inputs.

The 74LS138 was introduced previously as a decoder but can also serve as a DEMUX. When connected as a DEMUX, data is applied to one of the enable inputs, and routed to the selected output line depending on the select variables. Note that the outputs are active-LOW as illustrated in the following example…

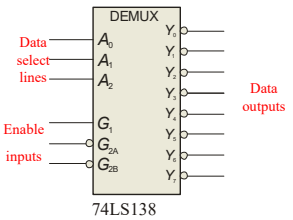


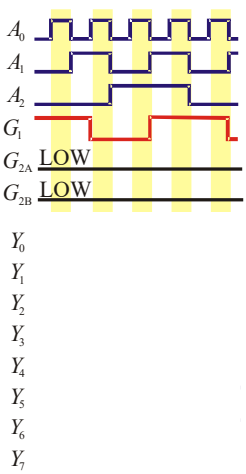
Demultiplexers

Example Determine the outputs, given the inputs shown.

Solution

The output logic is opposite to the input because of the active-LOW convention. (Red shows the selected line).





Parity Generators/Checkers

Parity is an error detection method that uses an extra bit appended to a group of bits to force them to be either odd or even. In even parity, the total number of ones is even; in odd parity the total number of ones is odd.

Example The ASCII letter S is 1010011. Show the parity bit for the letter S with odd and even parity.

Solution

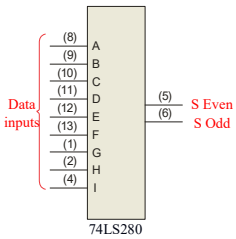
S with odd parity = 11010011
S with even parity = 01010011

Parity Generators/Checkers

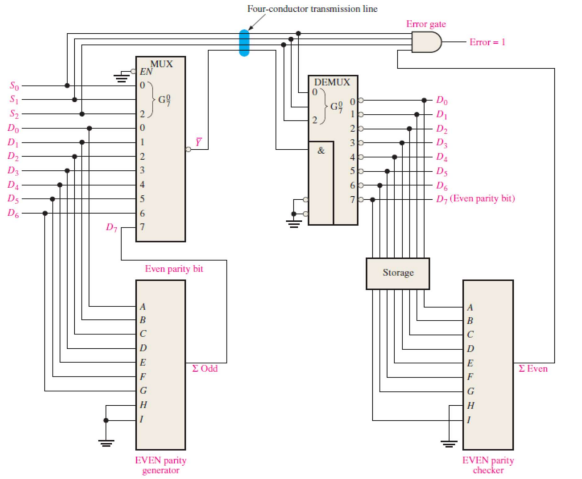
The 74LS280 can be used to generate a parity bit or to check an incoming data stream for even or odd parity.

Checker: The 74LS280 can test codes with up to 9 bits. The even output will normally be HIGH if the data lines have even parity; otherwise it will be LOW. Likewise, the odd output will normally be HIGH if the data lines have odd parity; otherwise it will be LOW.

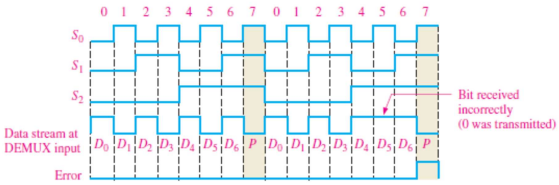
Generator: To generate even parity, the parity bit is taken from the odd parity output. To generate odd parity, the output is taken from the even parity output.



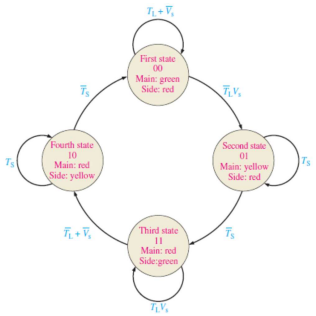
A Data Transmission System with Error Detection



A Data Transmission System with Error Detection

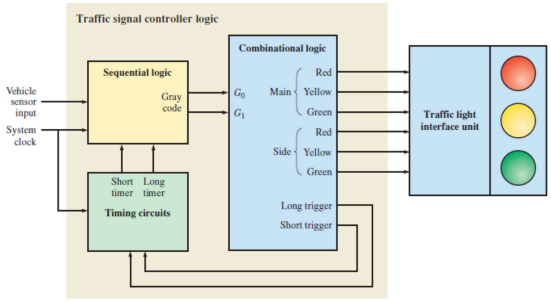


Applied Logic
Traffic Signal Controller: Part 1



State diagram for the traffic signal control

Applied Logic
Traffic Signal Controller: Part 1



Block diagram of the traffic signal controller

Homework

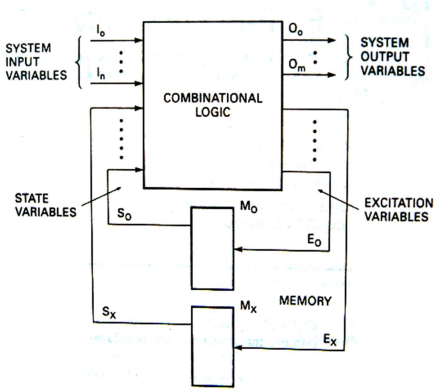
Using 4-1 Multiplexer to implement following expressions

(a) Z = f(A,B,C) = A'B+BC'+BC+AB'C'

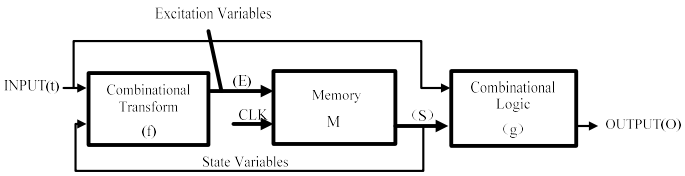
(b) Z = f(A,B,C) =A'•B' •C'+A'•B+A•B•C'+A•C

CHAPTER 7:
Sequential Circuits: Latches & Flip-Flops

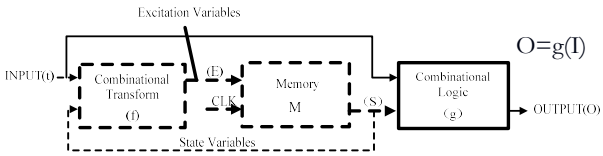
Sequential Circuit Models



Sequential Circuit Models

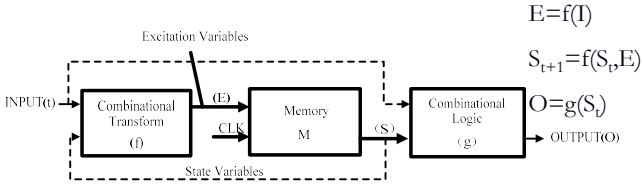


Several Subsets of the general sequential circuit model



O=g(I)

Combinational logic model



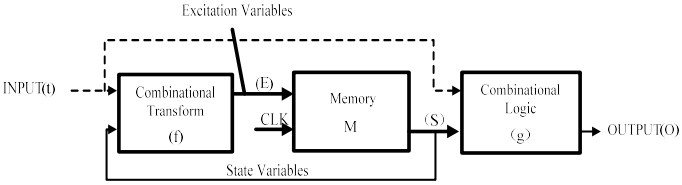
E=f(I)

$S_{t+1}=f(S_t,E)$

$O=g(S_t)$

sequential delay model

Several Subsets of the general sequential circuit model



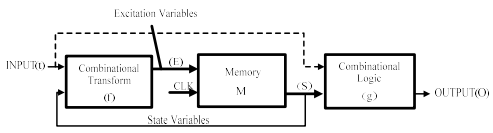
Simple sequential counter model

$E=f(S_t)$

$S_{t+1}=f(S_t,E)$

$O=g(S_t)$

Several Subsets of the general sequential circuit model

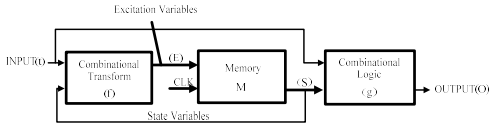


Moore sequential circuit model

$E=f(I,S_t)$

$S_{t+1}=f(S_t,E)$

$O=g(S_t)$



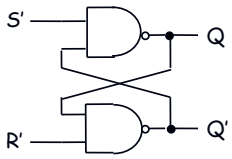
Mealy sequential circuit model

$E=f(I,S_t)$

$S_{t+1}=f(S_t,E)$

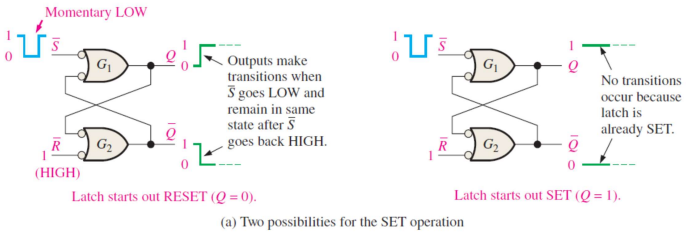
$O=g(I,S_t)$

S'R' Latch (NAND version)



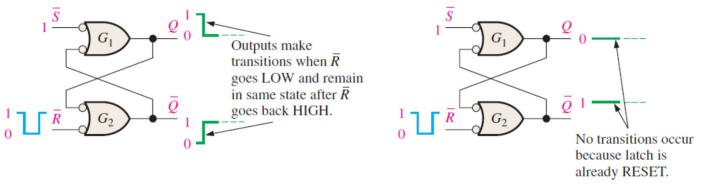
S'	R'	Q_{n+1}	Q_{n+1}'	
0	0	1	1	Disallowed
0	1	1	0	Set
1	0	0	1	Reset
1	1	Q_n	Q_n'	Hold

S'R' Latch (NAND version)



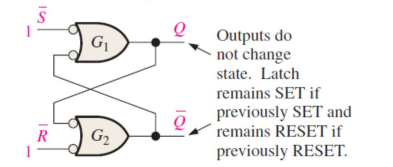
(a) Two possibilities for the SET operation

S'R' Latch (NAND version)

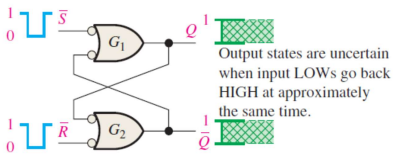


(b) Two possibilities for the RESET operation

S'R' Latch (NAND version)



(c) No-change condition



(d) Invalid condition

S'R' Latch (NAND version)

Truth table

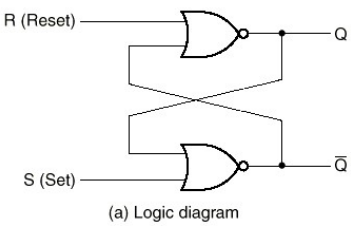
R'	S'	Q^n	Q^{n+1}	Function
0	0	0	1	Disallowed
0	0	1	1	Disallowed
0	1	0	0	Reset
0	1	1	0	Reset
1	0	0	1	Set
1	0	1	1	Set
1	1	0	0	Hold
1	1	1	1	Hold

Q^n \ R' S'	00	01	11	10
0	×	0	0	1
1	×	0	1	1

Characteristic Equation

$$\begin{cases} Q^{n+1}=S+R'Q^n \\ S'+R'=1 \end{cases}$$

SR latch (NOR version)



(a) Logic diagram

S	R	Q	$\bar{Q}$	
1	0	1	0	Set state
0	0	1	0	
0	1	0	1	Reset state
0	0	0	1	
1	1	0	0	Undefined

(b) Function table

-- SR: "set-reset", bi-stable element with two extra inputs; note the "undefined" output for S=R=1.

SR latch (NOR version)

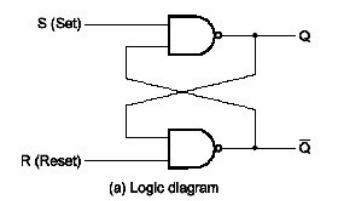
R	S	Q^n	Q^{n+1}	Function
0	0	0	0	Hold
0	0	1	1	Hold
0	1	0	1	Set
0	1	1	1	Set
1	0	0	0	Reset
1	0	1	0	Reset
1	1	0	0	Disallowed
1	1	1	0	Disallowed

Q^n \ R S	00	01	11	10
0	0	1	×	0
1	1	1	×	0

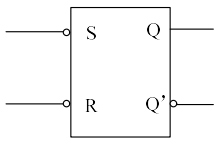
Characteristic Equation

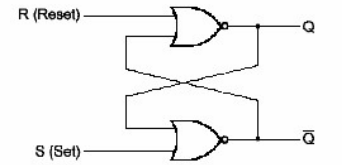
$$\begin{cases} Q^{n+1}=S+R'Q^n \\ SR=0 \end{cases}$$

SR Latches

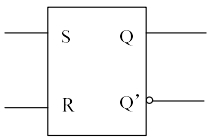


(a) Logic diagram

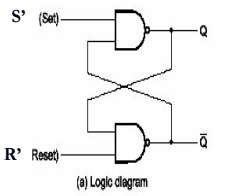




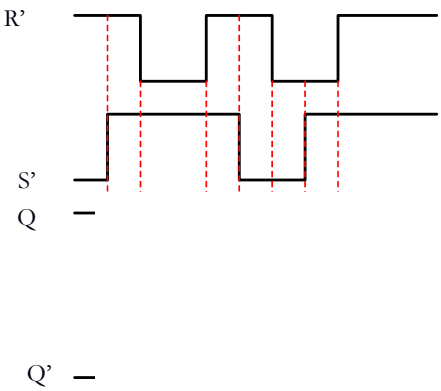
(a) Logic diagram



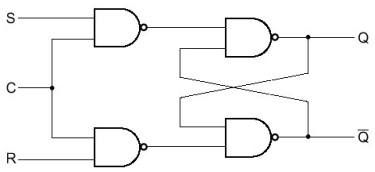
SR Latch Simulation



(a) Logic diagram



SR Latch with Clock signal



(a) Logic diagram

C	S	R	Next state of Q
0	X	X	No change
1	0	0	No change
1	0	1	Q = 0; Reset state
1	1	0	Q = 1; Set state
1	1	1	Undefined

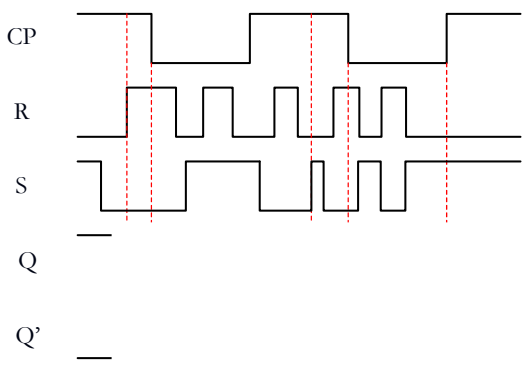
(b) Function table

Latch is sensitive to input changes ONLY when C=1

Characteristic Equation

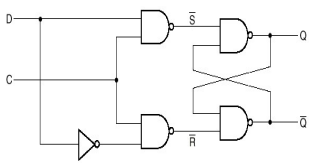
$$\begin{cases} Q^{n+1}=S+R'Q^n \\ SR=0 \end{cases}$$

SR Latch with Clock signal



D Latch

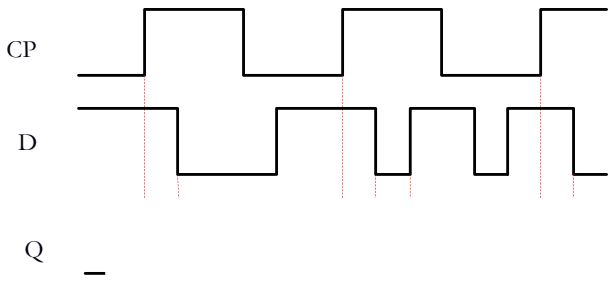
- One way to eliminate the undesirable indeterminate state in the RS flip flop is to ensure that inputs S and R are never 1 simultaneously. This is done in the *D latch:*



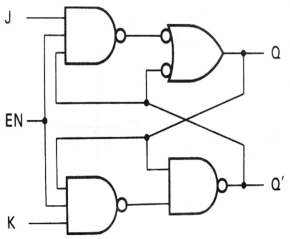
C	D	Next state of Q
0	X	No change
1	0	Q = 0; Reset state
1	1	Q = 1; Set state

Characteristic Equation $Q^{n+1}=S+R'Q^n=D+D"Q^n=D$

D Latch



JK Latch



When EN=1

$R'=(KQ^{n})'$ $S'=(JQ^{n})'$

So :

$Q^{n+1}=S+R'Q^{n}=JQ^{n}+(KQ^{n})'Q^{n}$

$= JQ^{n}+K'Q^{n}$

$Q^{n+1}=JQ^{n}+K'Q^{n}$

$S'+R'=(JQ^{n})'+(KQ^{n})'$

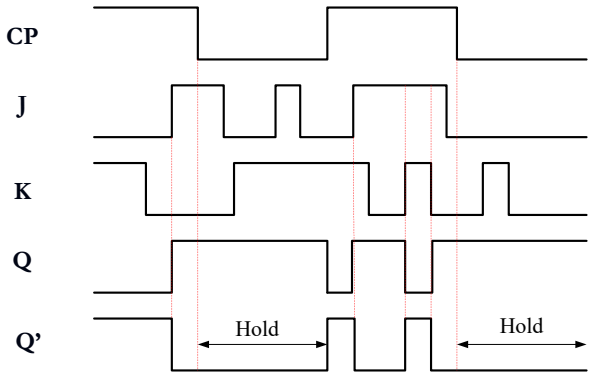
$=J'+Q^{n}+K'+Q^{n}$

$=1$

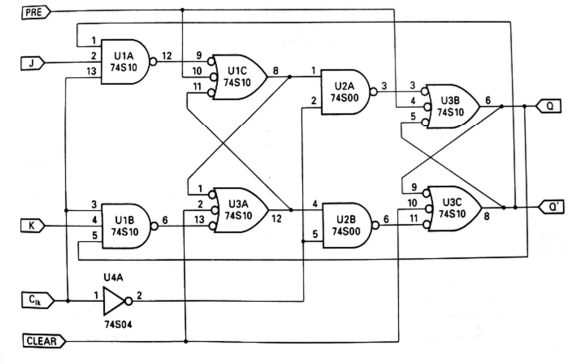
JK Latch

J	K	Q^{n}	Q^{n+1}	Function
0	0	0	0	Hold
0	0	1	1	Hold
0	1	0	0	Reset
0	1	1	0	Reset
1	0	0	1	Set
1	0	1	1	Set
1	1	0	1	Toggle
1	1	1	0	Toggle

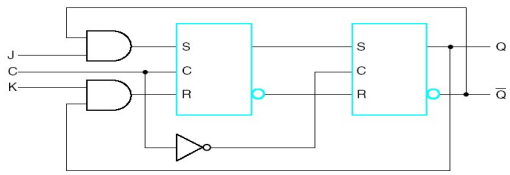
JK Latch



JK master-slave Latch

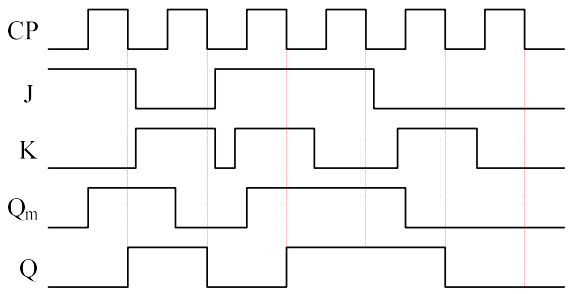


JK master-slave Latch

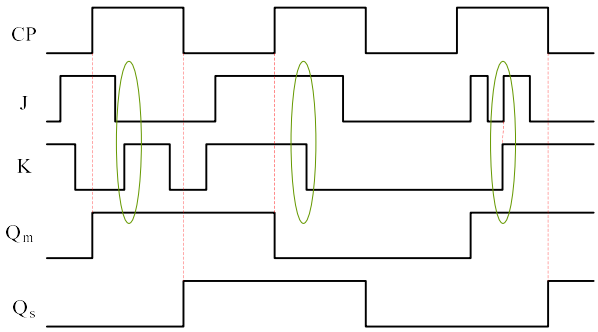


J	K	Next State of Q
0	0	Q
0	1	0
1	0	1
1	1	$\overline{Q}$

JK master-slave Latch



JK master-slave Latch



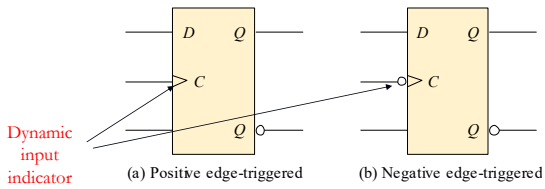
Edge Triggered Flip-flops

- Flip-flops are synchronous bi-stable devices
- The term *synchronous* means that the output changes state only at a specified point(leading or trailing edge) on the triggering input called the **clock** (CLK), which is designated as a control input, *C*
- Flip-flops are edge-triggered or edge-sensitive whereas gated latches are level-sensitive

Edge Triggered Flip-flops

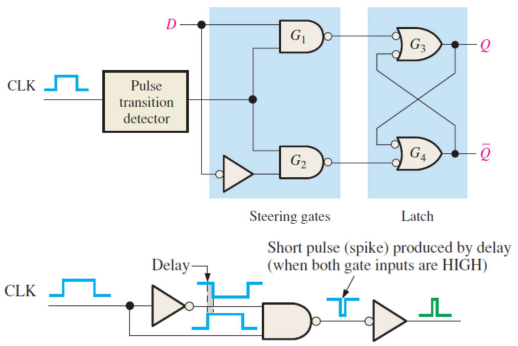
A flip-flop differs from a latch in the manner it changes states. A flip-flop is a clocked device, in which only the clock edge determines when a new bit is entered.

The active edge can be positive or negative.



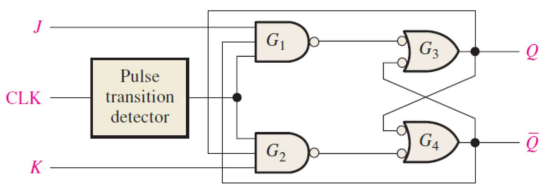
Edge Triggered Flip-flops

The basic D flip-flop differs from the gated D latch only in that it has a pulse transition detector



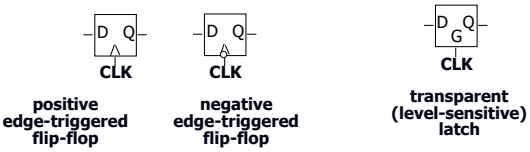
Edge Triggered Flip-flops

The basic JK flip-flop differs from the gated JK latch only in that it has a pulse transition detector



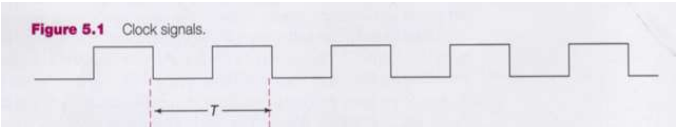
Alternatives in FF choice

- Type of triggering
  - Untriggered (asynchronous)
  - Level-triggered (C=1)
  - Edge-triggered (rising or falling edge of C)



Clock Signal

Clock generator: Periodic train of clock pulses



Edge Triggered Flip-flops

The truth table for a positive-edge triggered D flip-flop shows an up arrow to remind you that it is sensitive to its *D* input only on the rising edge of the clock; otherwise it is latched. The truth table for a negative-edge triggered D flip-flop is identical except for the direction of the arrow.

Inputs		Outputs		
D	CLK	*Q*	$\overline{Q}$	Comments
1	↑	1	0	SET
0	↑	0	1	RESET

(a) Positive-edge triggered

Inputs		Outputs		
D	CLK	*Q*	$\overline{Q}$	Comments
1	↓	1	0	SET
0	↓	0	1	RESET

(b) Negative-edge triggered

Edge Triggered Flip-flops

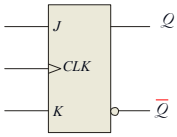
The J-K flip-flop is more versatile than the D flip flop. In addition to the clock input, it has two inputs, labeled *J* and *K*. When both *J* and *K* = 1, the output changes states (toggles) on the active clock edge (in this case, the rising edge).

Inputs			Outputs		
J	*K*	CLK	*Q*	$\overline{Q}$	Comments
0	0	↑	Q_0	$\overline{Q}_0$	No change
0	1	↑	0	1	RESET
1	0	↑	1	0	SET
1	1	↑	$\overline{Q}_0$	Q_0	Toggle

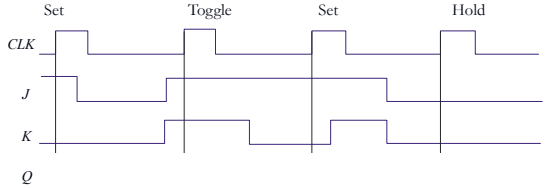
Edge Triggered Flip-flops

Example

Determine the *Q* output for the *J-K* flip-flop, given the inputs shown.



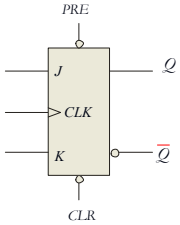
Notice that the outputs change on the leading edge of the clock.



Edge Triggered Flip-flops

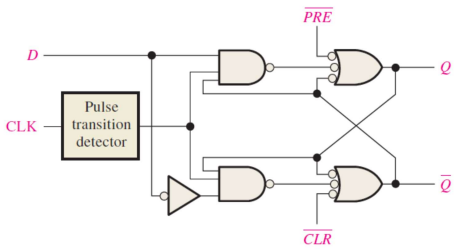
Synchronous inputs are transferred in the triggering edge of the clock (for example the *D* or *J-K* inputs). Most flip-flops have other inputs that are *asynchronous*, meaning they affect the output independent of the clock.

Two such inputs are normally labeled preset (*PRE*) and clear (*CLR*). These inputs are usually active LOW. A J-K flip flop with active LOW preset and CLR is shown.



Asynchronous Preset and Clear Inputs

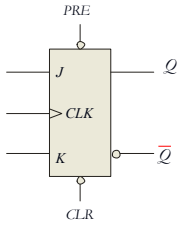
Two such inputs are normally labeled preset (*PRE*) and clear (*CLR*). These inputs are usually active LOW. A J-K flip flop with active LOW preset and CLR is shown.

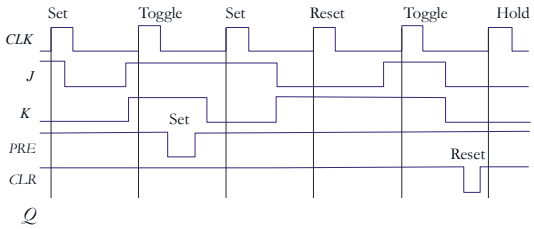


Asynchronous Preset and Clear Inputs

Example

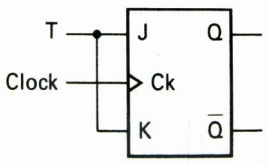
Determine the *Q* output for the *J-K* flip-flop, given the inputs shown.





T Flip-Flop

Toggle flip-flop logic diagram



Truth table for a *T* flip-flop

Input		Output
T	Q_t	Q_{t+1}
0	0	0
0	1	1
1	0	1
1	1	0

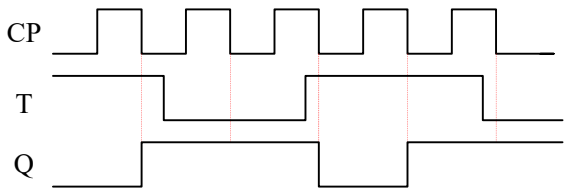
Characteristic equation for the *T* flip-flop:

$$Q_{t+1} = T'Q_t + TQ_t'$$

$Q^{n+1}=JQ^{n'}+K'Q^n$

$=TQ^{n'}+T'Q^n$

T Flip-Flop



Characteristic equations compare

Flip-flop type	Characteristic equations
RS	$Q_{t+1} = S + R'Q_t$
JK	$Q_{t+1} = JQ'_t + K'Q_t$
D	$Q_{t+1} = D$
T	$Q_{t+1} = Q_tT' + Q_t'T$

Simple counters

- Devide by 2,4,8 Counters

Figure 5.58
Divide-by-2 counter

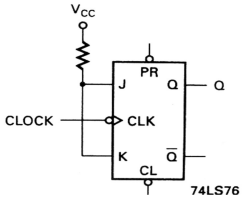
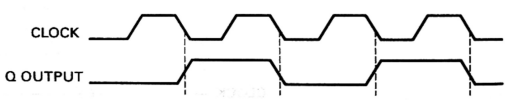
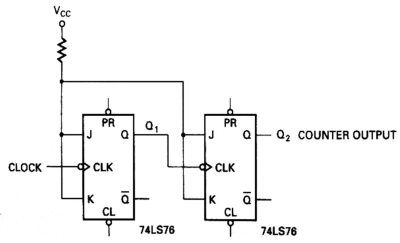


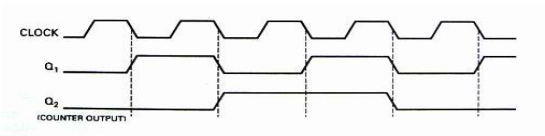
Figure 5.59
Divide-by-2 timing waveforms



Simple counters

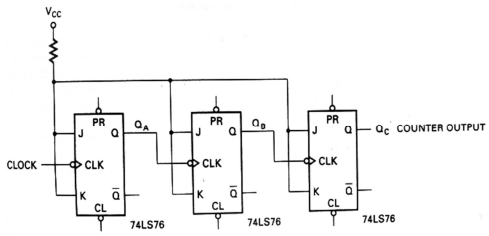
- Devide-by-4 Counter

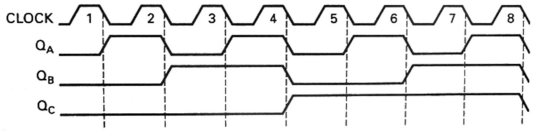




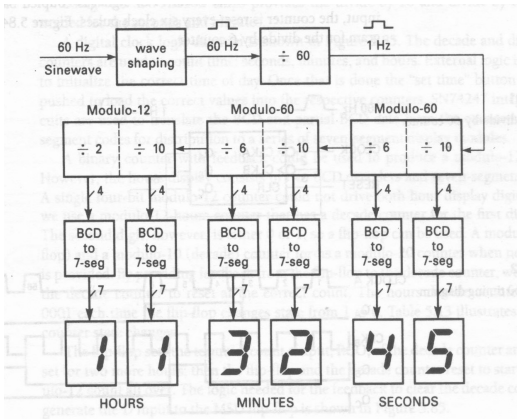
Simple counters

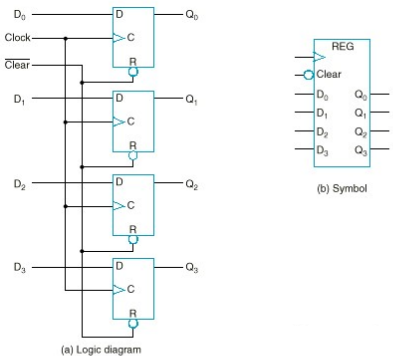
- Devide-by-8 Counter





A Counter Application: Digital Clock





Homework

- 2, 4, 6, 8,14

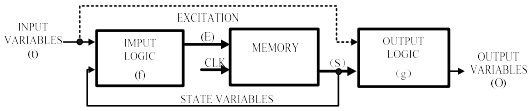
CHAPTER 8
Sequential Circuits

Mealy Vs Moore machines

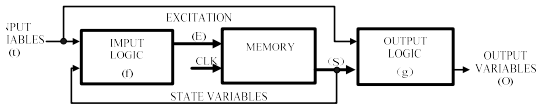
- Mealy model:
  - Both outputs and next state depend both on primary inputs AND present state.
- Moore model:
  - Only next state depends directly on primary inputs AND present state. Outputs depend only on present state.

Mealy Vs Moore machines

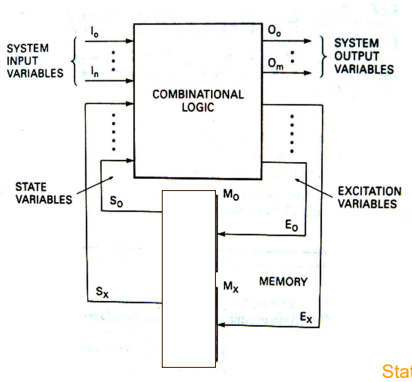
Moore sequential circuit model



Mealy sequential circuit model



State Machine Notation



- ✓Input variable
- ✓Output variable
- ✓State variable
- ✓Excitation variable
- ✓State

State variables and states are related by the expression:

$2^x = y$

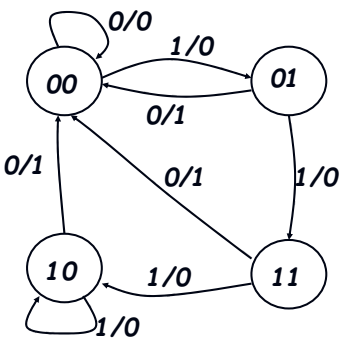
State variables — states

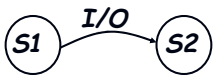
State Diagram

- Graphical representation of a state table.
- Graph ***node*** with label ***s*** denotes state ***s***

- Graph ***edge*** with label ***X*** denotes transition between two states when input ***X*** is applied

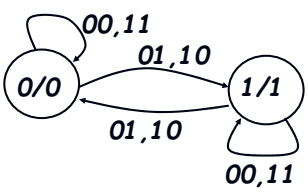
State Diagram of Mealy model

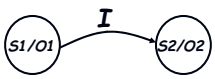




Reads as:
When at state ***s1*** and apply input ***I***, we get output ***O*** and proceed to state ***s2***.

State Diagram of Moore model



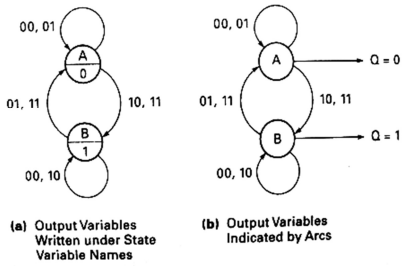


Reads as:
When at state ***s1*** with output ***O1*** and apply input ***I***, we proceed to state ***s2*** with Output ***O2***.

State Diagram

J	K	Next State of Q
0	0	Q
0	1	0
1	0	1
1	1	$\overline{Q}$

Figure 6.5
Moore circuit notation of a J–K flip-flop

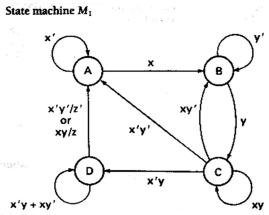


(a) Output Variables Written under State Variable Names

(b) Output Variables Indicated by Arcs

State Table

- Enumerates the relationship between inputs, outputs, and states of the sequential circuit.



State Table

Present state	Next state xy/z 00 z	01 z	11 z	10 z
A				
B				
C				
D				

State Table

State Table

Present state	00 z	01 z	11 z	10 z
A	A 0	A 0	B 0	B 0
B	B 0	C 0	C 0	B 0
C	A 0	D 0	C 0	B 0
D	A 0	D 0	A 1	D 0

State Table

Present State	Input X	Input Y	Next State	Output Z
A	0	0	A	0
A	0	1	A	0
A	1	0	B	0
A	1	1	B	0
B	0	0	B	0
B	0	1	D	0
B	1	0	B	0
B	1	1	D	0
C	0	0	A	0
C	0	1	C	0
C	1	0	C	0
C	1	1	A	1
D	0	0	A	0
D	0	1	C	0
D	1	0	B	0
D	1	1	D	0

Transition Table

Each state is assigned a unique code.

FA	FB	State
0	0	A
0	1	B
1	0	C
1	1	D

Transition Table for state machine M_1

Present state F_A F_B	Next state xy/z 00 z ($F_A F_B$)	01 z ($F_A F_B$)	11 z ($F_A F_B$)	10 z ($F_A F_B$)
0 0	0 0 0	0 0 0	0 1 0	0 1 0
0 1	0 1 0	1 1 0	1 1 0	0 1 0
1 1	0 0 0	1 0 0	1 1 0	0 1 0
1 0	0 0 0	1 0 0	0 0 1	1 0 0

State Table

Transition Table for state machine M_1

Present state F_A F_B	Next state xy/z 00 z ($F_A F_B$)	01 z ($F_A F_B$)	11 z ($F_A F_B$)	10 z ($F_A F_B$)
0 0	0 0 0	0 0 0	0 1 0	0 1 0
0 1	0 1 0	1 1 0	1 1 0	0 1 0
1 1	0 0 0	1 0 0	1 1 0	0 1 0
1 0	0 0 0	1 0 0	0 0 1	1 0 0

Transition Table for state machine M_1

Present State F_A	Present State F_B	Input X	Input Y	Next State F_A	Next State F_B	Output Z
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	0	1	0	0	1	0
0	0	1	1	0	1	0
0	1	0	0	0	1	0
0	1	0	1	1	1	0
0	1	1	0	0	1	0
0	1	1	1	1	1	0
1	0	0	0	0	0	0
1	0	0	1	1	0	0
1	0	1	0	1	0	0
1	0	1	1	0	0	1
1	1	0	0	0	0	0
1	1	0	1	1	0	0
1	1	1	0	0	1	0
1	1	1	1	1	1	0

Excitation Table and Equations

- D flip-flop used to realize circuit

D flip-flop characteristic table and equation

D	Present, Q_t	Next, Q_{t+1}
0	0	0
1	0	1
0	1	0
1	1	1

$Q_{t+1} = D$

D flip-flop excitation table

Q_t	Q_{t+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

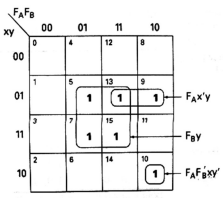
Transition Table for state machine M_1

Present state F_A F_B	Next state xy/z 00 z ($F_A F_B$)	01 z ($F_A F_B$)	11 z ($F_A F_B$)	10 z ($F_A F_B$)
0 0	0 0 0	0 0 0	0 1 0	0 1 0
0 1	0 1 0	1 1 0	1 1 0	0 1 0
1 1	0 0 0	1 0 0	1 1 0	0 1 0
1 0	0 0 0	1 0 0	0 0 1	1 0 0

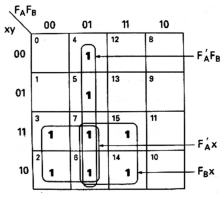
state machine M_1 excitation table using D flip-flops

Present state F_A F_B	Next state xy/z 00 z ($D_A D_B$)	01 z ($D_A D_B$)	11 z ($D_A D_B$)	10 z ($D_A D_B$)
0 0	0 0 0	0 0 0	0 1 0	0 1 0
0 1	0 1 0	1 1 0	1 1 0	0 1 0
1 1	0 0 0	1 0 0	1 1 0	0 1 0
1 0	0 0 0	1 0 0	0 0 1	1 0 0

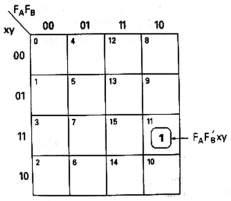
Excitation Table and Equations



D_A K-Map



D_B K-Map



Z K-Map

$D_A=f(F_A,F_B,x,y)=\Sigma(5,7,10,13,15)=F_AF_B'xy'+F_Ax'y+F_By$

$D_B=f(F_A,F_B,x,y)=\Sigma(2,3,4,5,6,7,14,15)=F_A'F_B+F_A'x+F_Bx$

$Z= f(F_A,F_B,x,y)=\Sigma(11)=F_AF_B'xy$

Excitation Table and Equations

- T flip-flop used to realize circuit

T flip-flop characteristic table and equation

Present		Next
T	Q_t	Q_{t+1}
0	0	0
1	0	1
1	1	0
0	1	1

$Q_{t+1} = T \oplus Q_t$

Transition Table for state machine M_1

Present state F_A F_B	Next state xy/z: 00 z	01 z	11 z	10 z
	F_AF_B	F_AF_B	F_AF_B	F_AF_B
0 0	0 0 0	0 0 0	0 1 0	0 1 0
0 1	0 1 0	1 1 0	1 1 0	0 1 0
1 1	0 0 0	1 0 0	1 1 0	0 1 0
1 0	0 0 0	1 0 0	0 0 1	1 0 0

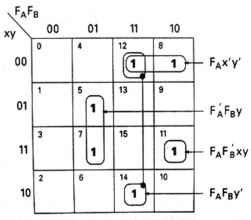
T flip-flop excitation table

Q_t	Q_{t+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

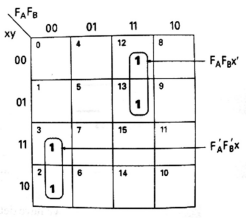
T flip-flop excitation table for state machine M_1

Present state F_A F_B	Next state xy/z: 00 z	01 z	11 z	10 z
	T_AT_B	T_AT_B	T_AT_B	T_AT_B
0 0				
0 1				
1 1				
1 0				

Excitation Table and Equations



T_A K-Map



T_B K-Map

$T_A=f(F_A,F_B,x,y)=\Sigma(5,7,8,11,12,14)=F_AF_B'xy+F_Ax'y'+F_AF_By'+F_A'F_By$

$T_B=f(F_A,F_B,x,y)=\Sigma(2,3,12,13)=F_A'F_B'x+F_AF_Bx'$

Excitation Table and Equations

- S-R flip-flop used to realize circuit

R–S flip-flop characteristic table and equation

Present			Next
S	R	Q_t	Q_{t+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	Undefined
1	1	1	Undefined

$Q_{t+1} = S + R'Q_t$

S-R flip-flop excitation table

Q_t	Q_{t+1}	S	R
0	0	0	d
0	1	1	0
1	0	0	1
1	1	d	0

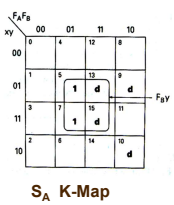
Transition Table for state machine M_1

Present state F_A F_B	Next state xy/z: 00 z	01 z	11 z	10 z
	F_AF_B	F_AF_B	F_AF_B	F_AF_B
0 0	0 0 0	0 0 0	0 1 0	0 1 0
0 1	0 1 0	1 1 0	1 1 0	0 1 0
1 1	0 0 0	1 0 0	1 1 0	0 1 0
1 0	0 0 0	1 0 0	0 0 1	1 0 0

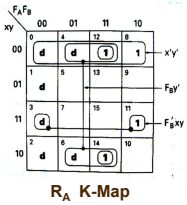
state machine M_1 excitation table using S-R flip-flops

Present state F_A F_B	Next state xy/z: 00 z	01 z	11 z	10 z
	S_AR_A S_BR_B	S_AR_A S_BR_B	S_AR_A S_BR_B	S_AR_A S_BR_B
0 0				
0 1				
1 1				
1 0				

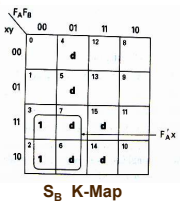
Excitation Table and Equations



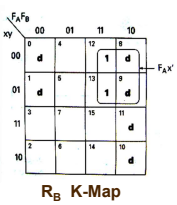
S_A K-Map



R_A K-Map



S_B K-Map



R_B K-Map

$S_A=f(F_A,F_B,x,y)=\Sigma(5,7) +\Sigma d(9,10,13,15) =F_By$

$R_A=f(F_A,F_B,x,y)=\Sigma(8,11,12,14) +\Sigma d(0,1,2,3,4,6) =F_B'xy+x'y'+F_By'$

$S_B=f(F_A,F_B,x,y)=\Sigma(2,3) +\Sigma d(4,5,6,7,14,15) =F_A'x$

$R_B=f(F_A,F_B,x,y)=\Sigma(12,13) +\Sigma d(0,1,8,9,10,11) =F_Ax'$

Excitation Table and Equations

- J-K flip-flop used to realize circuit

J–K flip-flop characteristic table and equation

Present			Next
J	K	Q_t	Q_{t+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

$Q_{t+1} = JQ_t' + K'Q_t$

J-K flip-flop excitation table

Q_t	Q_{t+1}	J	K
0	0	0	d
0	1	1	d
1	0	d	1
1	1	d	0

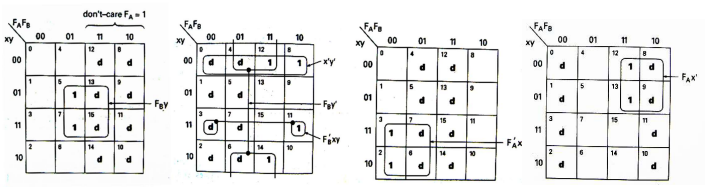
Transition Table for state machine M_1

Present state F_A F_B	Next state xy/z: 00 z	01 z	11 z	10 z
	F_AF_B	F_AF_B	F_AF_B	F_AF_B
0 0	0 0 0	0 0 0	0 1 0	0 1 0
0 1	0 1 0	1 1 0	1 1 0	0 1 0
1 1	0 0 0	1 0 0	1 1 0	0 1 0
1 0	0 0 0	1 0 0	0 0 1	1 0 0

state machine M_1 excitation table using J-K flip-flops

Present state F_A F_B	Next state xy/z: 00 z	01 z	11 z	10 z
	J_AK_A J_BK_B	J_AK_A J_BK_B	J_AK_A J_BK_B	J_AK_A J_BK_B
0 0				
0 1				
1 1				
1 0				

Excitation Table and Equations



J_A K-Map K_A K-Map J_B K-Map K_B K-Map

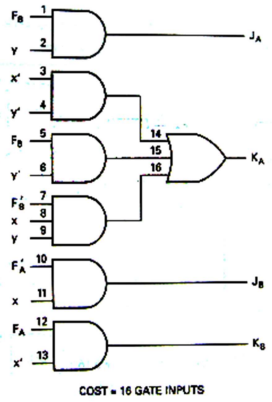
$J_A=f(F_A,F_B,x,y)=\Sigma(5,7)+\Sigma d(8,9,10,11,12,13,14,15)=F_By$

$K_A=f(F_A,F_B,x,y)=\Sigma(8,11,12,14)+\Sigma d(0,1,2,3,4,6,7)=F_B'xy+x'y'+F_By'$

$J_B=f(F_A,F_B,x,y)=\Sigma(2,3)+\Sigma d(4,5,6,7,12,13,14,15)=F_A'x$

$K_B=f(F_A,F_B,x,y)=\Sigma(12,13)+\Sigma d(0,1,2,3,8,9,10,11)=F_Ax'$

Excitation Realization Cost



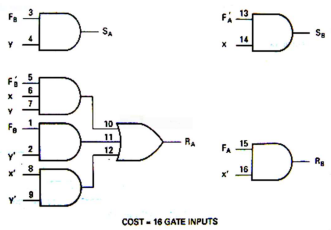
$J_A=F_By$

$K_A=F_B'xy+x'y'+F_By'$

$J_B=F_A'x$

$K_B=F_Ax'$

Excitation Realization Cost



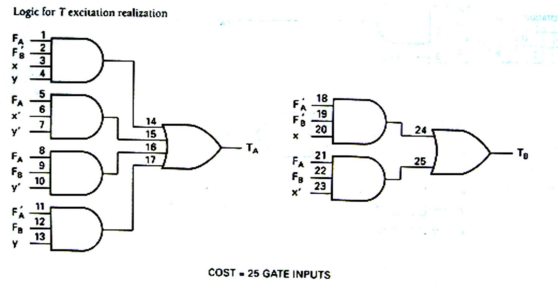
$S_A=F_By$

$R_A=F_B'xy+x'y'+F_By'$

$S_B=F_A'x$

$R_B=F_Ax'$

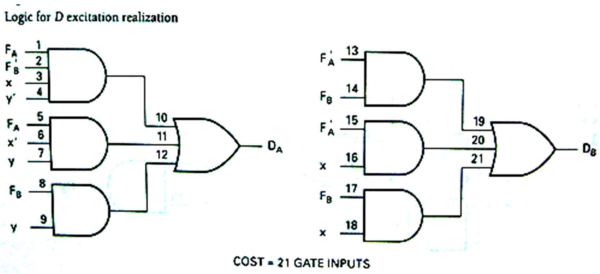
Excitation Realization Cost



$T_A=F_AF_B'xy+F_Ax'y'+F_AF_By'+F_A'F_By$

$T_B=F_A'F_B'x+F_AF_Bx'$

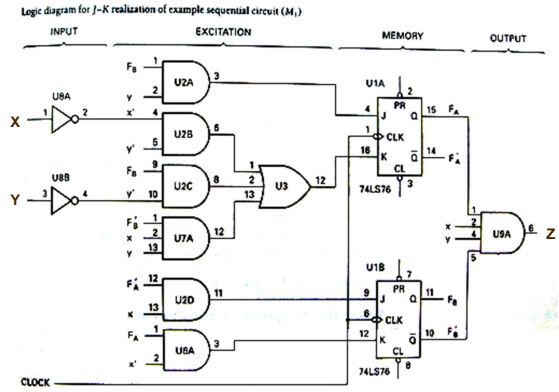
Excitation Realization Cost



$D_A=F_AF_B'xy'+F_Ax'y+F_By$

$D_B=F_A'F_B+F_A'x+F_Bx$

Excitation Realization Cost



$Z=F_AF_B'xy$

TO BE CONTINUED

CHAPTER 8
Counters

Overview

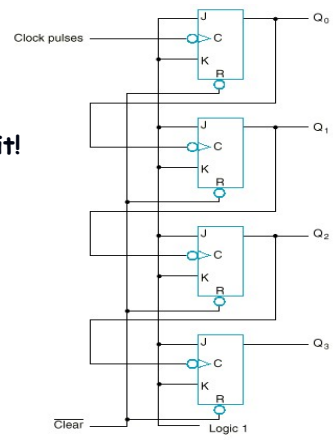
- Ripple Counter
- Synchronous Binary Counters
  - Design with D Flip-Flops
  - Design with J-K Flip-Flops
- Serial Vs. Parallel Counters
- Up-down Binary Counter
- Binary Counter with Parallel Load
- BCD Counter, Arbitrary sequence Counters

Counters

- A *counter* is a register that goes through a predetermined sequence of states upon the application of clock pulses.
- Counters are categorized as:
  - Ripple Counters:
    The FF output transition serves as a source for triggering other FFs. No common clock.
  - Synchronous Counter:
    All FFs receive the common clock pulse, and the change of state is determined from the present state.

Example: A 4-bit Upward Counting Ripple Counter

Less Significant Bit output is Clock for Next Significant Bit! (Clock is active low)



(a) *JK* Flip-Flop			
J	K	$Q(t+1)$	Operation
0	0	$Q(t)$	No change
0	1	0	Reset
1	0	1	Set
1	1	$\overline{Q}(t)$	Complement

Example (cont.)

- The output of each FF is connected to the C input of the next FF in sequence.
- The FF holding the least significant bit receives the incoming clock pulses.
- The J and K inputs of all FFs are connected to a permanent logic 1.
- The bubble next to the C label indicates that the FFs respond to the negative-going transition of the input.

Example (cont.)

Operation:

- The least significant bit (Q_0) is complemented with each negative-edge clock pulse input.
- Every time that Q_0 goes from 1 to 0, Q_1 is complemented.
- Every time that Q_1 goes from 1 to 0, Q_2 is complemented.
- Every time that Q_2 goes from 1 to 0, Q_3 is complemented, and so on.

Upward Counting Sequence

Q_3	Q_2	Q_1	Q_0
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	0	1
1	1	1	0
1	1	1	1

Synchronous Binary Counters

- The design procedure for a binary counter is the same as any other synchronous sequential circuit.
- The primary inputs of the circuit are the CLK and any control signals (EN, Load, etc).
- The primary outputs are the FF outputs (present state).
- Most efficient implementations usually use T-FFs or JK-FFs. We will examine JK and D flip-flop designs.

Synchronous Binary Counters:

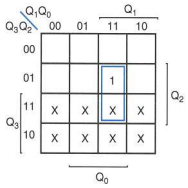
J-K Flip Flop Design of a 4-bit Binary Up Counter

Present state				Next state			
Q_3	Q_2	Q_1	Q_0	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	0
0	0	1	0	0	0	1	1
0	0	1	1	0	1	0	0
0	1	0	0	0	1	0	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	1	1
0	1	1	1	1	0	0	0
1	0	0	0	1	0	0	1
1	0	0	1	1	0	1	0
1	0	1	0	1	0	1	1
1	0	1	1	1	1	0	0
1	1	0	0	1	1	0	1
1	1	0	1	1	1	1	0
1	1	1	0	1	1	1	1
1	1	1	1	0	0	0	0

Synchronous Binary Counters:

J-K Flip Flop Design of a Binary Up Counter (cont.)

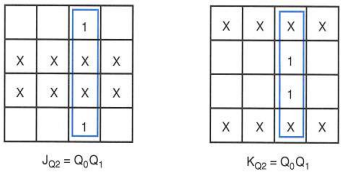
Present state				Next state					
Q_3	Q_2	Q_1	Q_0	Q_3	Q_2	Q_1	Q_0	J_{Q3}	K_{Q3}
0	0	0	0	0	0	0	1	0	X
0	0	0	1	0	0	1	0	0	X
0	0	1	0	0	0	1	1	0	X
0	0	1	1	0	1	0	0	0	X
0	1	0	0	0	1	0	1	0	X
0	1	0	1	0	1	1	0	0	X
0	1	1	0	0	1	1	1	0	X
0	1	1	1	1	0	0	0	1	X
1	0	0	0	1	0	0	1	X	0
1	0	0	1	1	0	1	0	X	0
1	0	1	0	1	0	1	1	X	0
1	0	1	1	1	1	0	0	X	0
1	1	0	0	1	1	0	1	X	0
1	1	0	1	1	1	1	0	X	0
1	1	1	0	1	1	1	1	X	0
1	1	1	1	0	0	0	0	X	1



Synchronous Binary Counters:

J-K Flip Flop Design of a Binary Up Counter (cont.)

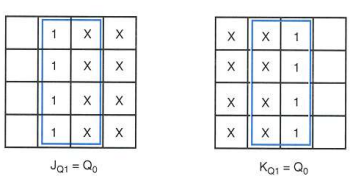
Present state				Next state				Flip-flo	
Q_3	Q_2	Q_1	Q_0	Q_3	Q_2	Q_1	Q_0	J_{Q2}	K_{Q2}
0	0	0	0	0	0	0	1	0	X
0	0	0	1	0	0	1	0	0	X
0	0	1	0	0	0	1	1	0	X
0	0	1	1	0	1	0	0	1	X
0	1	0	0	0	1	0	1	X	0
0	1	0	1	0	1	1	0	X	0
0	1	1	0	0	1	1	1	X	0
0	1	1	1	1	0	0	0	X	1
1	0	0	0	1	0	0	1	0	X
1	0	0	1	1	0	1	0	0	X
1	0	1	0	1	0	1	1	0	X
1	0	1	1	1	1	0	0	1	X
1	1	0	0	1	1	0	1	X	0
1	1	0	1	1	1	1	0	X	0
1	1	1	0	1	1	1	1	X	0
1	1	1	1	0	0	0	0	X	1



Synchronous Binary Counters:

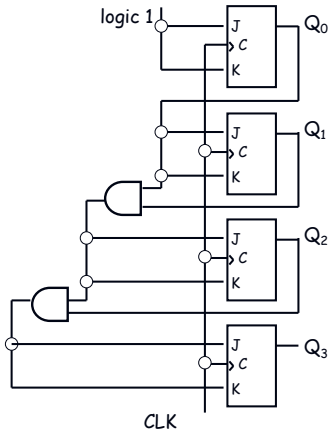
J-K Flip Flop Design of a Binary Up Counter (cont.)

Present state				Next state				p inputs	
Q_3	Q_2	Q_1	Q_0	Q_3	Q_2	Q_1	Q_0	J_{Q1}	K_{Q1}
0	0	0	0	0	0	0	1	0	X
0	0	0	1	0	0	1	0	1	X
0	0	1	0	0	0	1	1	X	0
0	0	1	1	0	1	0	0	X	1
0	1	0	0	0	1	0	1	0	X
0	1	0	1	0	1	1	0	1	X
0	1	1	0	0	1	1	1	X	0
0	1	1	1	1	0	0	0	X	1
1	0	0	0	1	0	0	1	0	X
1	0	0	1	1	0	1	0	1	X
1	0	1	0	1	0	1	1	X	0
1	0	1	1	1	1	0	0	X	1
1	1	0	0	1	1	0	1	0	X
1	1	0	1	1	1	1	0	1	X
1	1	1	0	1	1	1	1	X	0
1	1	1	1	0	0	0	0	X	1



Synchronous Binary Counters:

J-K Flip Flop Design of a Binary Up Counter (cont.)



$J_{Q0} = 1$
$K_{Q0} = 1$

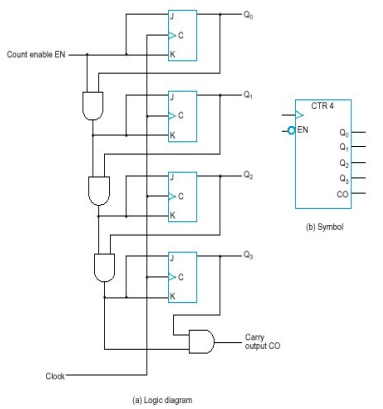
$J_{Q1} = Q_0$
$K_{Q1} = Q_0$

$J_{Q2} = Q_0 Q_1$
$K_{Q2} = Q_0 Q_1$

$J_{Q3} = Q_0 Q_1 Q_2$
$K_{Q3} = Q_0 Q_1 Q_2$

Synchronous Binary Counters:

J-K Flip Flop Design of a Binary Up Counter with EN and CO



EN = enable control signal, when 0 counter remains in the same state, when 1 it counts

CO = carry output signal, used to extend the counter to more stages

$J_{Q0} = 1 \cdot EN$
$K_{Q0} = 1 \cdot EN$
$J_{Q1} = Q_0 \cdot EN$
$K_{Q1} = Q_0 \cdot EN$
$J_{Q2} = Q_0 Q_1 \cdot EN$
$K_{Q2} = Q_0 Q_1 \cdot EN$
$J_{Q3} = Q_0 Q_1 Q_2 \cdot EN$
$K_{Q3} = Q_0 Q_1 Q_2 \cdot EN$
$CO = Q_0 Q_1 Q_2 Q_3 \cdot EN$

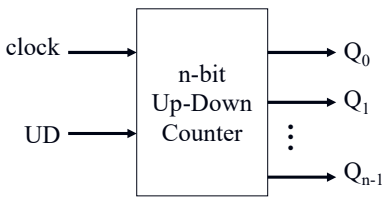
Synchronous binary counters using D flip-flops

- $D_{Q0} = Q_0 \otimes EN$
- $D_{Q1} = Q_1 \otimes (Q_0 \cdot EN)$
- $D_{Q2} = Q_2 \otimes (Q_0 Q_1 \cdot EN)$
- $D_{Q3} = Q_3 \otimes (Q_0 Q_1 Q_2 \cdot EN)$
- $CO = Q_0 Q_1 Q_2 Q_3 \cdot EN$

JK-FF equations

JK-based design calls for 4 AND gates
D-based design calls for 4 AND and 4 XOR gates

Up-Down Binary Counter



UD = 0: count up
UD = 1: count down

Up-Down Binary Counter (cont.)

UD	Q2	Q1	Q0	Q2.D	Q1.D	Q0.D	UD	Q2	Q1	Q0	Q2.D	Q1.D	Q0.D
0	0	0	0	0	0	1	1	0	0	0	1	1	1
0	0	0	1	0	1	0	1	0	0	1	0	0	0
0	0	1	0	0	1	1	1	0	1	0	0	0	1
0	0	1	1	1	0	0	1	0	1	1	0	1	0
0	1	0	0	1	0	1	1	1	0	0	0	1	1
0	1	0	1	1	1	0	1	1	0	1	1	0	0
0	1	1	0	1	1	1	1	1	1	0	1	0	1
0	1	1	1	0	0	0	1	1	1	1	1	1	0

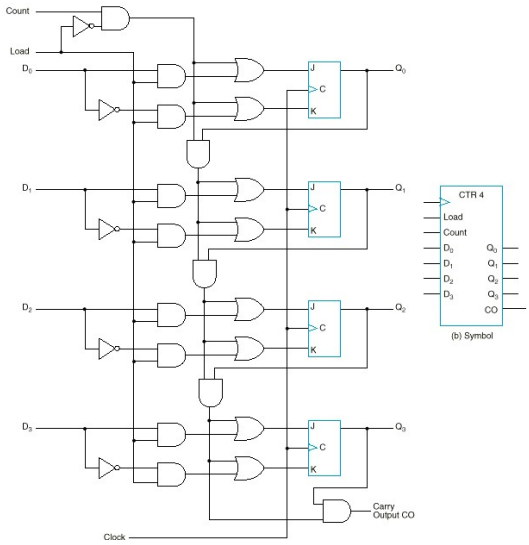
Up-Counter Down-Counter

Finish the design in class

Binary Counter with Parallel Load

- (Next slide) gives the logic diagram and symbol of a 4-bit synchronous binary counter with parallel load capability. The function table for this binary counter is

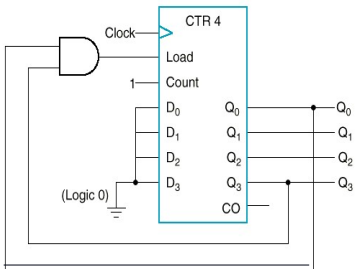
Load	Count	Operation
0	0	Nothing
0	1	Count
1	x	Load



BCD counter

- The binary counter with parallel load can be converted into a synchronous BCD counter by connecting an external AND gate to it.

Load	Count	Operation
0	0	Nothing
0	1	Count
1	x	Load



BCD counter (cont.)

- The counter starts with an all-zero output.
- As long as the output of the AND gate is 0, each positive clock pulse transition increments the counter by one.
- When the output reaches the count of 1001, both Q_0 and Q_3 become 1, making the output of the AND gate equal to 1. This condition makes Load active, so on the next clock transition, the counter does not count, but is loaded from its four inputs.
- The value loaded then is 0000.

Arbitrary Sequence Counter

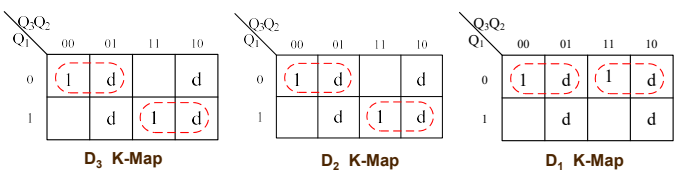
- Given an arbitrary sequence, design a counter that will generate this sequence.
- Procedure:
  - Derive state table/diagram based on give sequence
  - Simplify (using K-maps, etc)
  - Draw logic diagram
- Example: Use D-FFs to draw the logic diagram for sequence generator (counter) for: 0 → 7 → 6 → 1 → 0 (000 → 111 → 110 → 001 → 000)

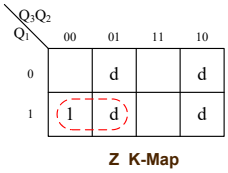
Example:

- Use D-FFs to draw the logic diagram for sequence generator (counter) for: 0 → 7 → 6 → 1 → 0 (000 → 111 → 110 → 001 → 000)

$Q_3Q_2Q_1$	$Q_3^+Q_2^+Q_1^+$	$D_3D_2D_1$	Z
000	111	111	0
001	000	000	1
010	ddd	ddd	d
011	ddd	ddd	d
100	ddd	ddd	d
101	ddd	ddd	d
110	001	001	0
111	110	110	0

Example:





$D_3=Q_3'Q_1'+Q_3Q_1$

$D_2=Q_3'Q_1'+Q_3Q_1$

$D_1=Q_1'$

$Z= Q_3'Q_1$

Example:

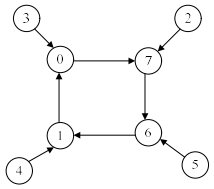
$D_3=Q_3'Q_1'+Q_3Q_1$

$D_2=Q_3'Q_1'+Q_3Q_1$

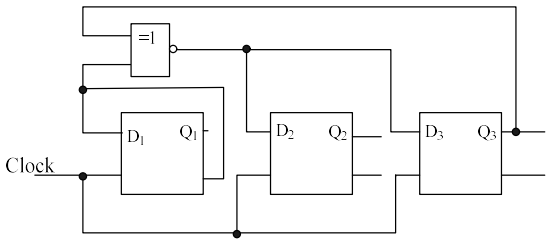
$D_1=Q_1'$

$Z= Q_3'Q_1$

$Q_3Q_2Q_1$	$Q_3+Q_2+Q_1+$
010	111
011	000
100	001
101	110



Example:



Homework

- Page326: 20, 21

CHAPTER 9
Sequential Circuits' Analysis &Registers

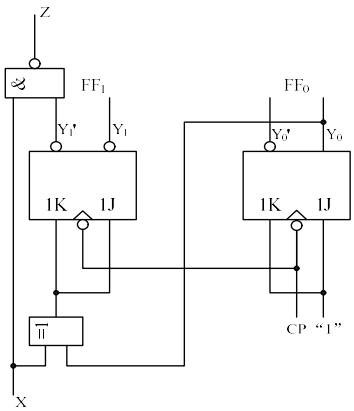
Sichuan University Software College

Synchronous Sequential Circuit Analysis

- Analysis Principles
  - 1. Determine the system variables: input, state, and output.
  - 2. Determine the flip-flop type. Write the characteristic equations.
  - 3. write the excitation equations.
  - 4. write the next state equations.
  - 5. Write the output variable equations.
  - 6. Construct a transition table.
  - 7. Assign symbols to the states and construct a table or state diagram.
  - 8. When possible, construct a timing diagram.
  - 9. Functionality analysis

Analysis Examples

- E.g.1: Analysis the following synchronous sequential circuit, suppose the present state is 00, the input sequence is 0000011111, give the timing diagram.



Analysis Examples

- 1. Determine the system variables: input, state, and output.

  input=x

  output=Z

  state variables=y_1 and y_0
- 2. Determine the flip-flop type. Write the characteristic equations.

  $y^{n+1}=Jy^{n}{}'+k'y^{n}$
- 3. write the excitation equations.

  $K_0=J_0=1$

  $K_1=J_1=x \oplus y_0$

Analysis Examples

- 4. write the next state equations.

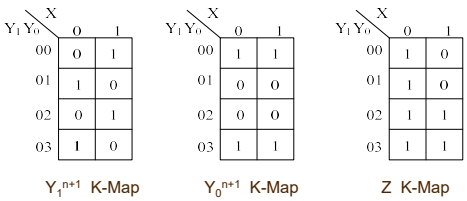
$$\left.\begin{aligned} y_1^{n+1}&=J_1y_1'+k_1'y_1 \\ y_0^{n+1}&=J_0y_0'+k_0'y_0 \end{aligned}\right\} \Longrightarrow \left\{\begin{aligned} y_1^{n+1}&=x'y_1'y_0+x'y_1y_0'+xy_1'y_0'+xy_1y_0 \\ y_0^{n+1}&=y_0' \end{aligned}\right.$$

- 5. Write the output variable equations.

  $Z=(xy_1')'=x'+y_1$
- 6. Construct a transition table.

Analysis Examples

$$\begin{cases} y_1^{n+1}= x'y_1'y_0+x'y_1y_0' \\ \quad +xy_1'y_0'+xy_1y_0 \\ y_0^{n+1}=y_0' \\ Z=(xy_1')'=x'+y_1 \end{cases}$$



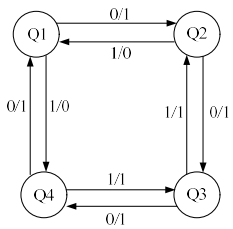
Present state Y_0 Y_1	Next state x/z	
	X=0	X=1
	Y_0^+ Y_1^+ / Z	Y_0^+ Y_1^+ / Z
00	01/1	11/0
01	10/1	00/0
11	00/1	10/1
10	11/1	01/1

transition table

Analysis Examples

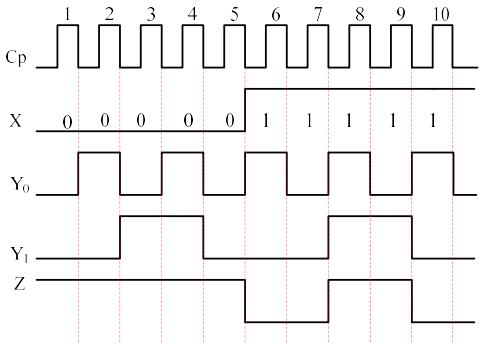
- 7. Assign symbols to the states and construct a table or state diagram.

Present state Y_0 Y_1	Next state x/z	
	X=0	X=1
	Y_0^+ Y_1^+ / Z	Y_0^+ Y_1^+ / Z
Q_1	Q_2/1	Q_4/0
Q_2	Q_3/1	Q_1/0
Q_3	Q_4/1	Q_2/1
Q_4	Q_1/1	Q_3/1



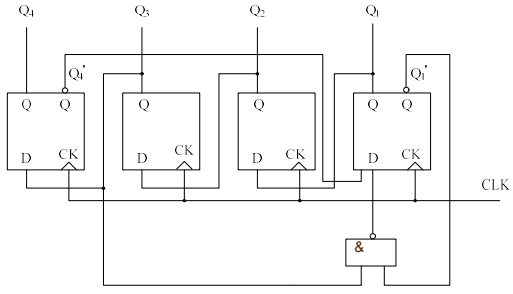
Analysis Examples

- 8. construct a timing diagram.



Analysis Examples

- E.g.2: Analysis the following circuit



Analysis Examples

- 1. Determine the system variables: input, state, and output.

  state variables: Q_1,Q_2,Q_3, and Q_4
- 2. Determine the flip-flop type. Write the characteristic equations.

  $Q^{n+1}=D$
- 3. excitation equations.

  $D_4=Q_3$

  $D_3=Q_2$

  $D_2=Q_1$

  $D_1=Q_4'(Q_3Q_1')'=Q_4'Q_3'+Q_4'Q_1$

Analysis Examples

- 4. the next state equations.

  $Q_4^{n+1}=Q_3$

  $Q_3^{n+1}=Q_2$

  $Q_2^{n+1}=Q_1$

  $Q_1^{n+1}=Q_4'Q_3'+Q_4'Q_1$

Analysis Examples

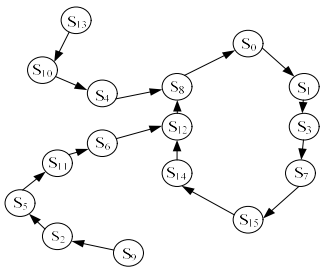
- 5. Construct a transition table.

Q_4	Q_3	Q_2	Q_1	Q_4^{n+1}	Q_3^{n+1}	Q_2^{n+1}	Q_1^{n+1}
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	1
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	1
0	1	0	0	1	0	0	0
0	1	0	1	1	0	1	1
0	1	1	0	1	1	0	0
0	1	1	1	1	1	1	1
1	0	0	0	0	0	0	0
1	0	0	1	0	0	1	0
1	0	1	0	0	1	0	0
1	0	1	1	0	1	1	0
1	1	0	0	1	0	0	0
1	1	0	1	1	0	1	0
1	1	1	0	1	1	0	0
1	1	1	1	1	1	1	0

Analysis Examples

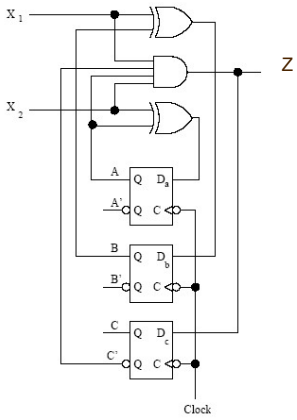
- 6. Assign symbols to the states and construct a table or state diagram.

Q_4 Q_3 Q_2 Q_1	S_0	S_1	S_2	S_3	S_4	S_5	S_6	S_7	S_8	S_9	S_{10}	S_{11}	S_{12}	S_{13}	S_{14}	S_{15}
$Q_4^{n+1}Q_3^{n+1}Q_2^{n+1}Q_1^{n+1}$	S_1	S_3	S_5	S_7	S_8	S_{11}	S_{12}	S_{15}	S_0	S_2	S_4	S_6	S_8	S_{10}	S_{12}	S_{14}



Analysis Examples

- E.g.3: Find the transition table and the state table for the Mealy sequential circuit below.



Analysis Examples

- 1. Determine the system variables: input, state, and output.

  input: x_1,x_2

  output: Z

  state variable: D_a , D_b ,D_c
- 2. Determine the flip-flop type. Write the characteristic equations.

  $Q_{n+1}=D$
- 3. Excitation equations and output equations.

  $D_a=Q_a \oplus X_2$

  $D_b=Q_b \oplus X_1$

  $D_c=X_1X_2Q_c'Q_a$

  $Z=X_1X_2Q_c'Q_a$

Analysis Examples

- 4. write the next state equations.
  $Q_a^{n+1}=Q_aX_2'+Q_a'X_2$
  $Q_b^{n+1}=Q_bX_1'+Q_b'X_1$
  $Q_c^{n+1}=X_1X_2Q_c'Q_a$
- 5. Construct a transition table.

$Q_aQ_bQ_c$	$Q_a^+Q_b^+Q_c^+/Z$			
	X=00	X=01	X=10	X=11
000	000/0	100/0	010/0	110/0
001	000/0	100/0	010/0	110/0
010	010/0	110/0	000/0	100/0
011	010/0	110/0	000/0	100/0
100	100/0	000/0	110/0	011/1
101	100/0	000/0	110/0	010/0
110	110/0	010/0	100/0	001/1
111	110/0	010/0	100/0	000/0

Analysis Examples

- 6. Assign symbols to the states and construct a state table

$Q_aQ_bQ_c$	$Q_a^+Q_b^+Q_c^+/Z$			
	X=00	X=01	X=10	X=11
A	A/0	E/0	C/0	G/0
B	A/0	E/0	C/0	G/0
C	C/0	G/0	A/0	E/0
D	C/0	G/0	A/0	E/0
E	E/0	A/0	G/0	D/1
F	E/0	A/0	G/0	C/0
G	G/0	C/0	E/0	B/1
H	G/0	C/0	E/0	A/0

Registers

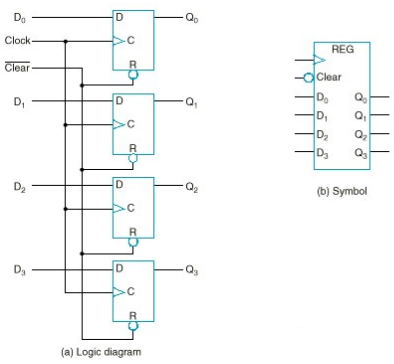
- A *n*-bit *register* is a set of *n* flip-flops that is capable of storing *n* bits of binary information.
- With added combinational gates, the register can perform data-processing tasks.

Register Overview

- Parallel Load Register
- Shift Registers
  - Serial Load
  - Serial Addition
- Shift Register with Parallel Load
- Bidirectional Shift Register

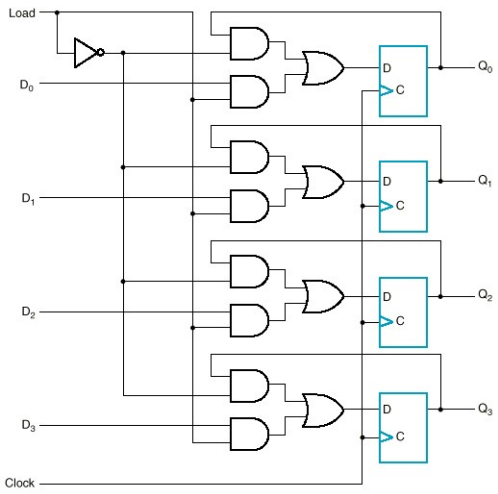
Registers

- Example (next slide) generic 4-bit register.
- The common *Clock* input triggers all flip-flops on the rising edge of each pulse, and the binary data available at the four D inputs are transferred into the 4-bit register.



Registers with parallel load

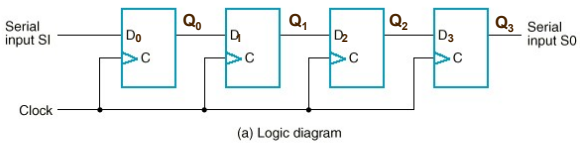
- Next page shows a 4-bit register with a control input *Load* that is directed through gates into the D inputs of the flip-flops.
- When *Load* is 1, the data on the four inputs is transferred into the register with the next positive transition of a clock pulse.
- When *Load* is 0, the data inputs are blocked, and the D inputs of the flip-flops are connected to their outputs.



Shift registers

- A register capable of shifting its stored bits laterally in one or both directions is called a *shift register*.
- The logical configuration of a shift register consists of a chain of flip-flops in cascade, with the output of one flip-flop connected to the input of the next FF.

Shift Register

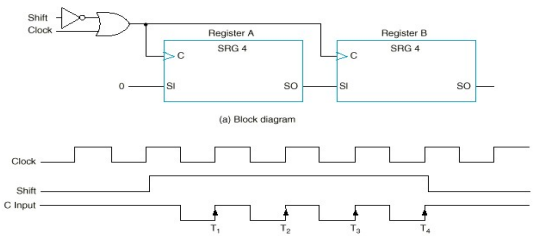


(a) Logic diagram

(b) Symbol

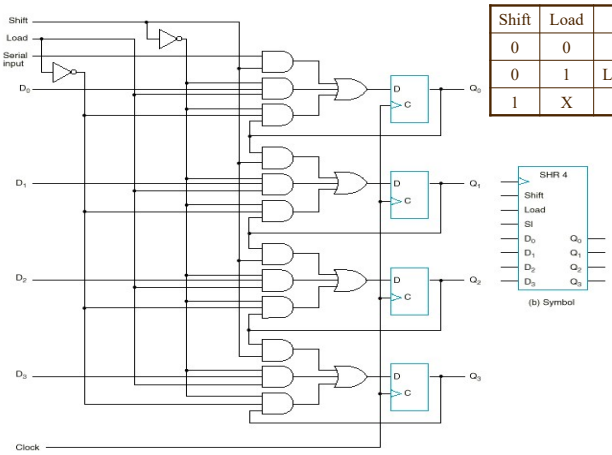
Shift registers (cont.)

- The next figure shows how the serial transfer of information from register A to register B can be done. One clock cycle per bit of data is required.



(a) Block diagram

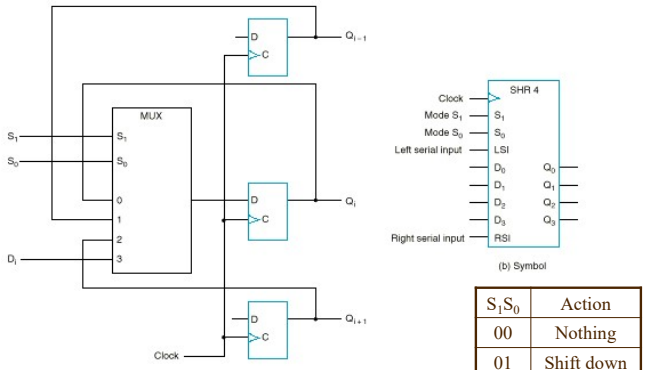
Shift register with parallel load



(b) Symbol

Shift	Load	Operation
0	0	Nothing
0	1	Load parallel
1	X	Shift

Bidirectional Shift Register



S_1S_0	Action
00	Nothing
01	Shift down
10	Shift up
11	Parallel load

Serial addition using shift registers

- The two binary numbers to be added serially are stored in two shift registers.
- Bits are added one pair at a time through a **single** full-adder circuit.
- The **carry out** of the full adder is transferred into a D flip-flop. The output of the carry FF is then used as the carry input for the next pair of bits.
- The sum bit on the S output of the full adder is transferred into the result register A.

Serial *vs.* parallel addition

- The parallel adder is a combinational circuit, whereas the serial adder is a sequential circuit.
- The parallel adder has *n* full adders for *n*-bit operands, whereas the serial adder requires only one full adder.
- The serial circuit takes *n* clock cycles to complete an addition.
- In summary, the parallel adder in space is n times larger than the serial adder, but it is *n* times faster.
- The serial adder, although it is *n* times slower, is *n* times smaller in space.(It's a trade-off!)

homework